

CSE 315

Microprocessors, Microcontrollers, and
Embedded Systems

Assembly Language: Stack and Procedures

Stack Segment

- A block of memory to store stack
- Syntax
 - .STACK** size
 - Where size is optional and specifies the stack area size in bytes
 - If size is omitted, 1 KB set aside for stack area
- For example:
.STACK 100h
- SS contains segment number of stack segment
- SP contains offset address of the top of the stack

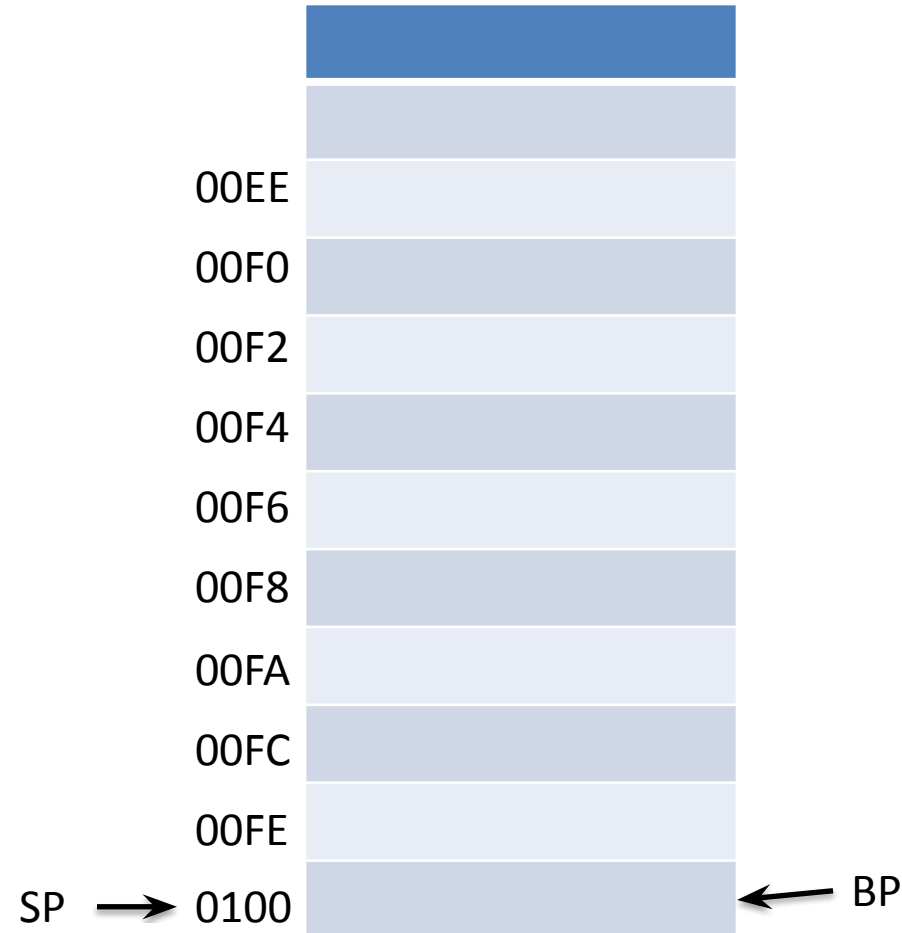
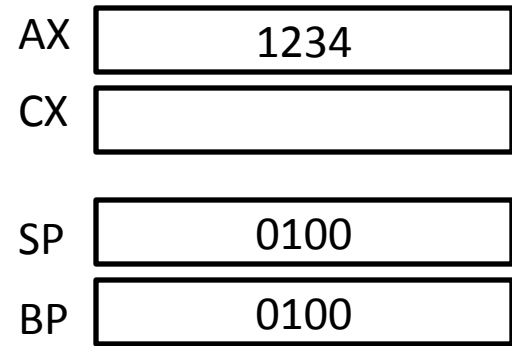
PUSH

PUSH source

- source is a 16-bit register / memory word

PUSH AX

1. SP is decreased by 2
2. Source content is copied to the address SS:SP



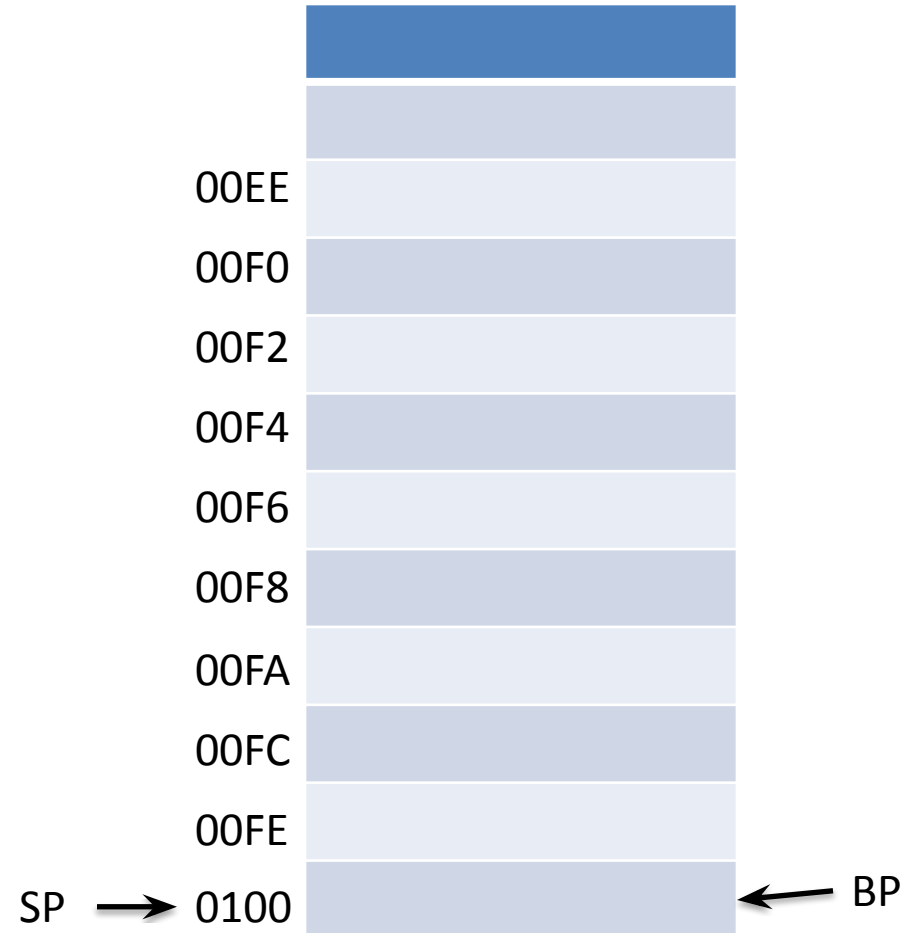
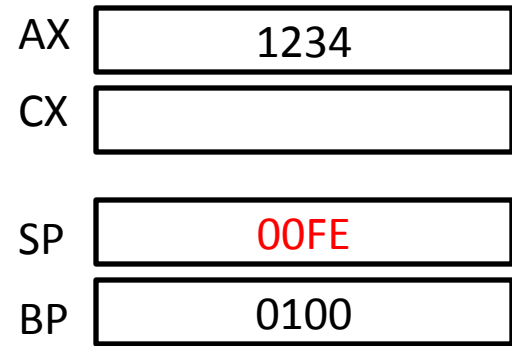
PUSH

PUSH source

- source is a 16-bit register / memory word

PUSH AX

1. SP is decreased by 2
2. Source content is copied to the address SS:SP



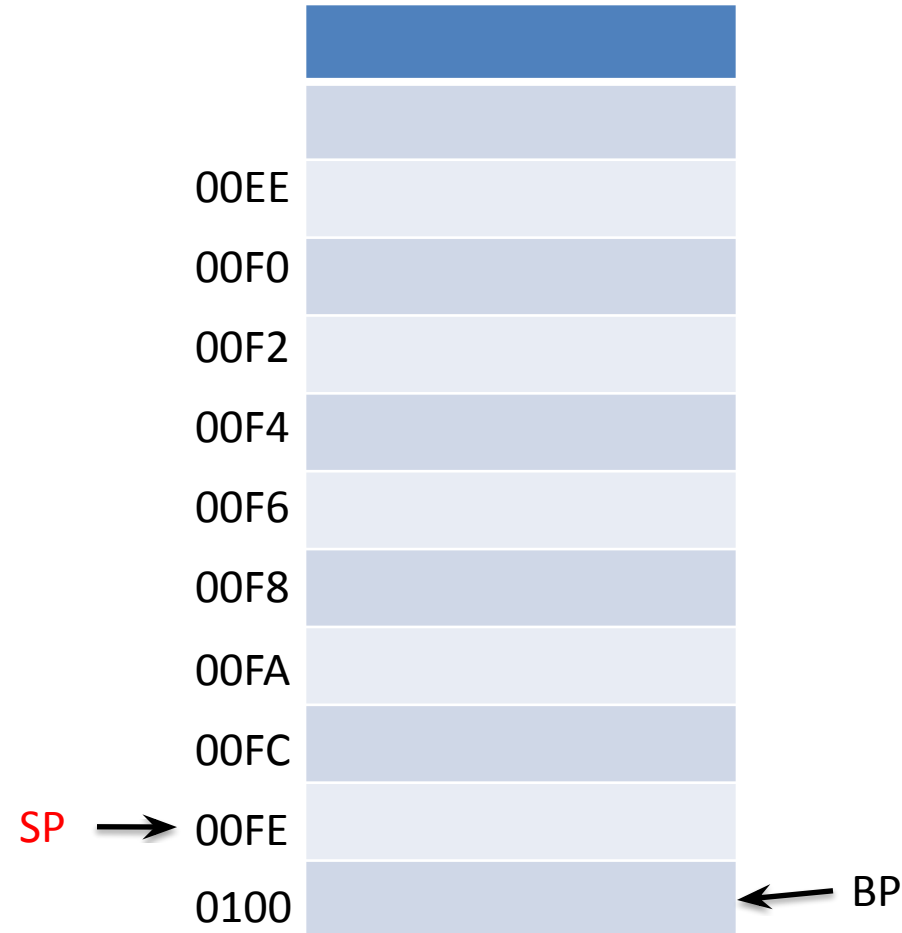
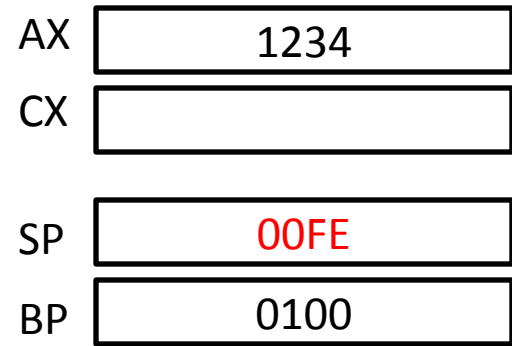
PUSH

PUSH source

- source is a 16-bit register / memory word

PUSH AX

1. SP is decreased by 2
2. Source content is copied to the address SS:SP



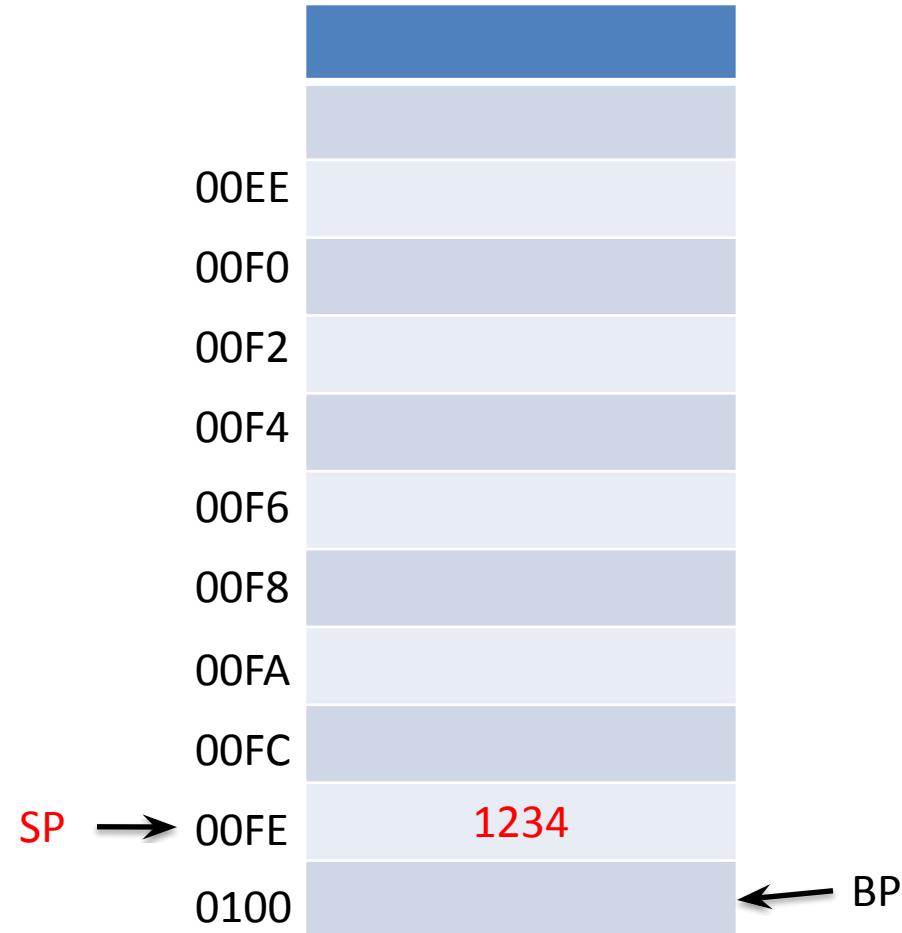
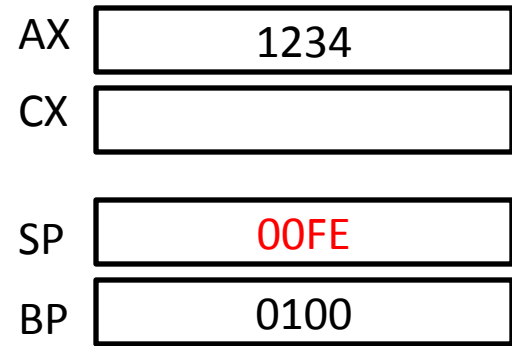
PUSH

PUSH source

- source is a 16-bit register / memory word

PUSH AX

1. SP is decreased by 2
2. Source content is copied to the address SS:SP



POP

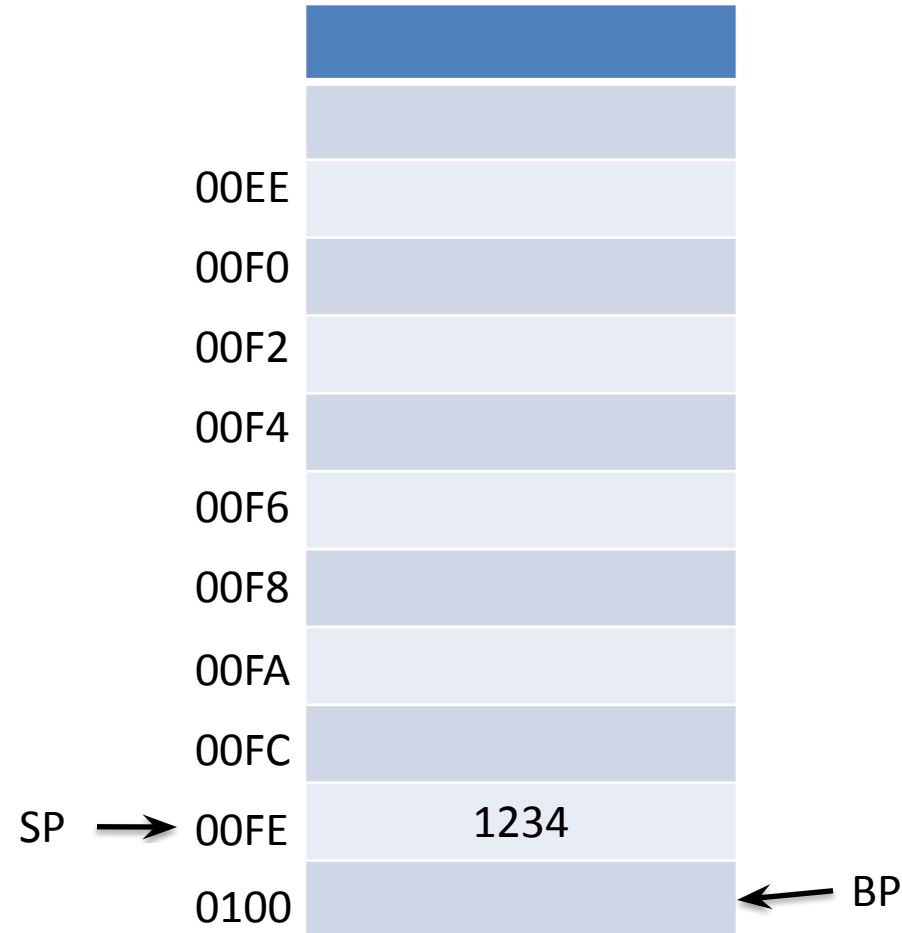
POP destination

- destination is a 16-bit register(except IP register) / memory word

POP CX

1. The content of SS:SP is moved to the destination
2. SP is increased by 2

AX	1234
CX	
SP	00FE
BP	0100



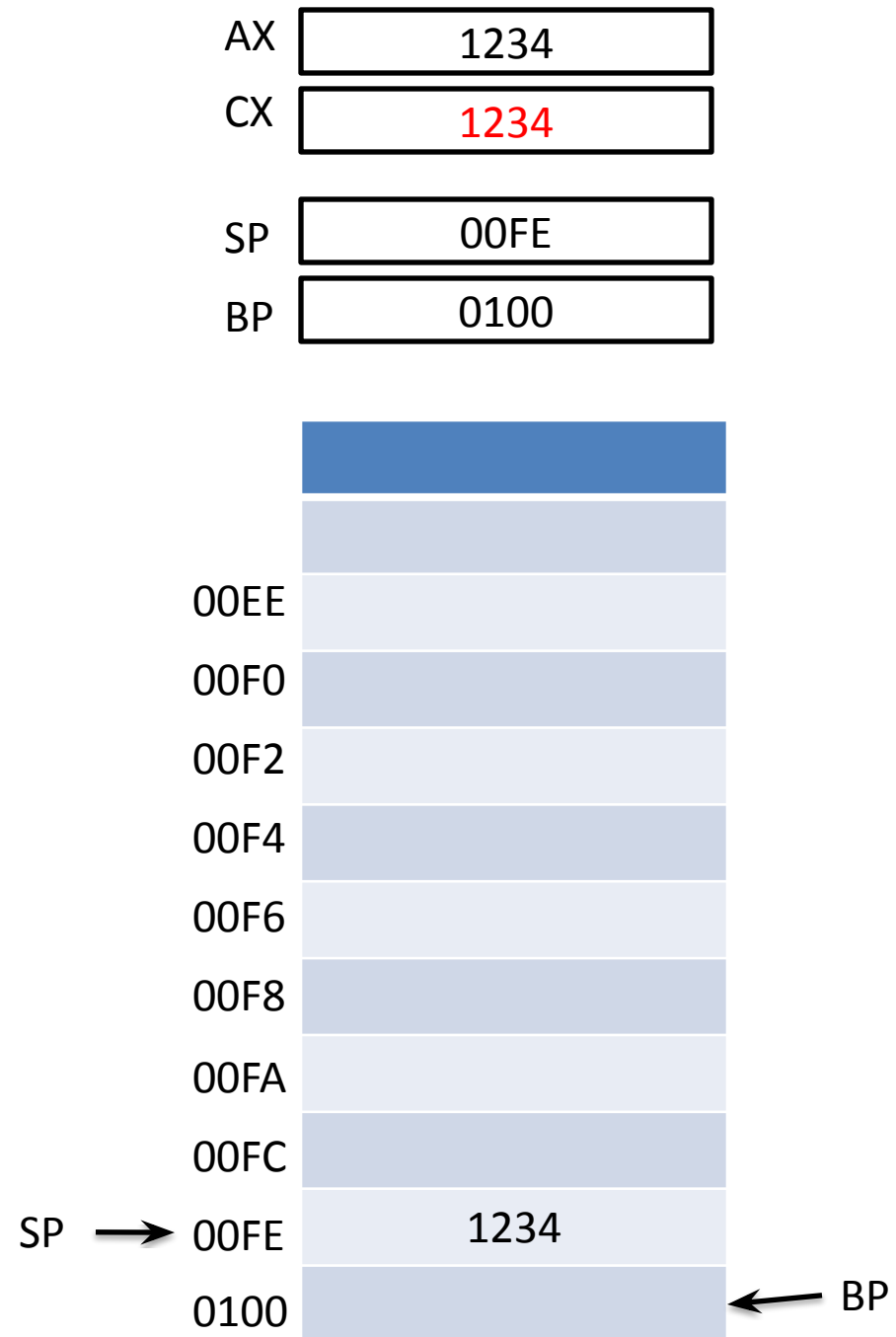
POP

POP destination

- destination is a 16-bit register(except IP register) / memory word

POP CX

1. The content of SS:SP is moved to the destination
2. SP is increased by 2



POP

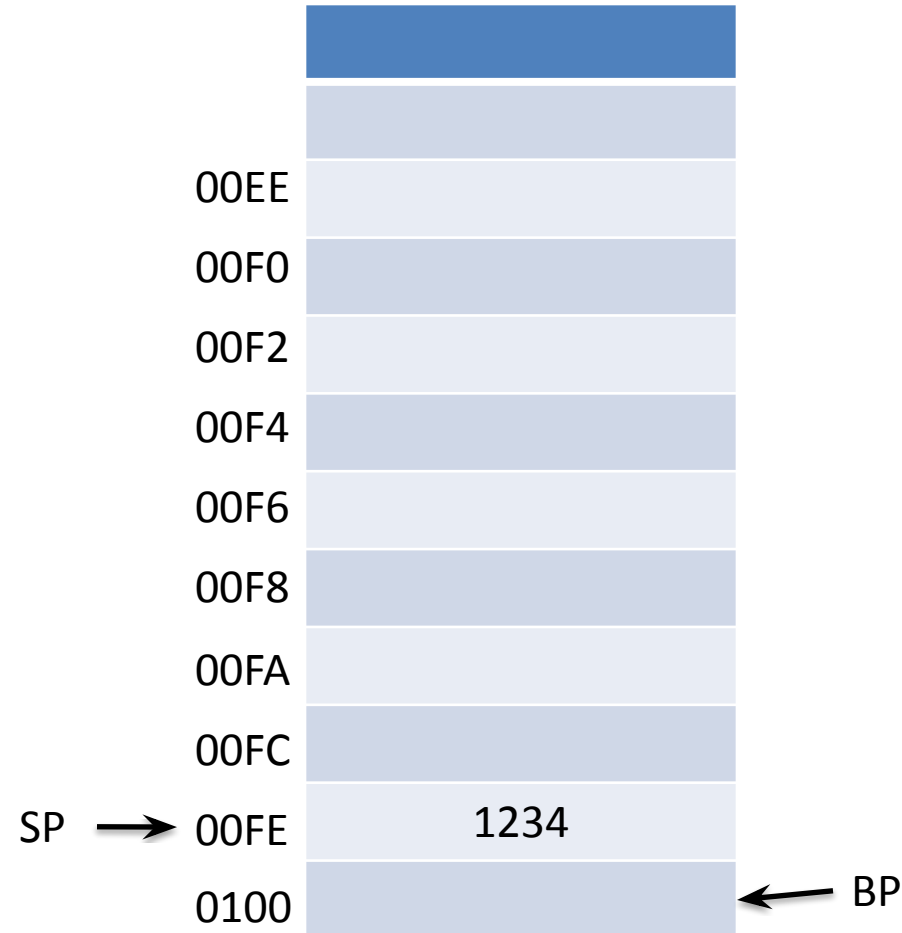
POP destination

- destination is a 16-bit register(except IP register) / memory word

POP CX

1. The content of SS:SP is moved to the destination
2. SP is increased by 2

AX	1234
CX	1234
SP	0100
BP	0100



POP

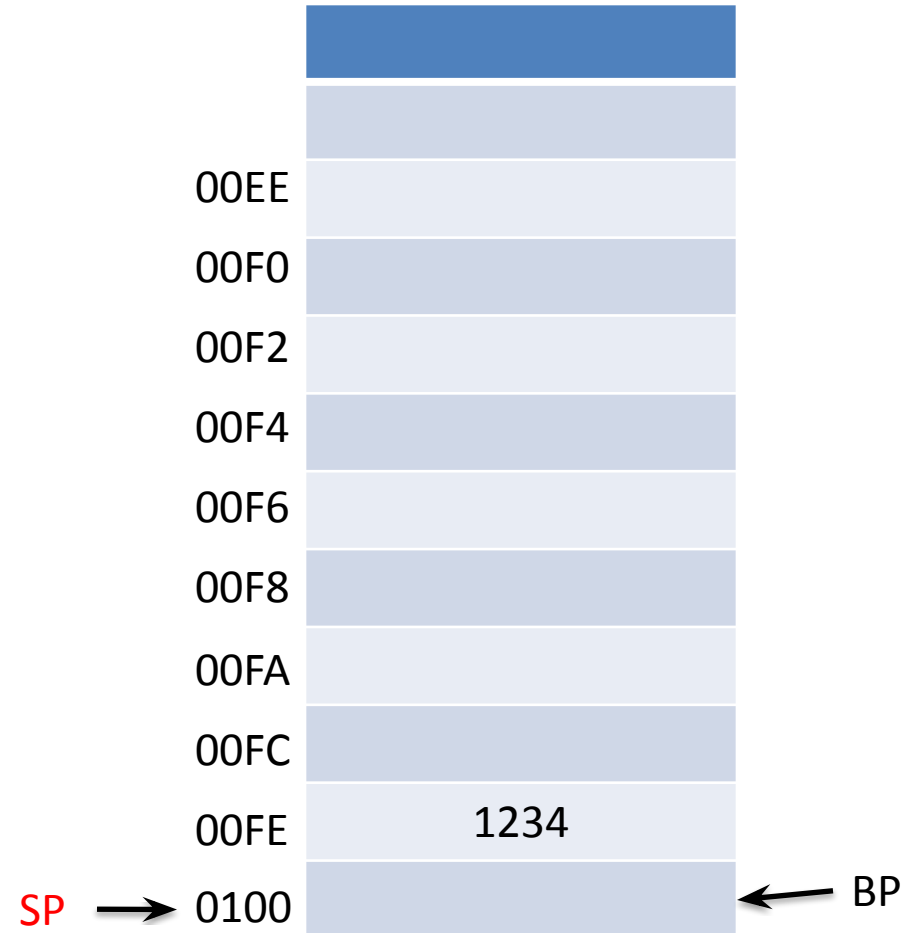
POP destination

- destination is a 16-bit register(except IP register) / memory word

POP CX

1. The content of SS:SP is moved to the destination
2. SP is increased by 2

AX	1234
CX	1234
SP	0100
BP	0100



PUSHF and POPF

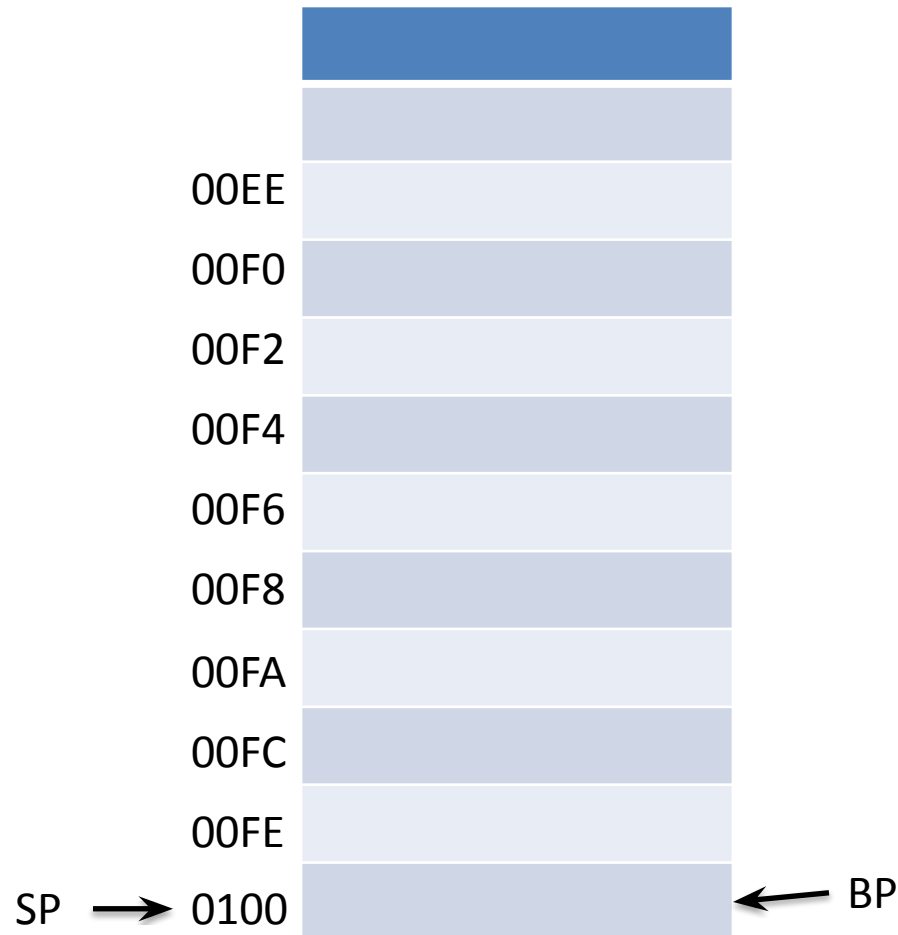
PUSHF

- No source operand is allowed here
- Pushes the contents of FLAGS register onto the stack

POPF

- Reverse of PUSHF
- Pops the top of the stack into FLAGS register

AX	1234
CX	
SP	0100
BP	0100



Order of PUSH and POP

PUSH A

PUSH B

PUSH C

...

...

POP C

POP B

POP A

Procedure Declaration

```
name    PROC    type  
; body of the procedure  
    RET  
name    ENDP
```

“name” is the user-defined name of the procedure

“type” is optional, can be “FAR”/“NEAR”

“RET” causes control back to the calling procedure

```
name    PROC    type  
; body of the procedure  
      RET  
name    ENDP
```

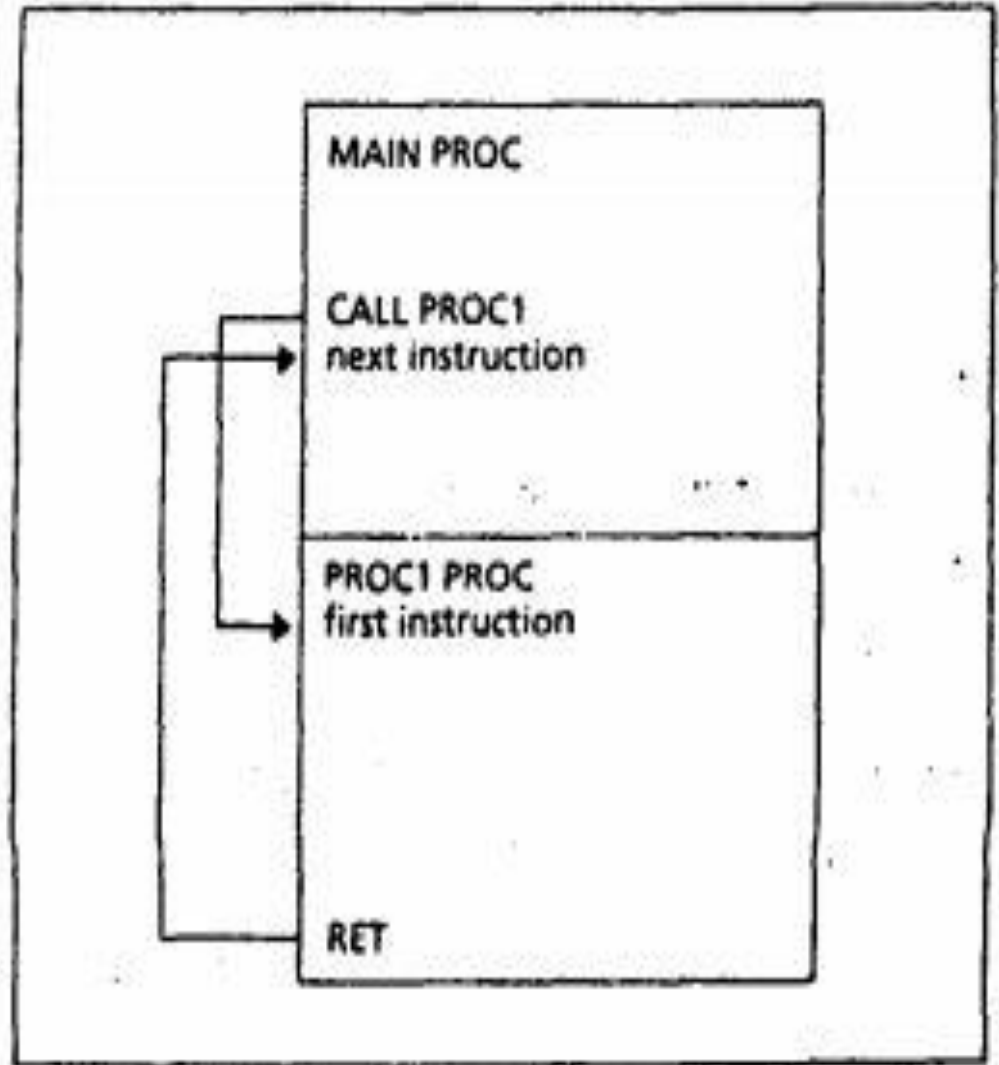


Fig: Procedure CALL and RET

CALL

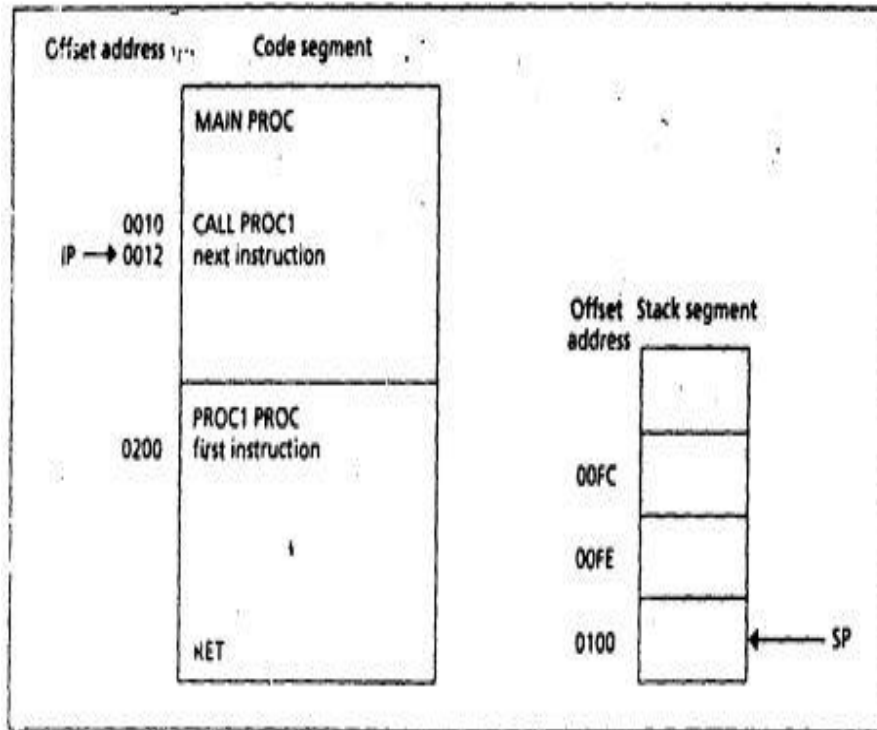


Figure 8.4A Before CALL

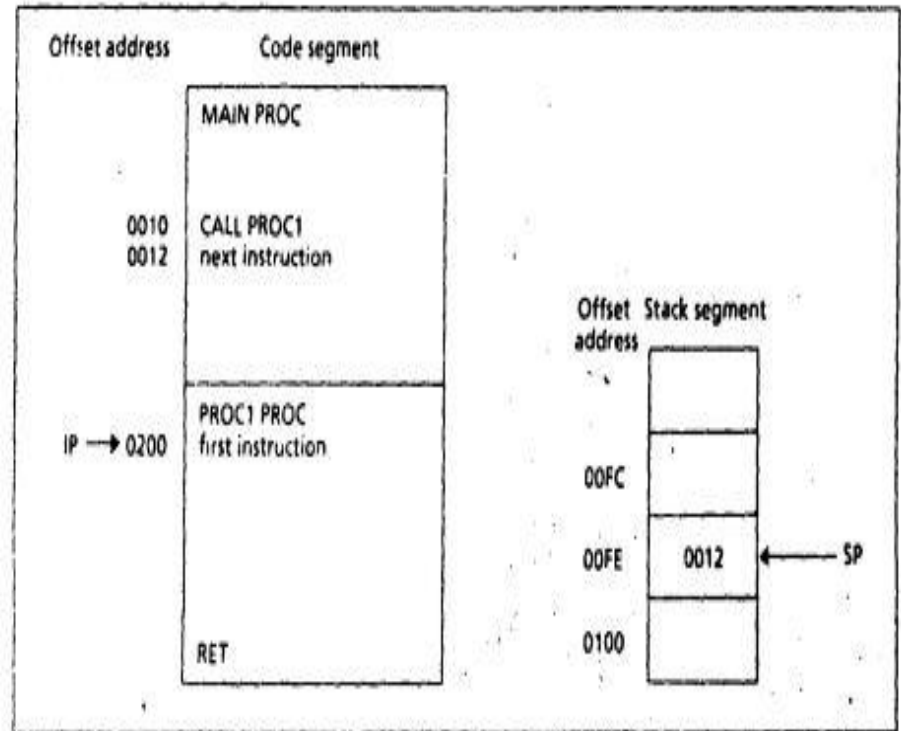


Figure 8.4B After CALL

RET

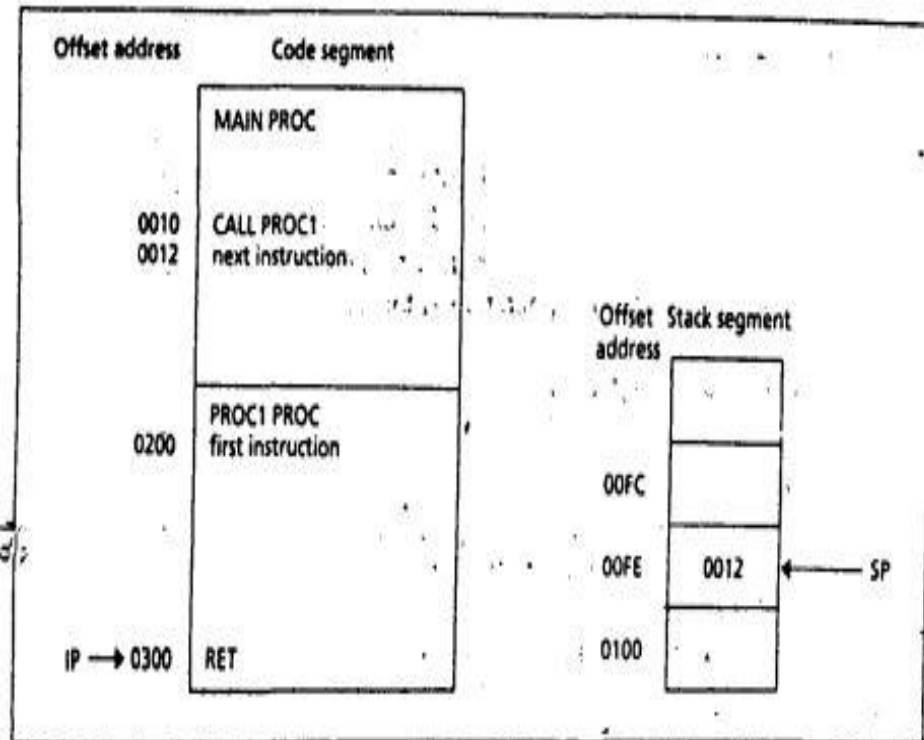


Figure 8.5A Before RET

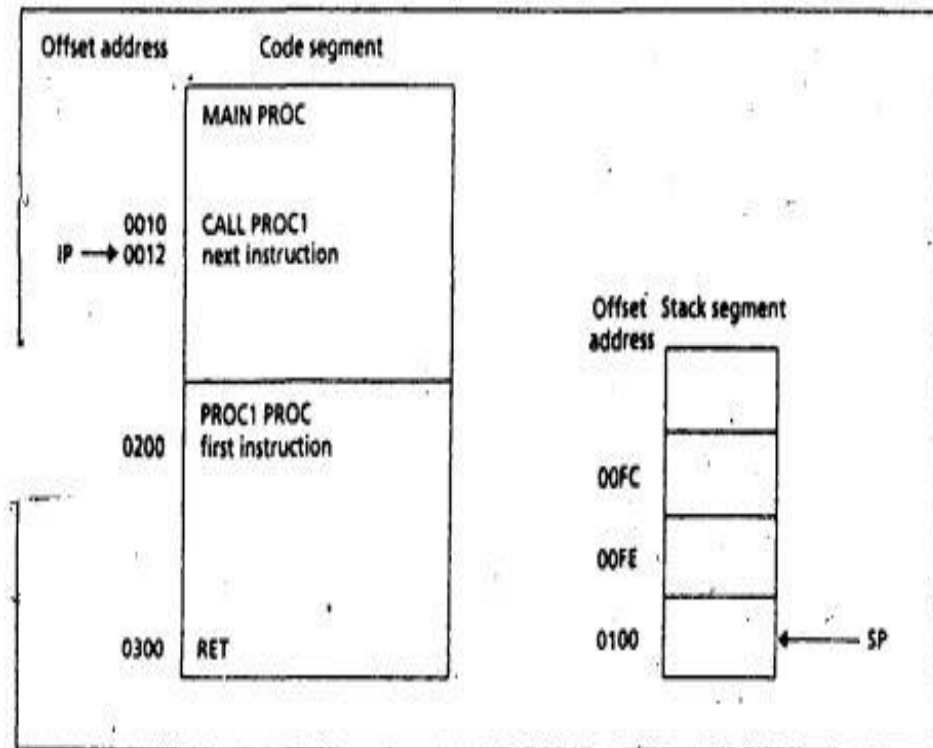


Figure 8.5B After RET

Example of a procedure

Unsigned multiplication of two positive integers A and B by addition and bit shifting

```
Product = 0
REPEAT
  IF lsb of B is 1
  THEN
    Product = Product + A
  END_IF
  Shift left A
  Shift right B
UNTIL B = 0
```

```
  111b
× 1101b
-----
  111
 000
 111
 111
-----
1011011b
```

```
Product = 0
Since lsb of B is 1, Product = 0 + 111b = 111b
Shift left A: A = 1110b
Shift right B: B = 110b
```

```
Since lsb of B is 0,
Shift left A: A = 11100b
Shift right B: B = 11b
```

```
Since lsb of B is 1
Product = 111b + 11100b = 100011b
Shift left A: A = 111000b
Shift right B: B = 1
```

```
Since lsb of B is 1
Product = 100011b + 111000b = 1011011b
Shift left A: A = 1110000b
Shift right B: B = 0
```

```
Since lsb of B = 0
Return Product = 1011011b = 91d
```

Example of a procedure

Unsigned multiplication of two positive integers A and B by addition and bit shifting

```
Product = 0
REPEAT
  IF lsb of B is 1
  THEN
    Product= Product+A
  END_IF
  Shift left A
  Shift left B
UNTIL B=0
```

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC

  MOV     AX, @DATA
  MOV     DS, AX
  MOV     AX, 10
  MOV     BX, 6
  CALL    MULTIPLY

  ...
  MOV     AH, 4CH
  INT     21H
MAIN     ENDP

MULTIPLY PROC
  PUSH    AX
  PUSH    BX
  XOR     DX, DX
```

```
REPEAT:
  TEST     BX, 1
  JZ       END_IF

  ADD     DX, AX
END_IF:
  SHL     AX, 1
  SHR     BX, 1
  JNZ     REPEAT

  POP     BX
  POP     AX
  RET

MULTIPLY ENDP
END MAIN
```

Chapter 17

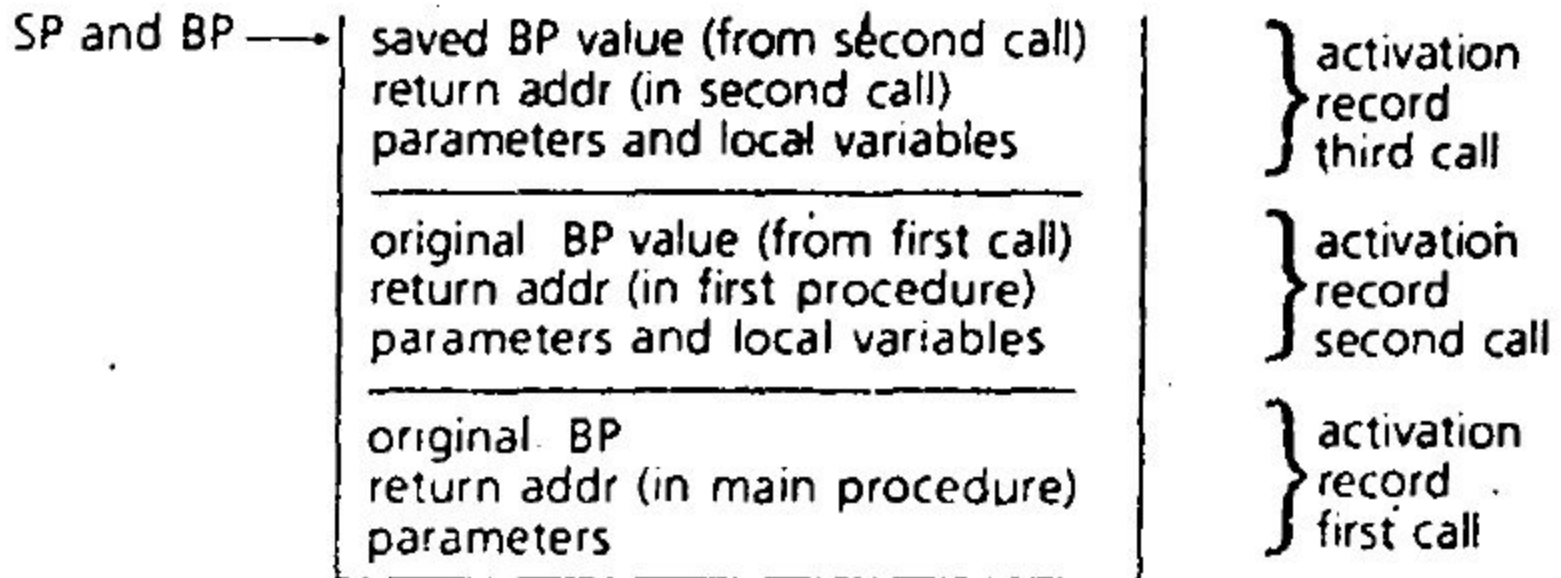
Recursion

Parameter passing

- We can push the parameters into the stack
- Use BP to access the parameters
 - BP is saved in the stack for later
- RET n
 - Return and pop n additional bytes

Activation Record

- Record to be stored before CALLing a recursive function



Factorial

- $\text{Factorial}(n) = n! = n * (n-1) * (n-2) * \dots * 3 * 2 * 1$

$$\text{Factorial}(n) = n * \text{Factorial}(n-1)$$

```

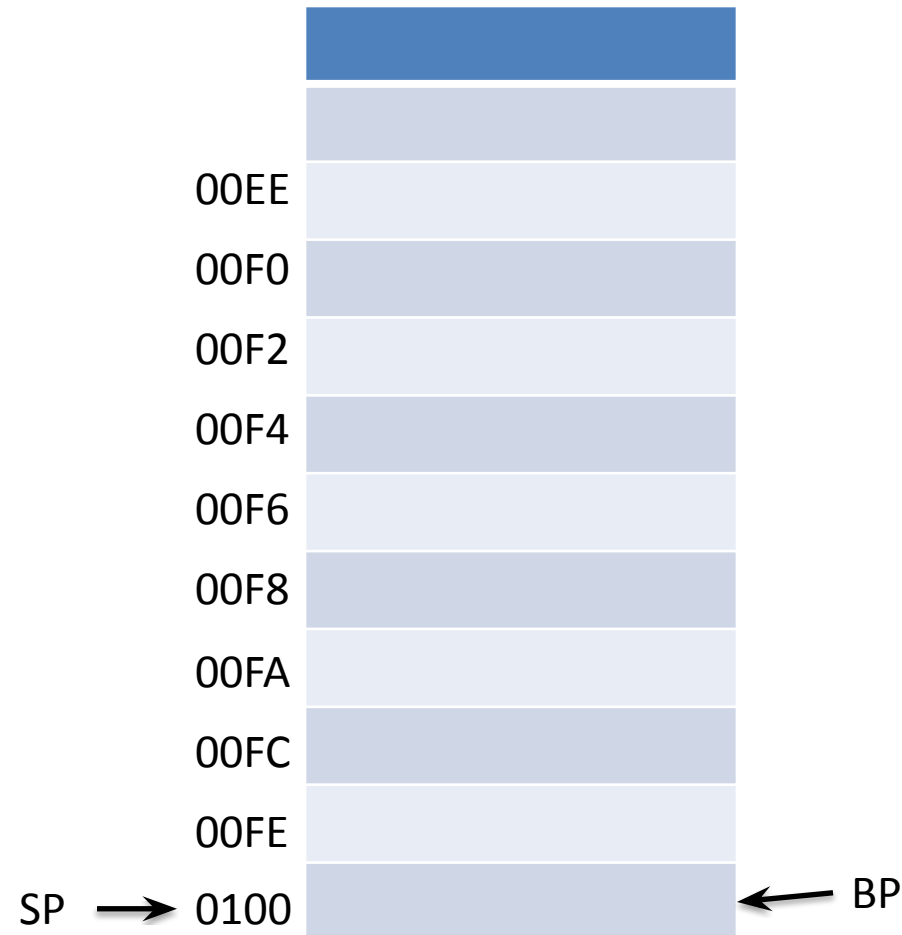
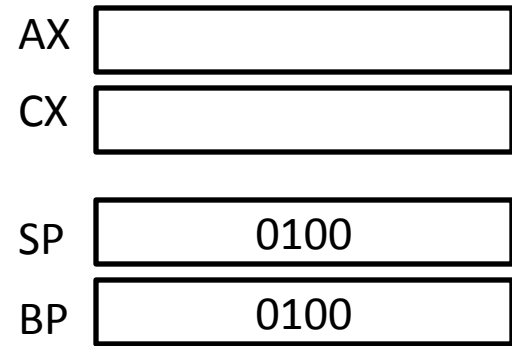
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN

```



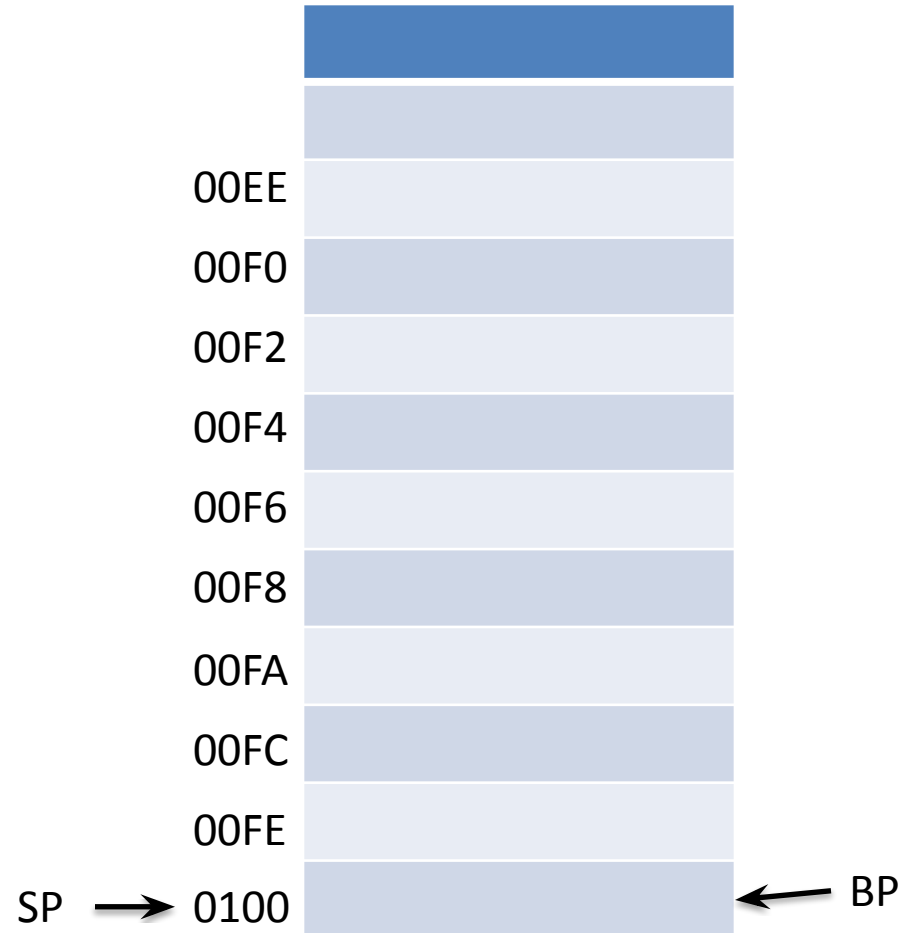
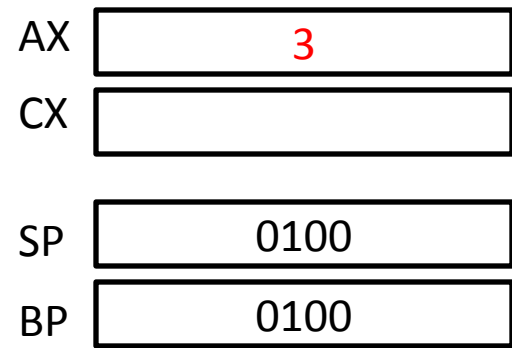
```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN

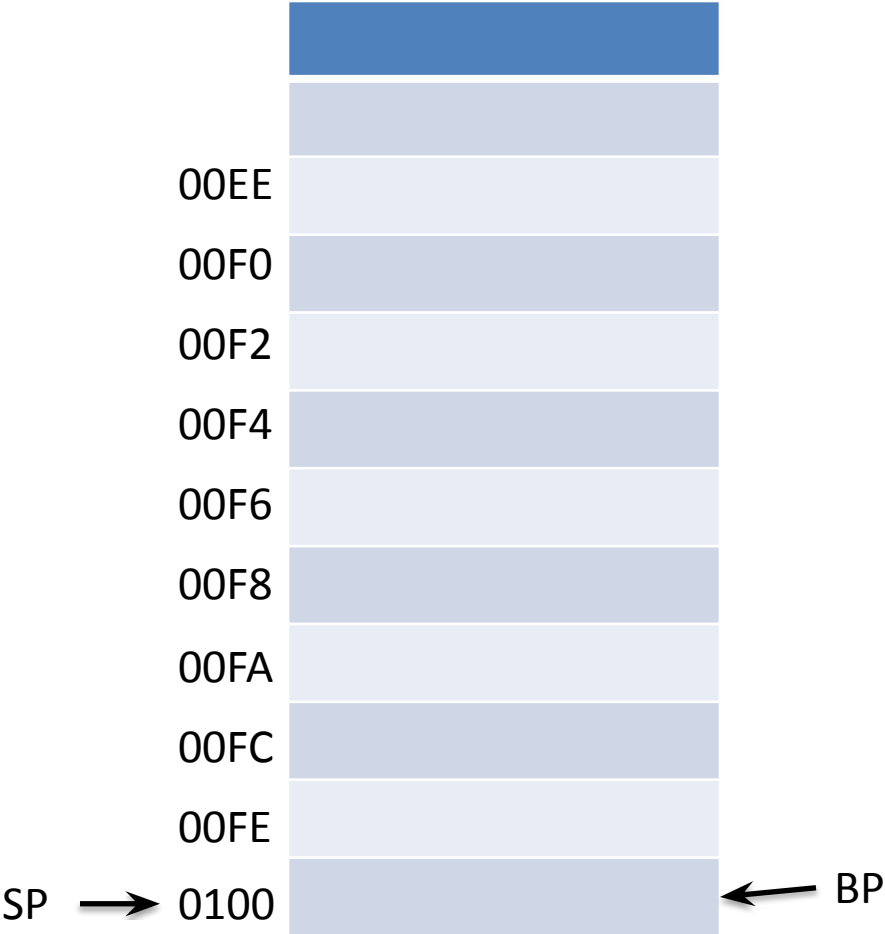
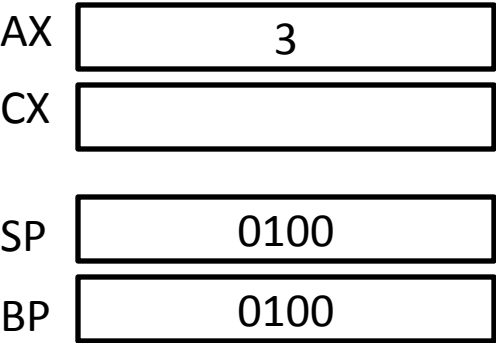
```




```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

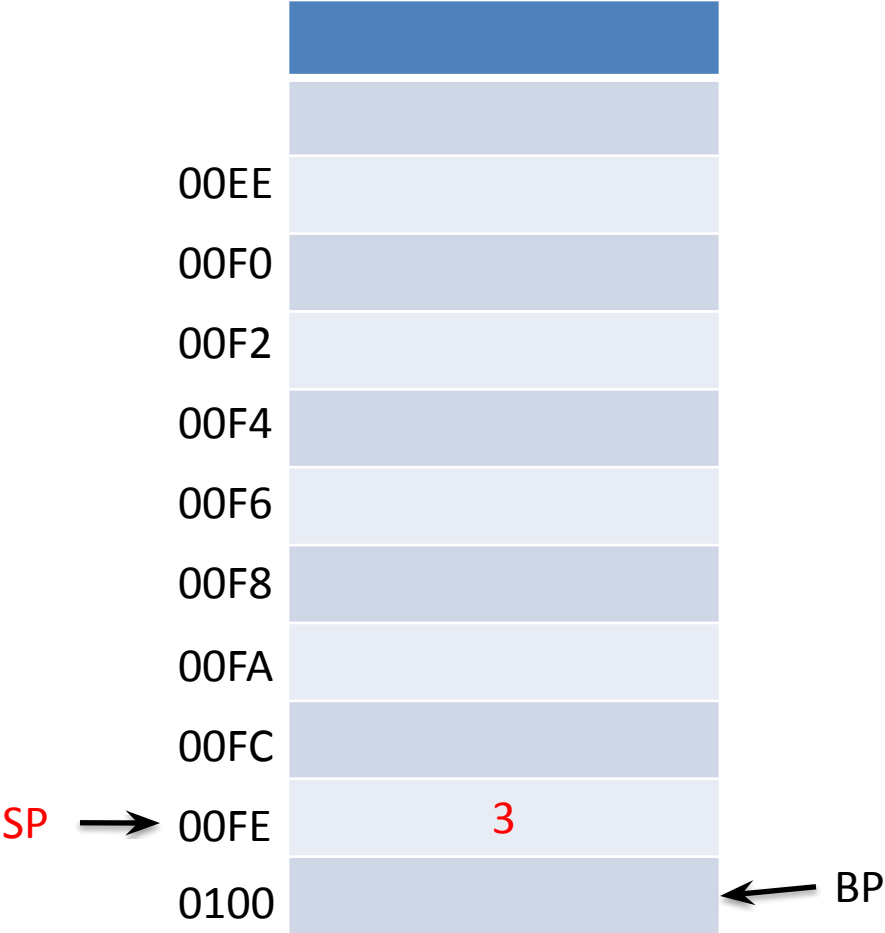
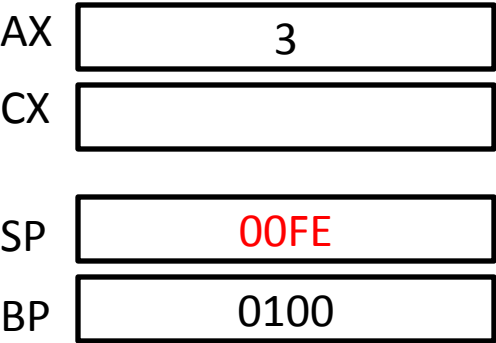
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

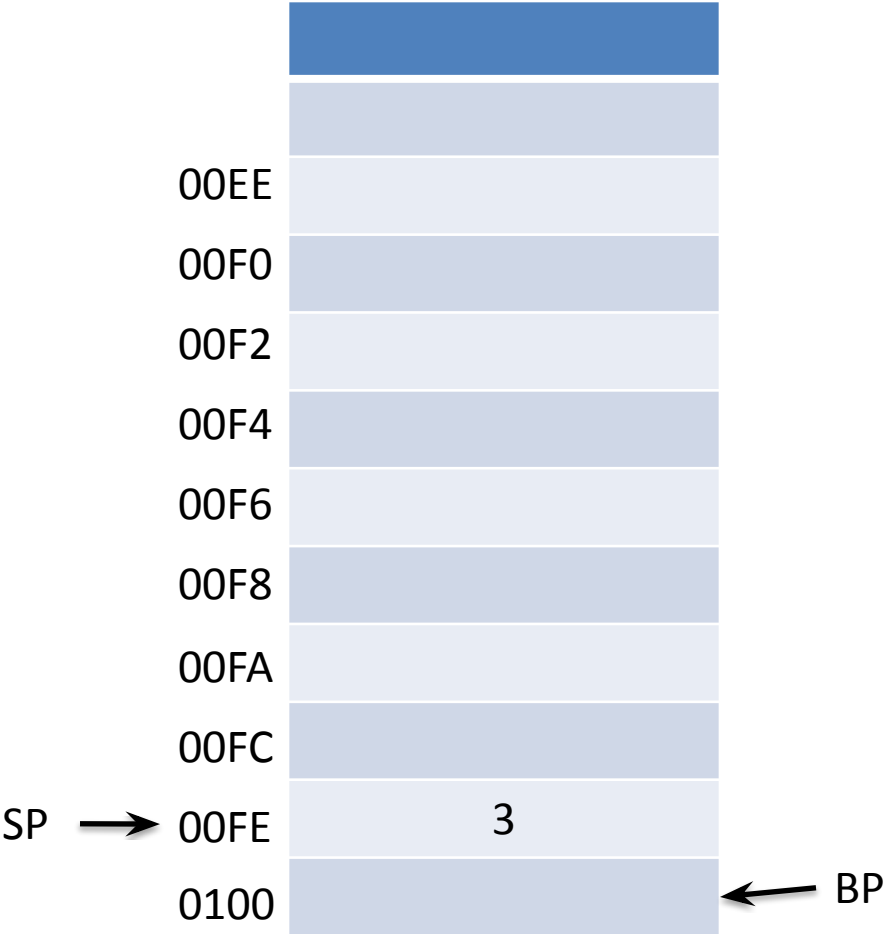
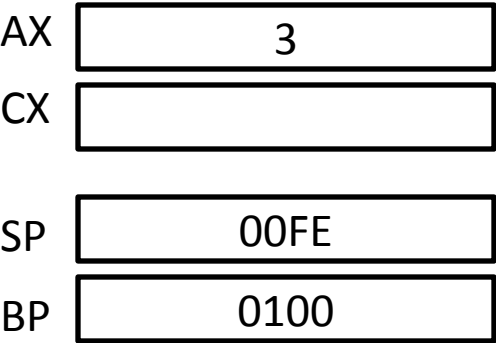
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

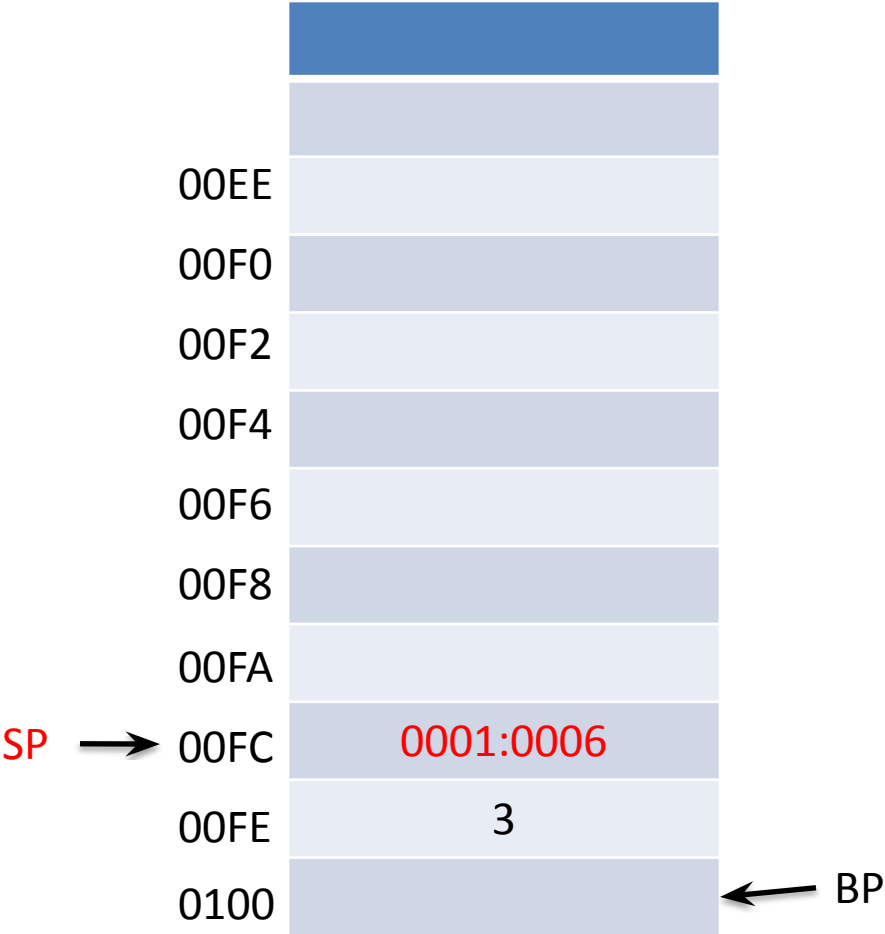
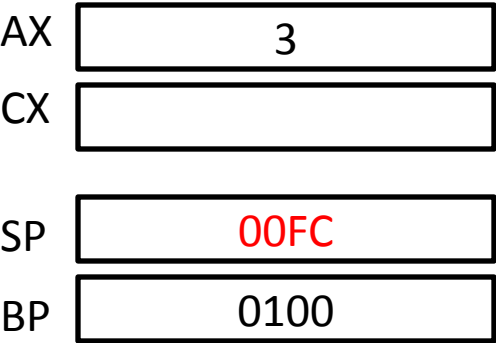
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

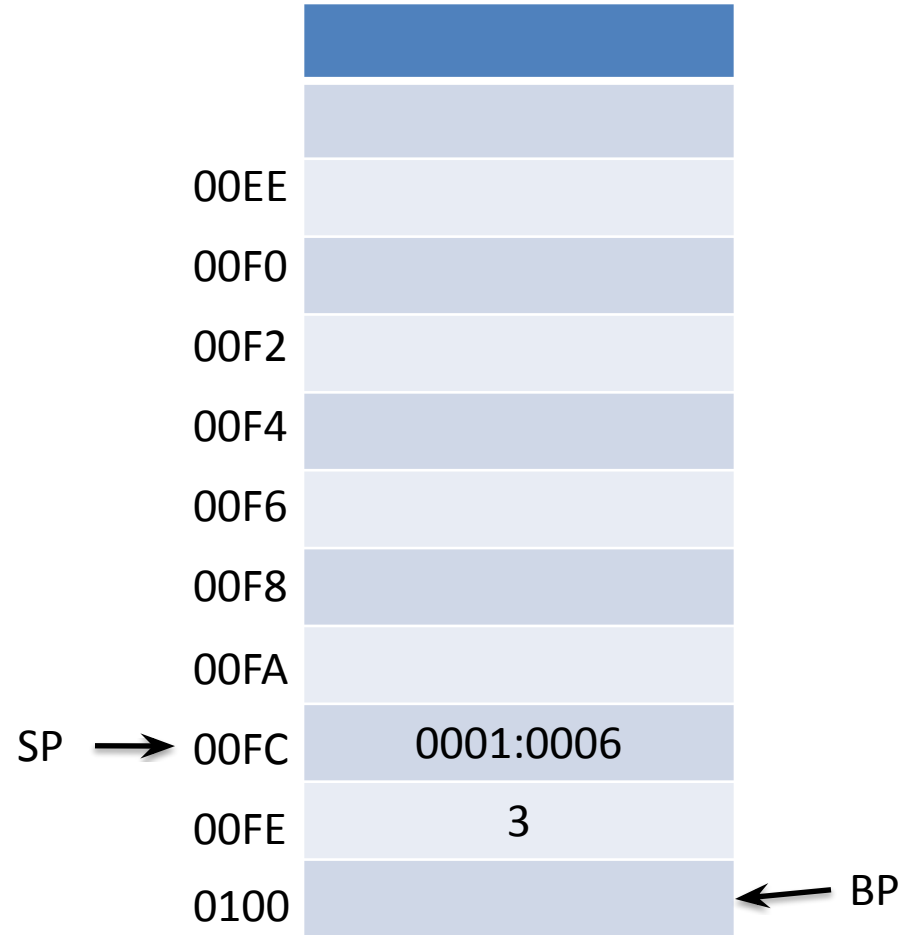
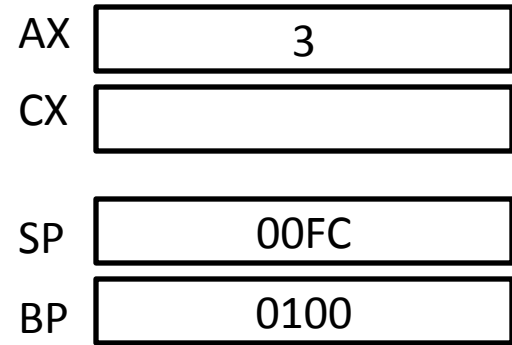
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



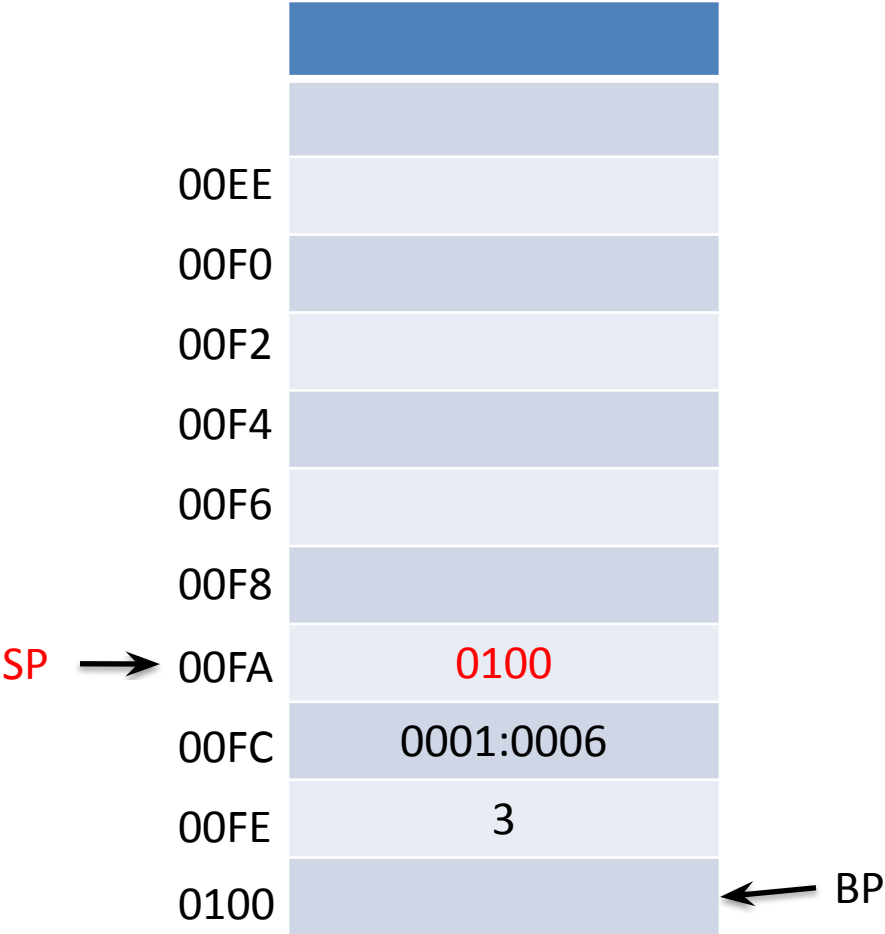
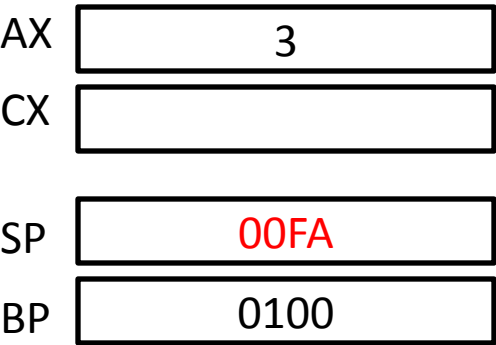
```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
```

MAIN ENDP

FACTORIAL PROC NEAR

```
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
```

END MAIN



```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN

```

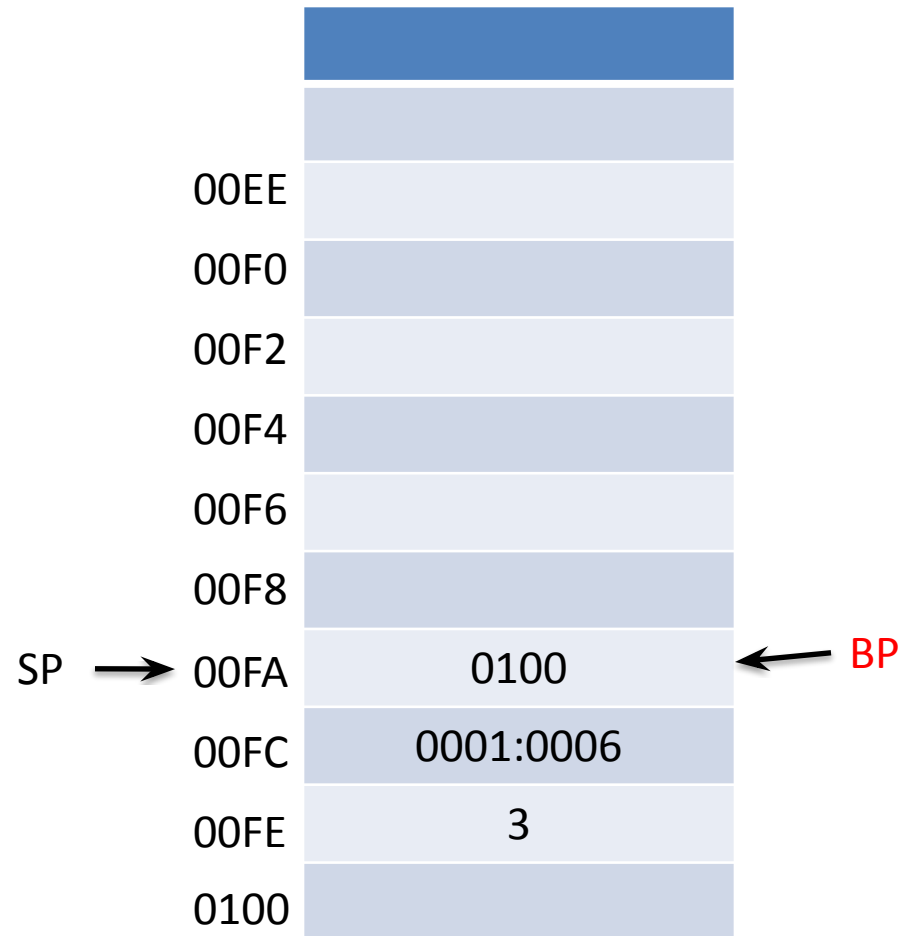
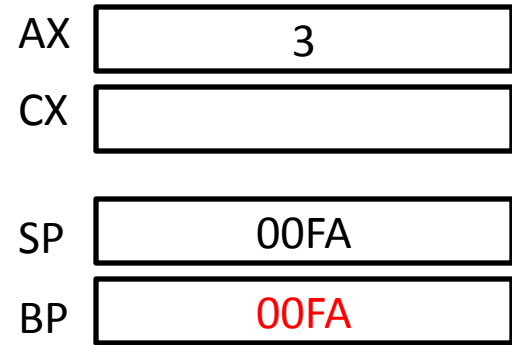
AX	3
CX	
SP	00FA
BP	0100

00EE		
00F0		
00F2		
00F4		
00F6		
00F8		
SP → 00FA	0100	
00FC	0001:0006	
00FE	3	
0100		← BP

```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```




```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN

```

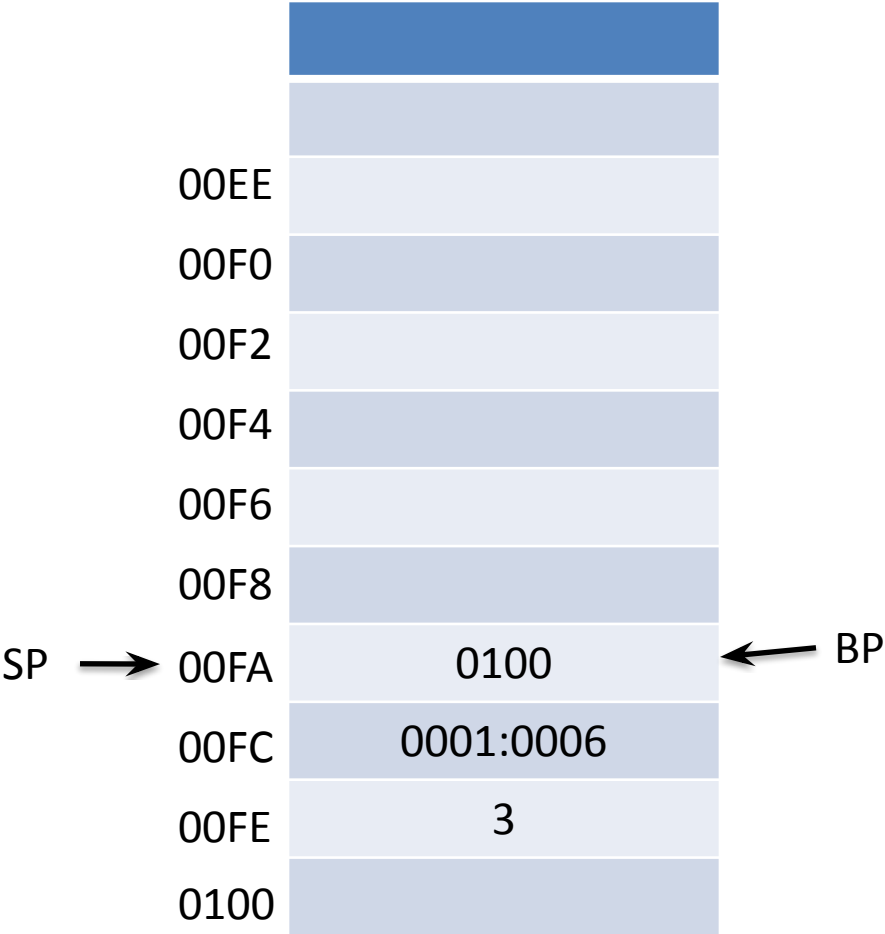
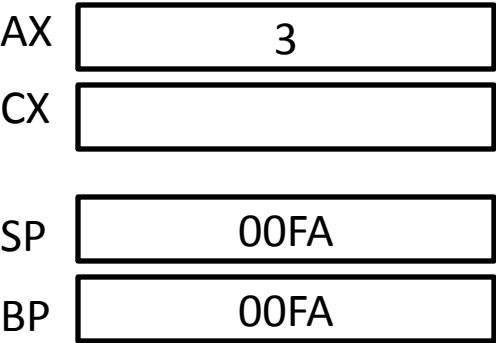
AX	3
CX	
SP	00FA
BP	00FA

00EE		
00F0		
00F2		
00F4		
00F6		
00F8		
SP → 00FA	0100	← BP
00FC	0001:0006	
00FE	3	
0100		

```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

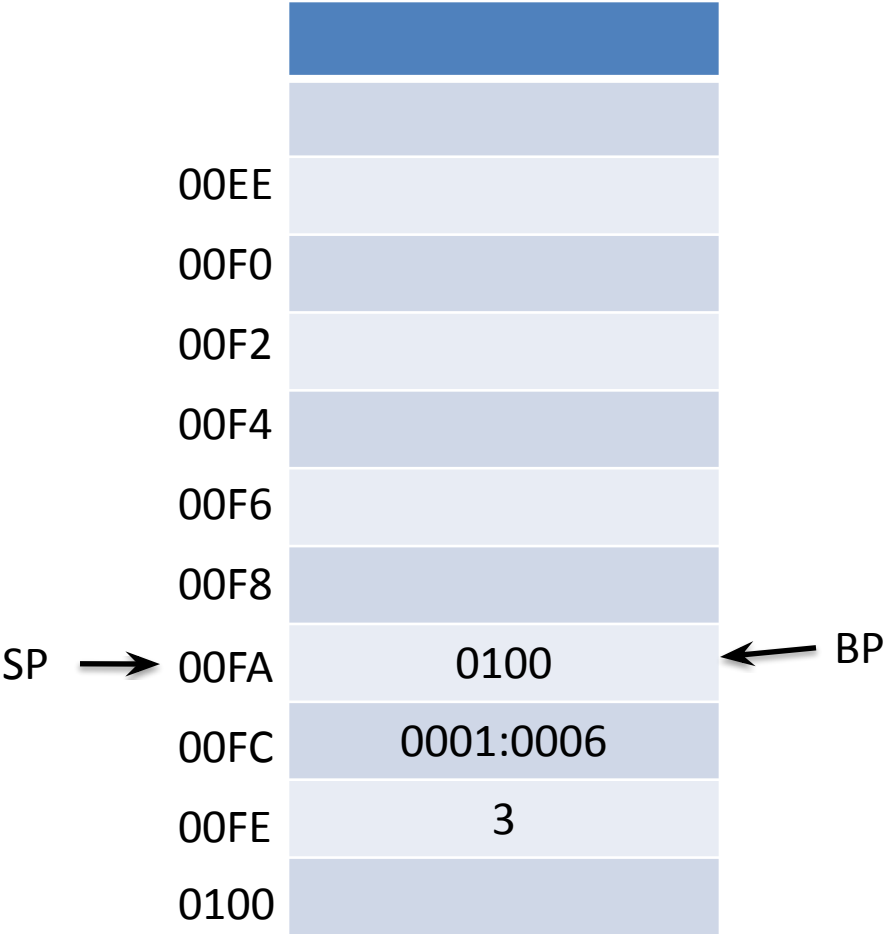
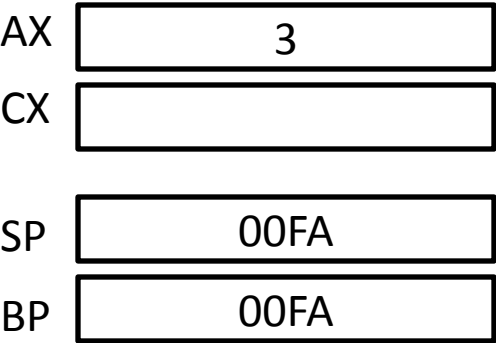
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

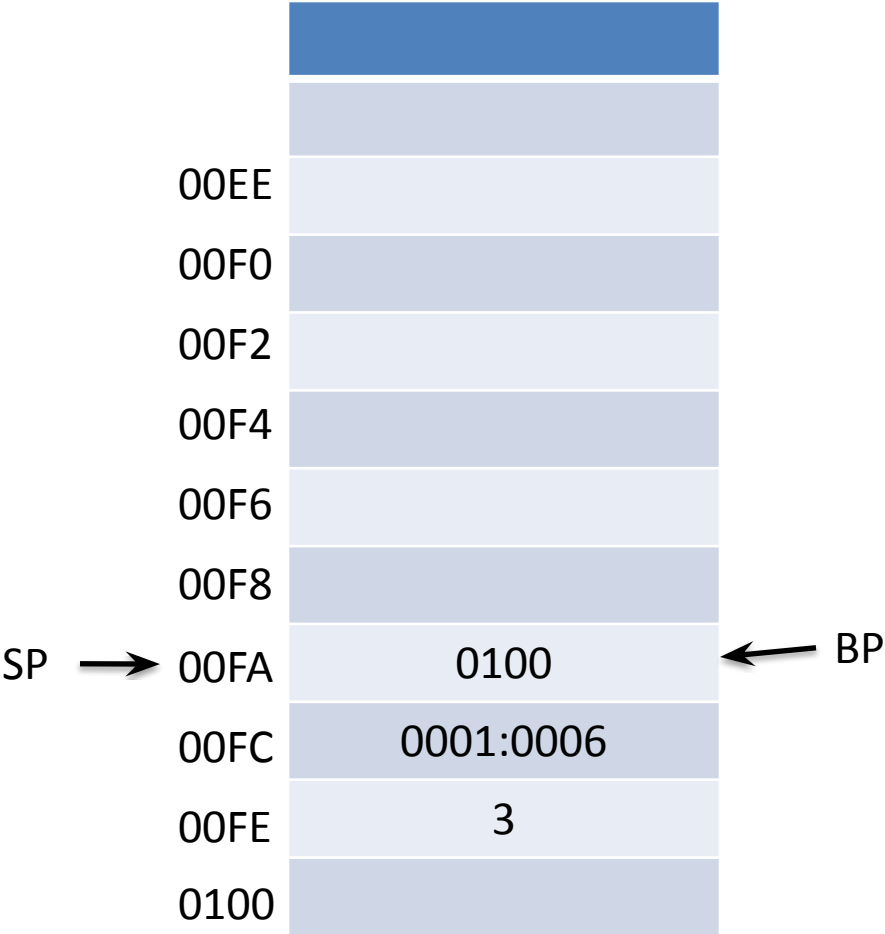
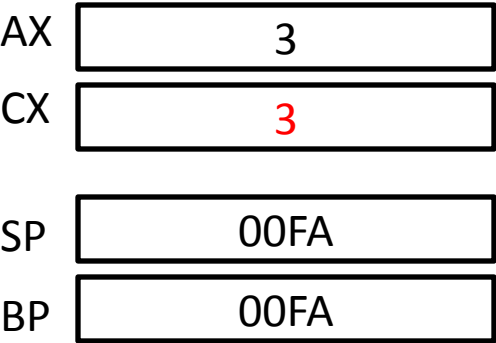
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP BP
0022      RET 2
      END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

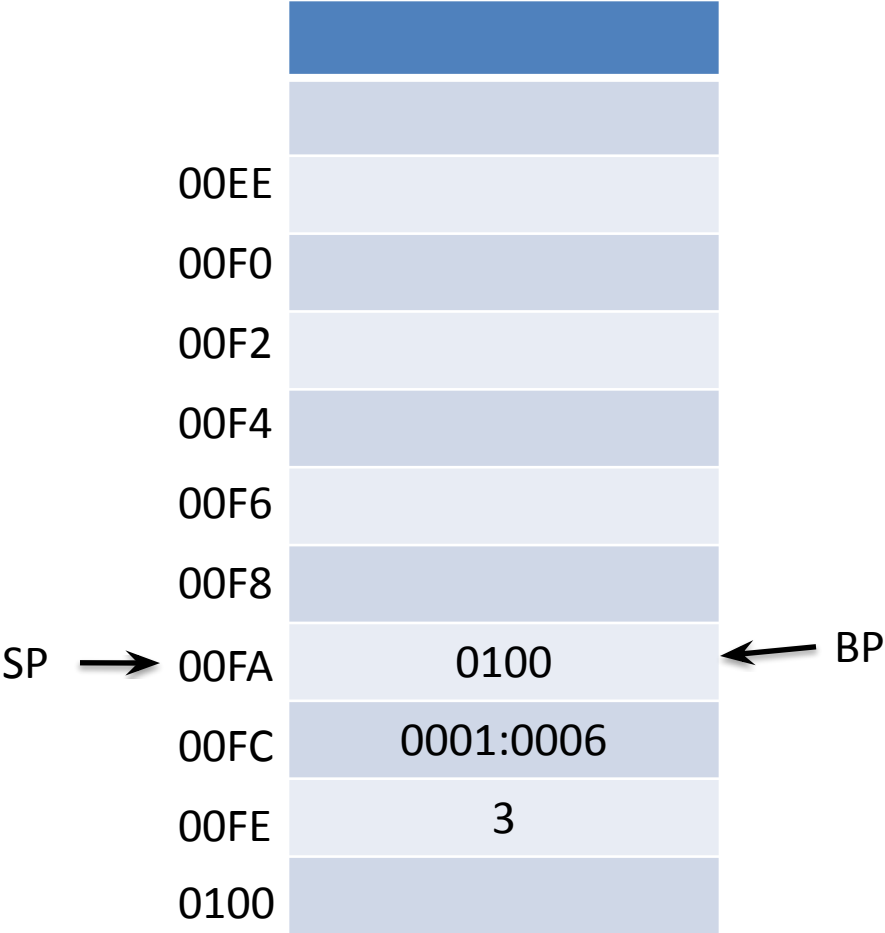
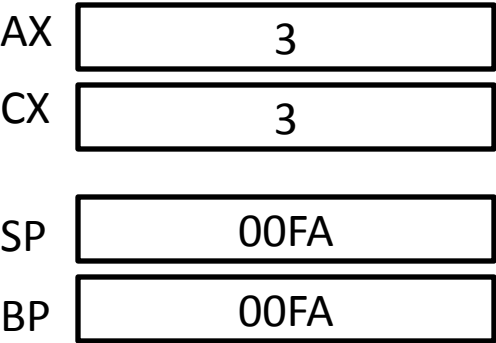
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

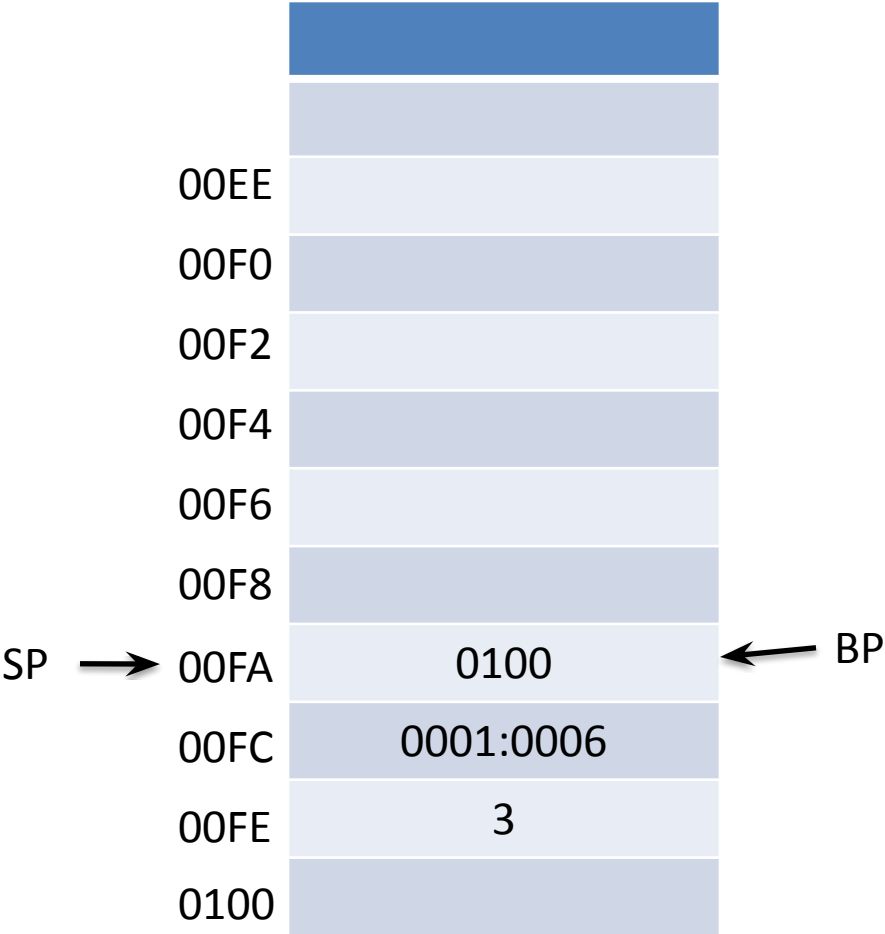
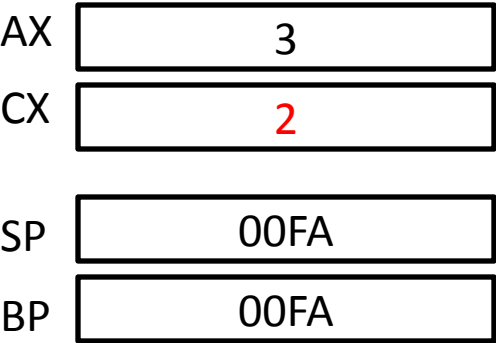
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

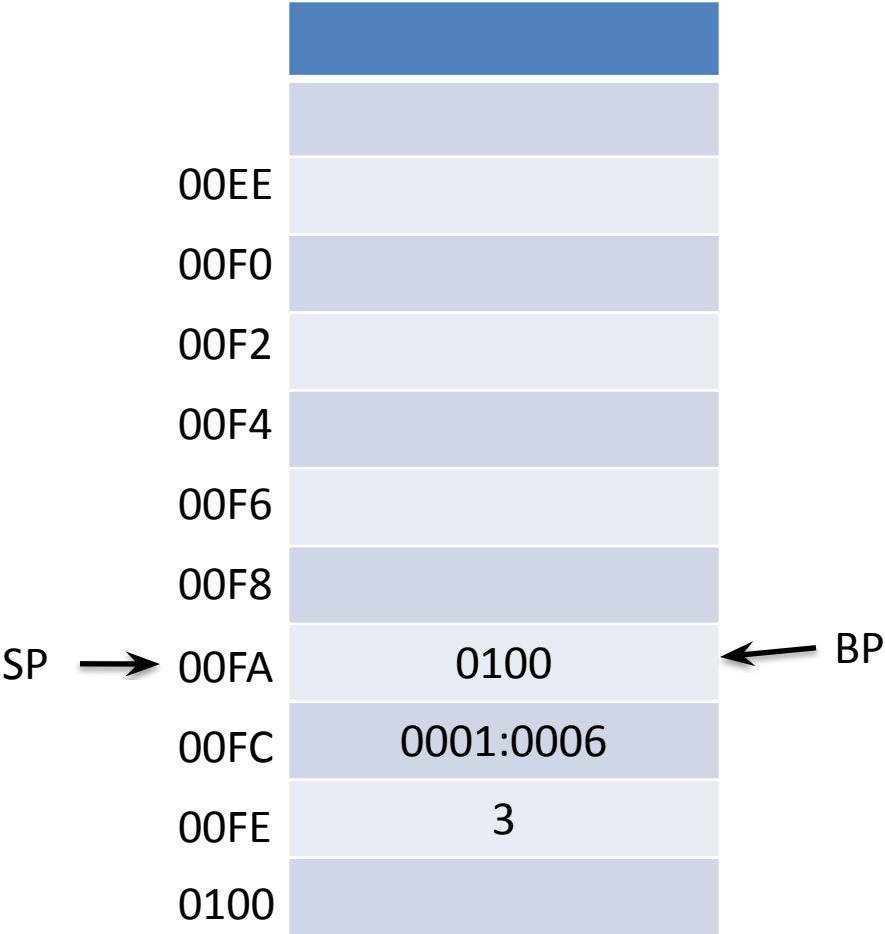
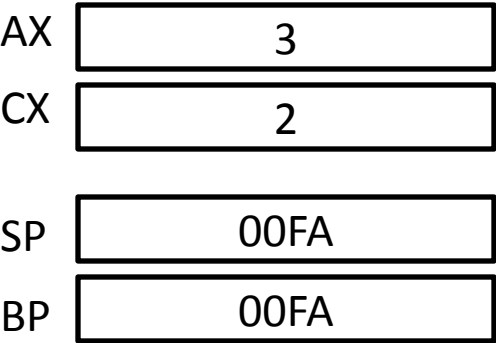
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
      END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
END MAIN
```

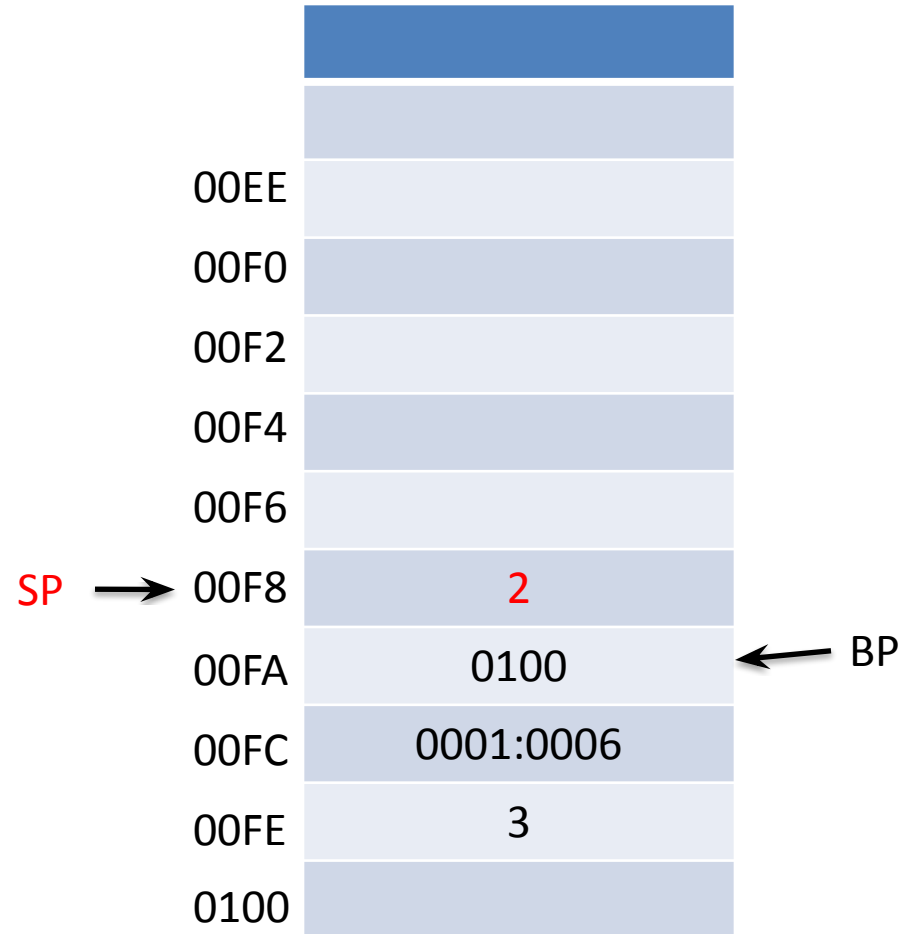
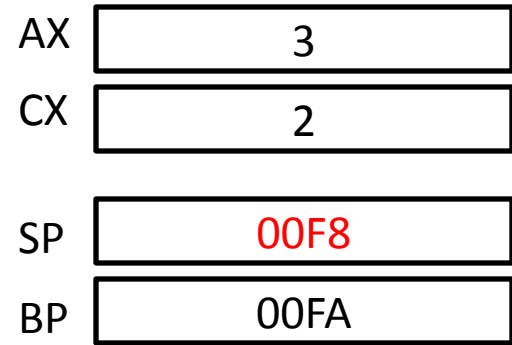


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

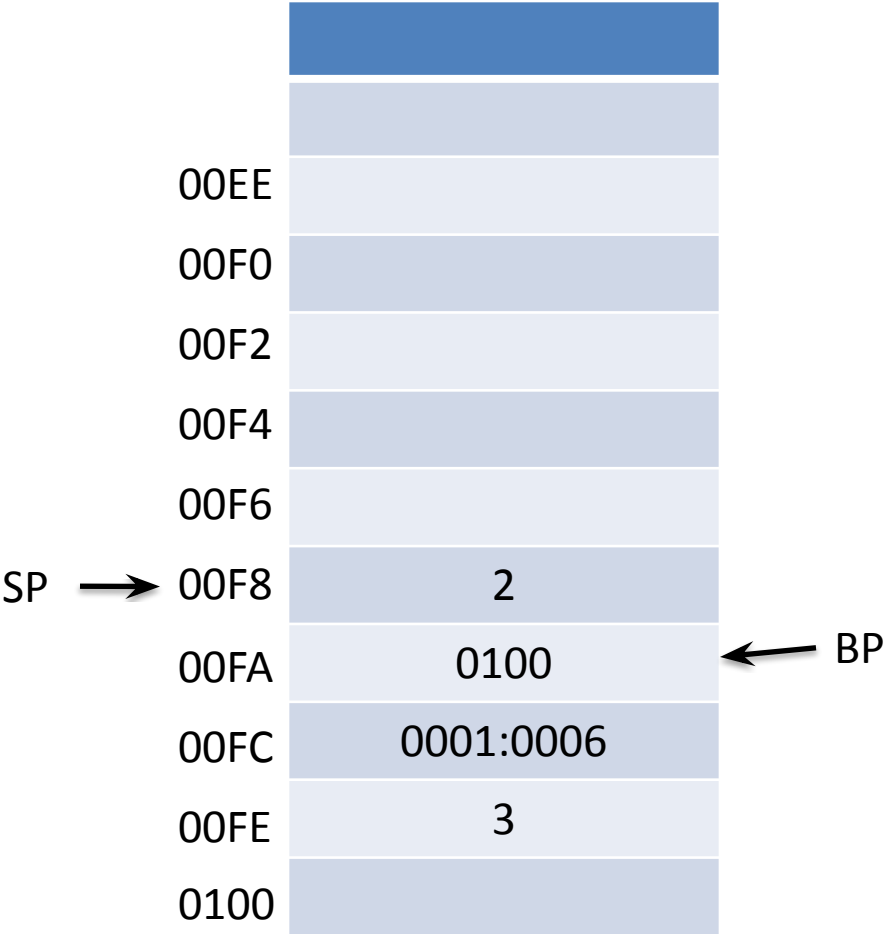
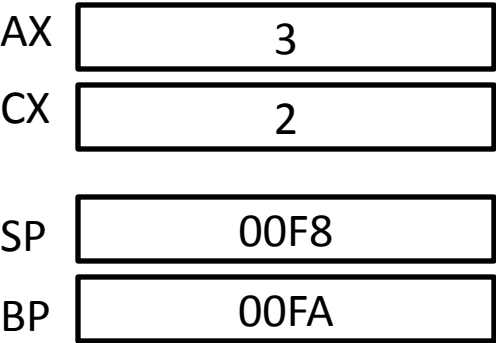



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

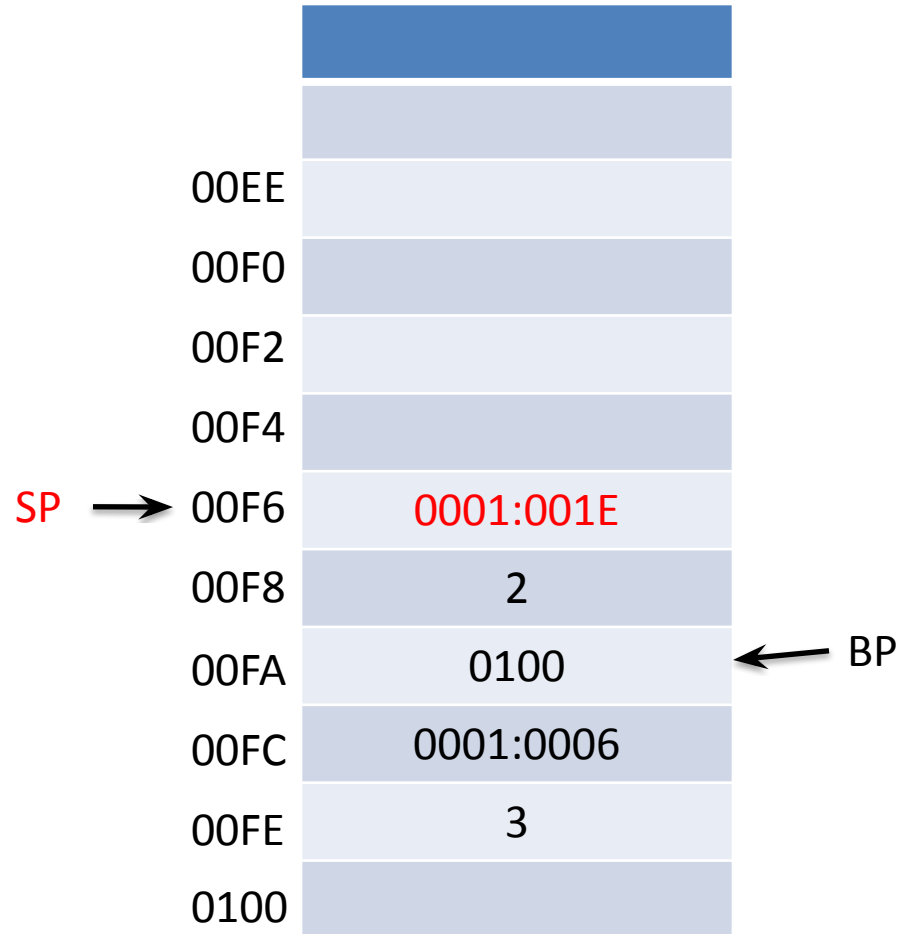
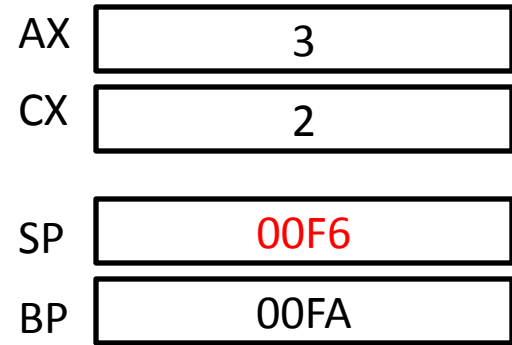
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

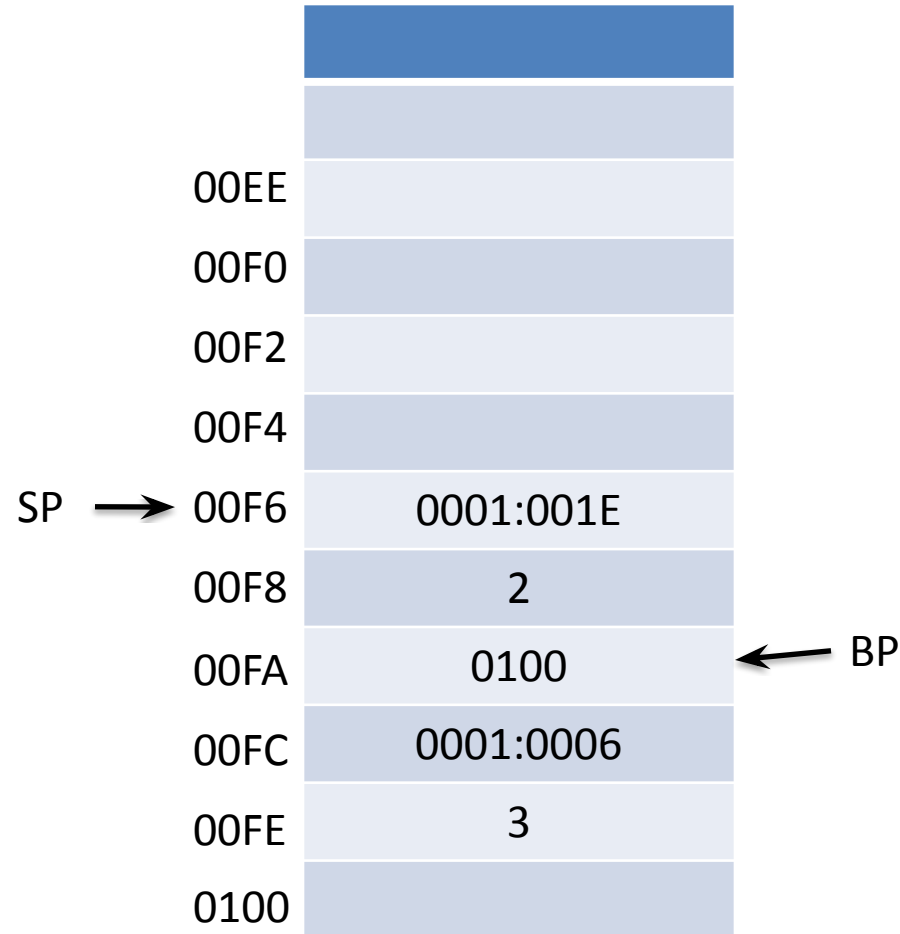
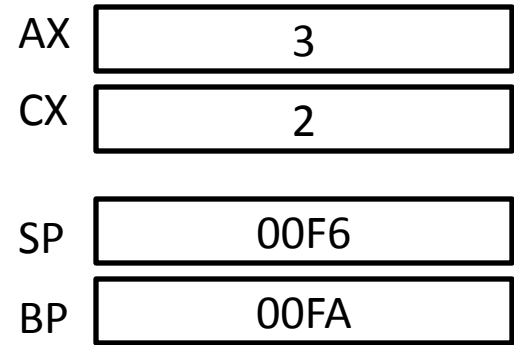
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP BP
0022      RET 2
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END MAIN
```



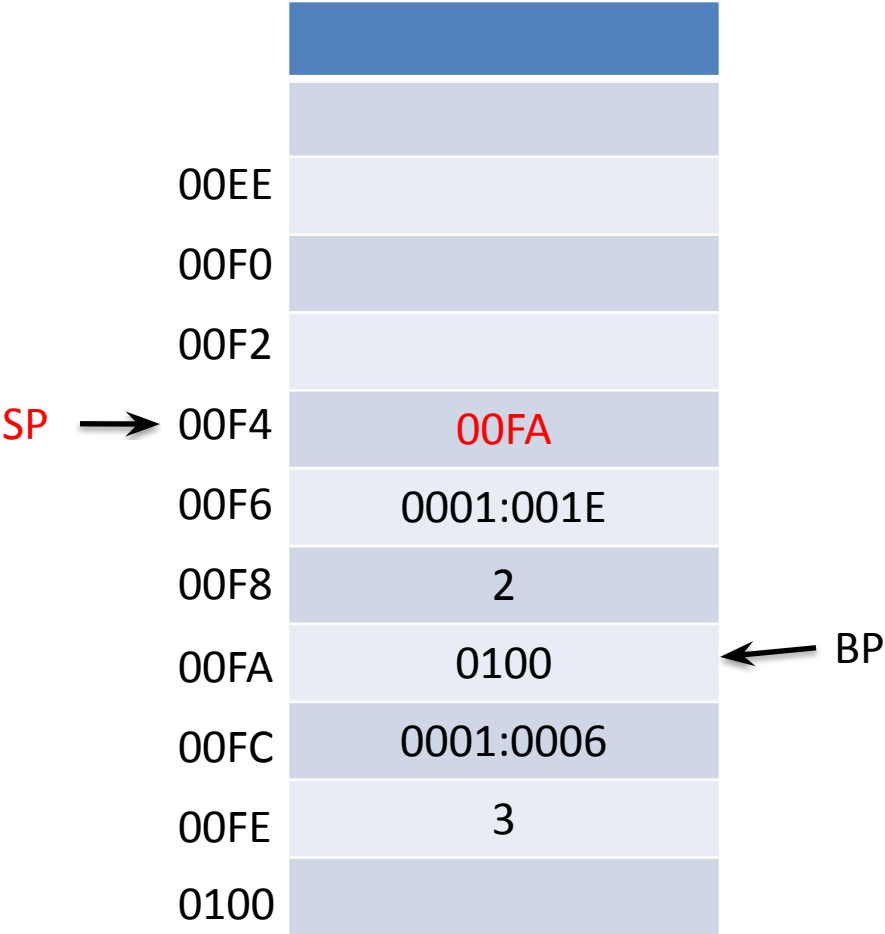
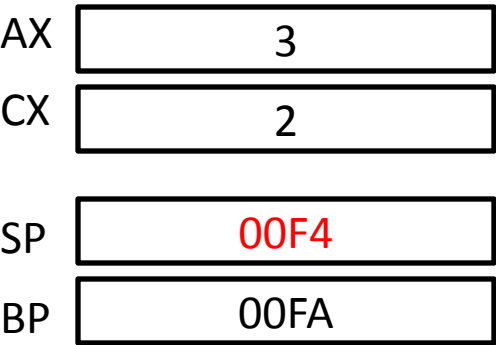
```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
```

MAIN ENDP

FACTORIAL PROC NEAR

```
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
```

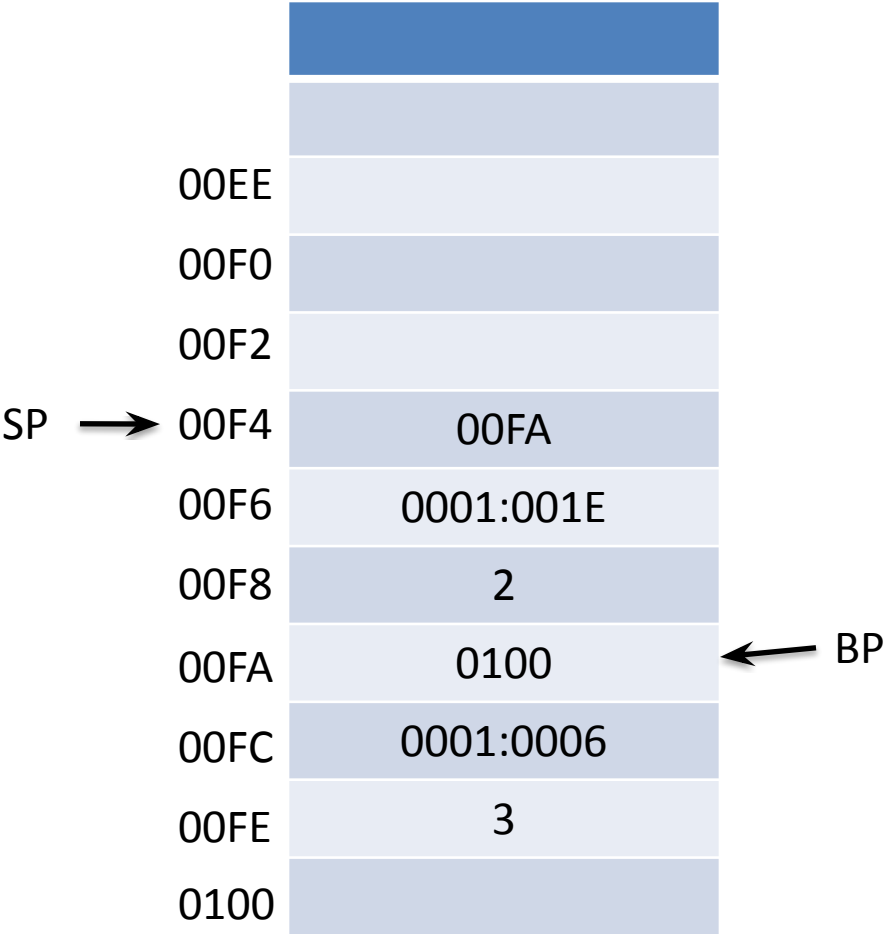
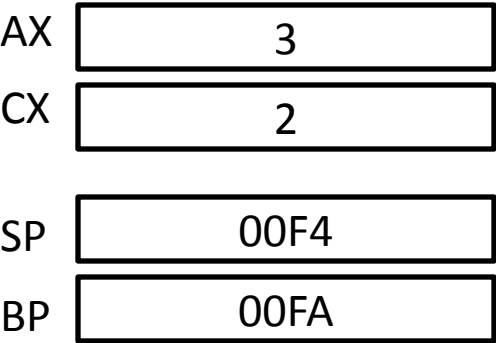
END MAIN



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```

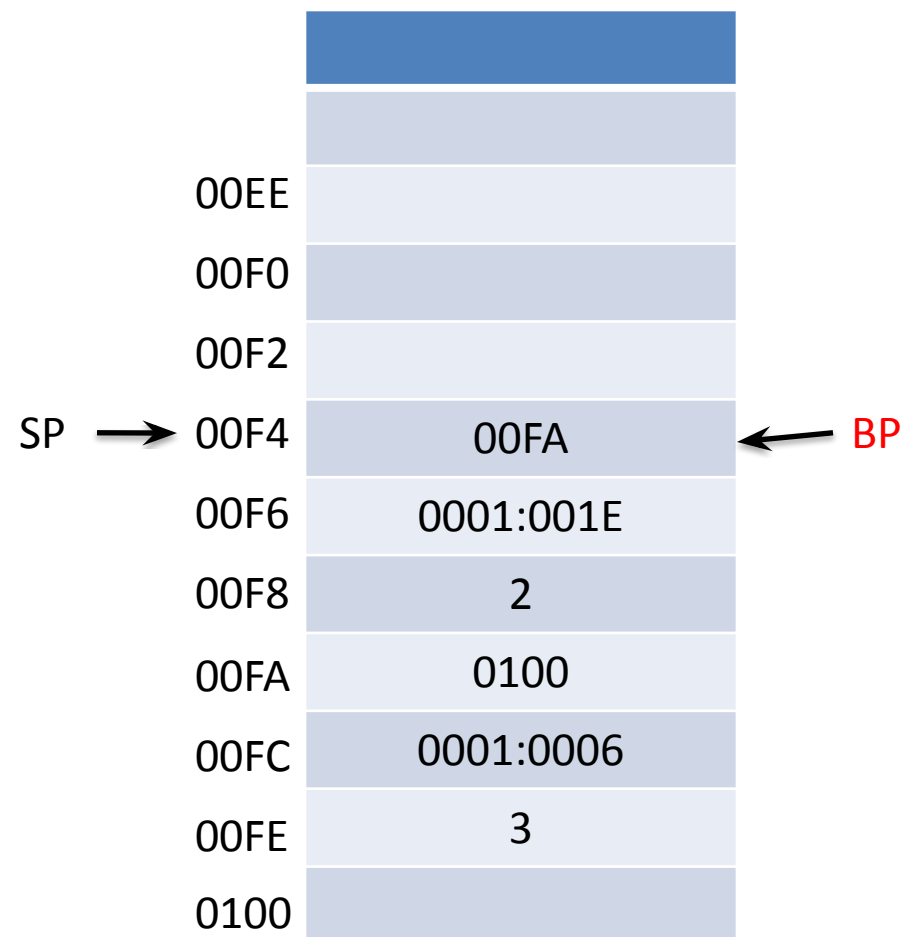
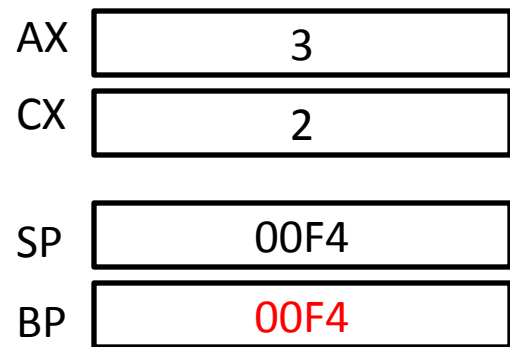


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

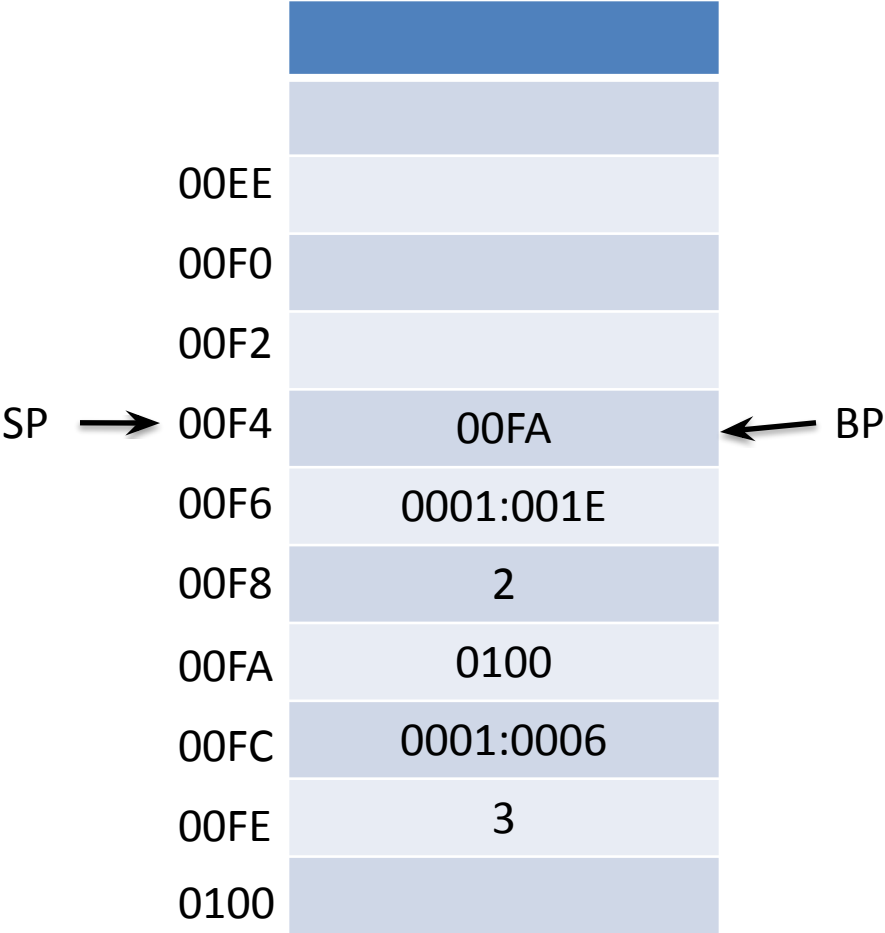
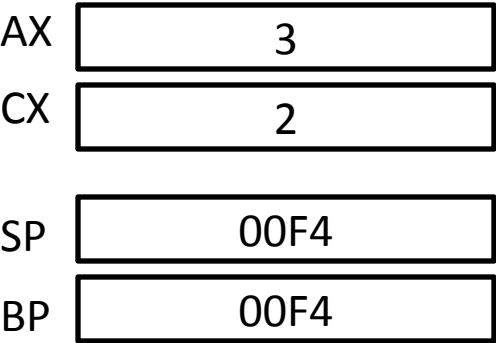


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

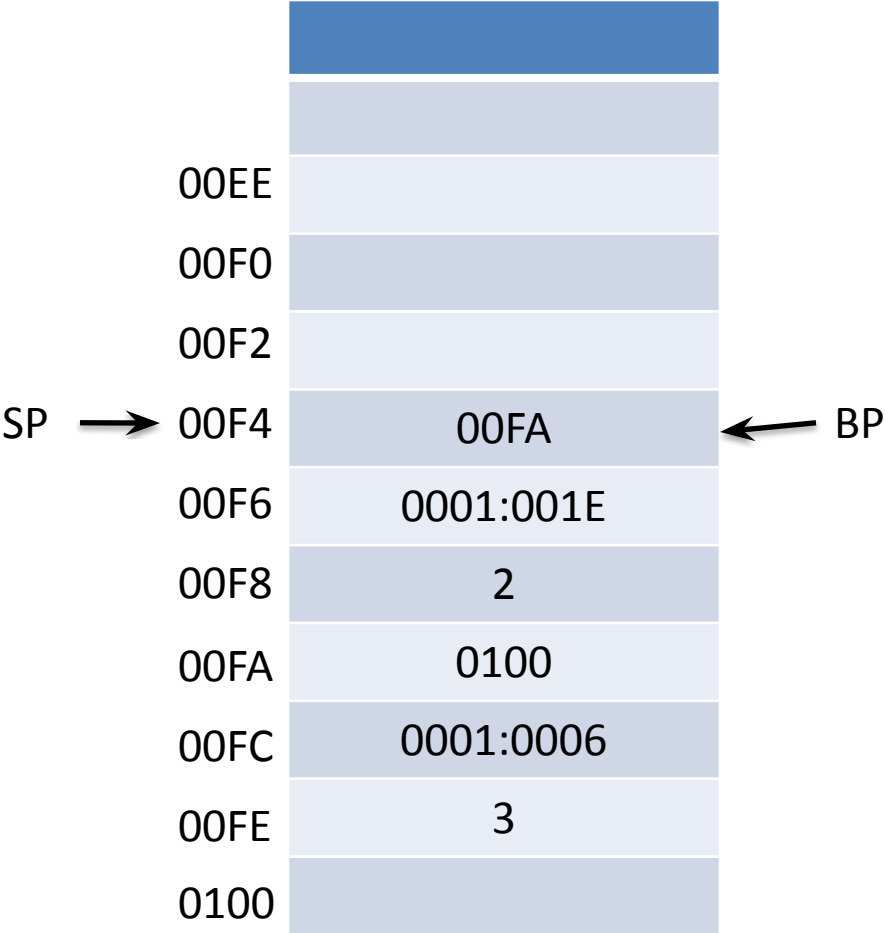
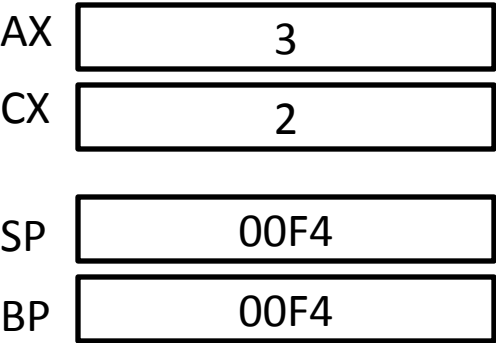


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

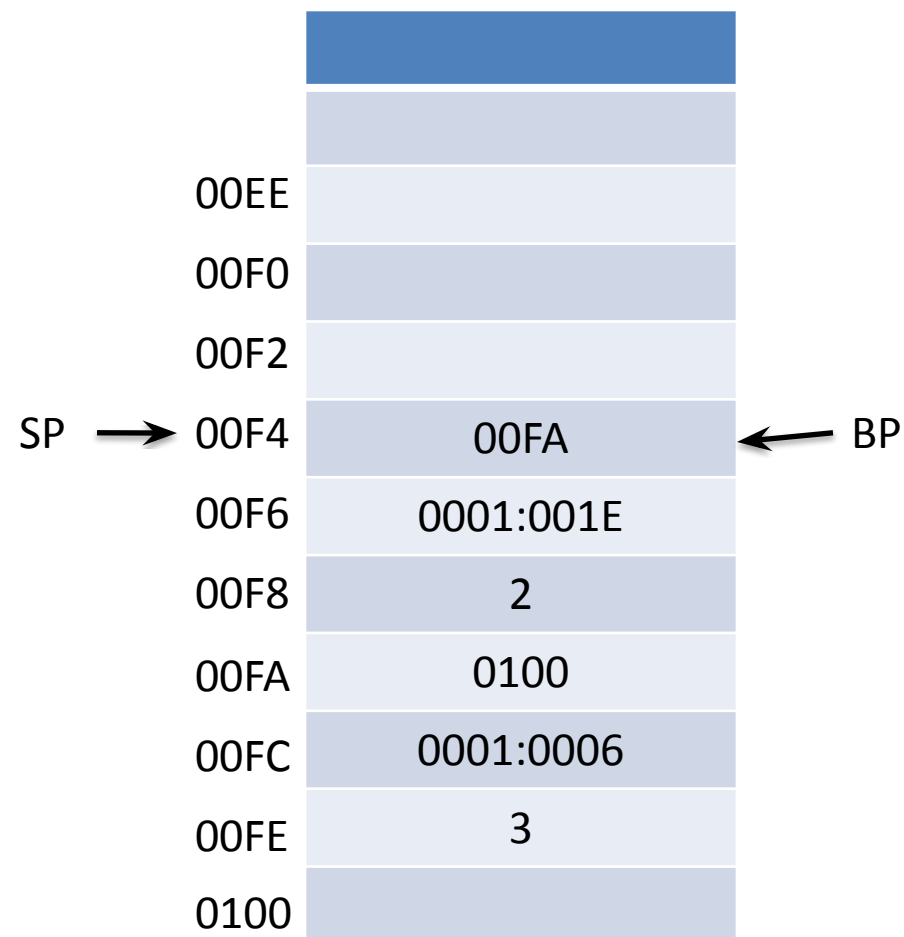
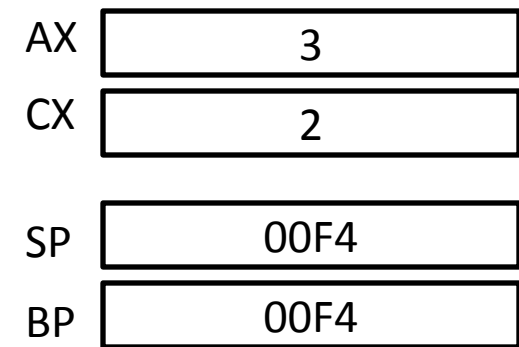
END MAIN
```




```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

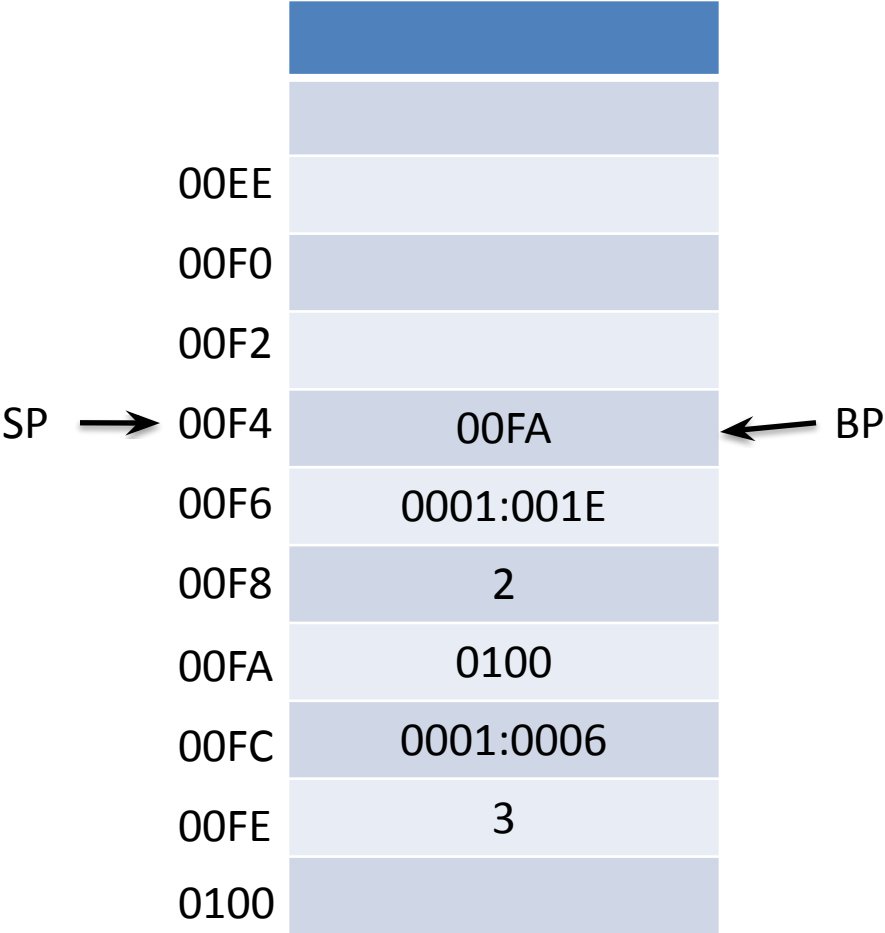
MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
END MAIN
```

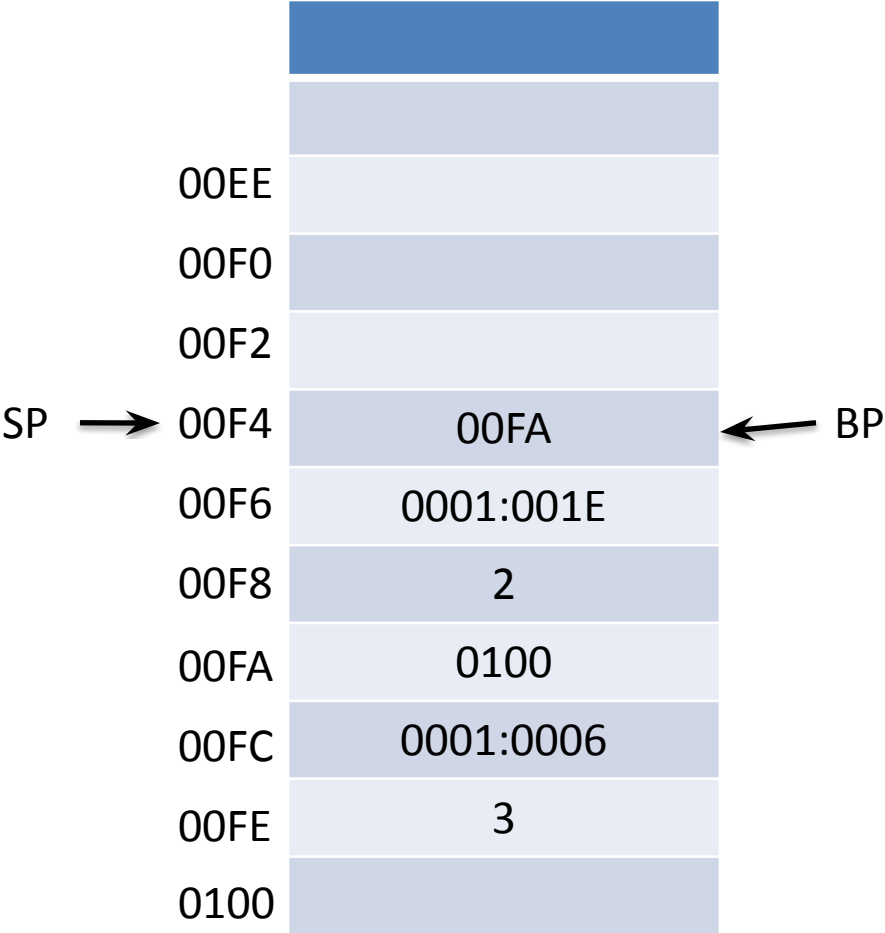
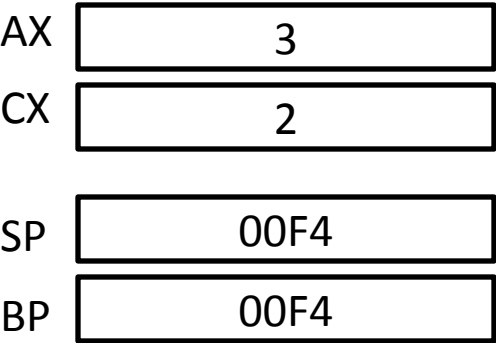


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```



```

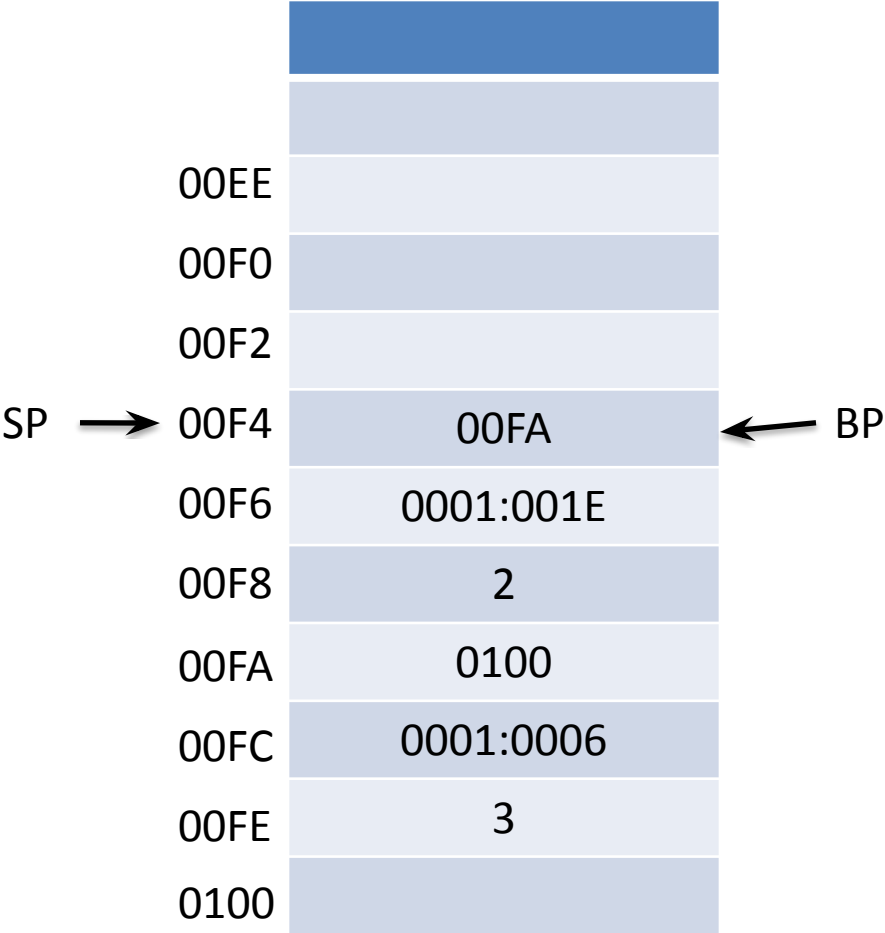
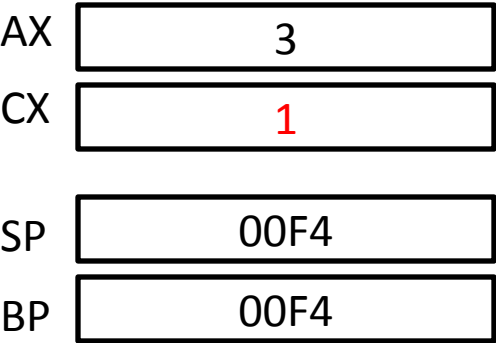
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN

```

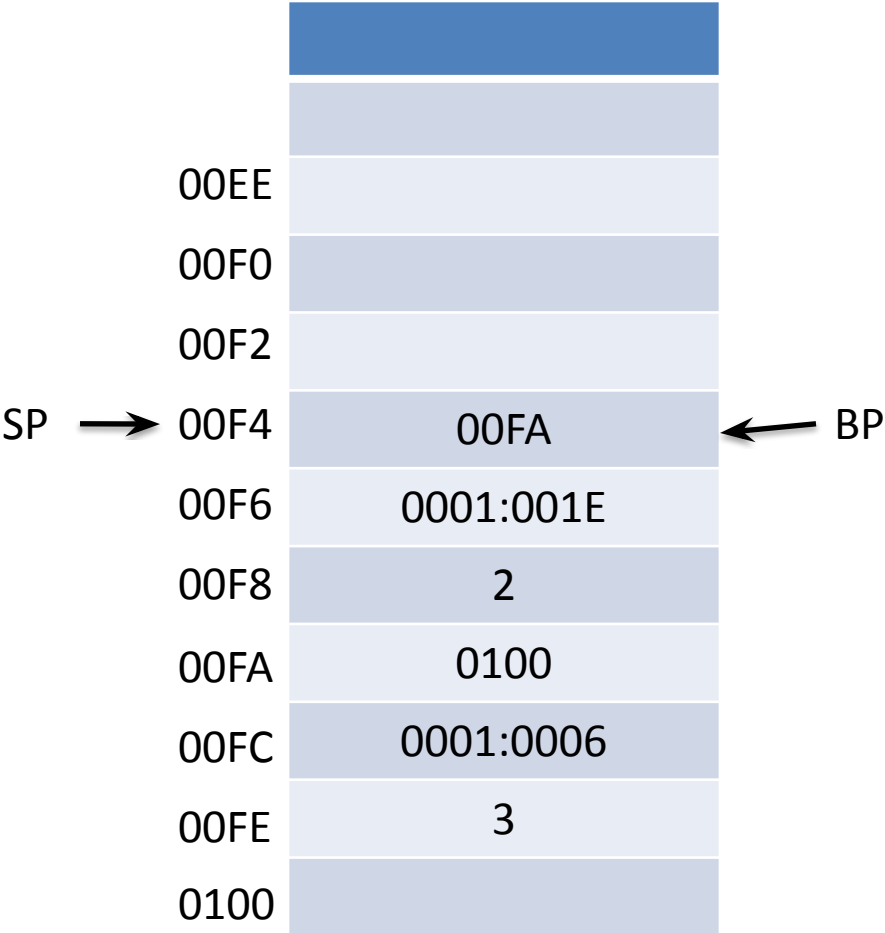
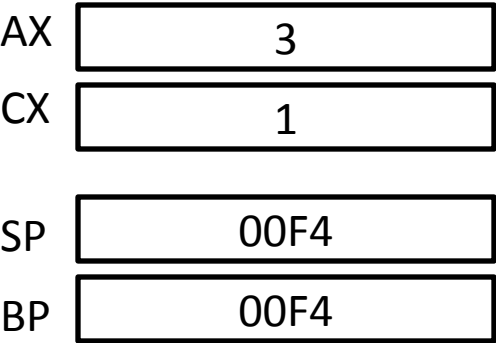


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

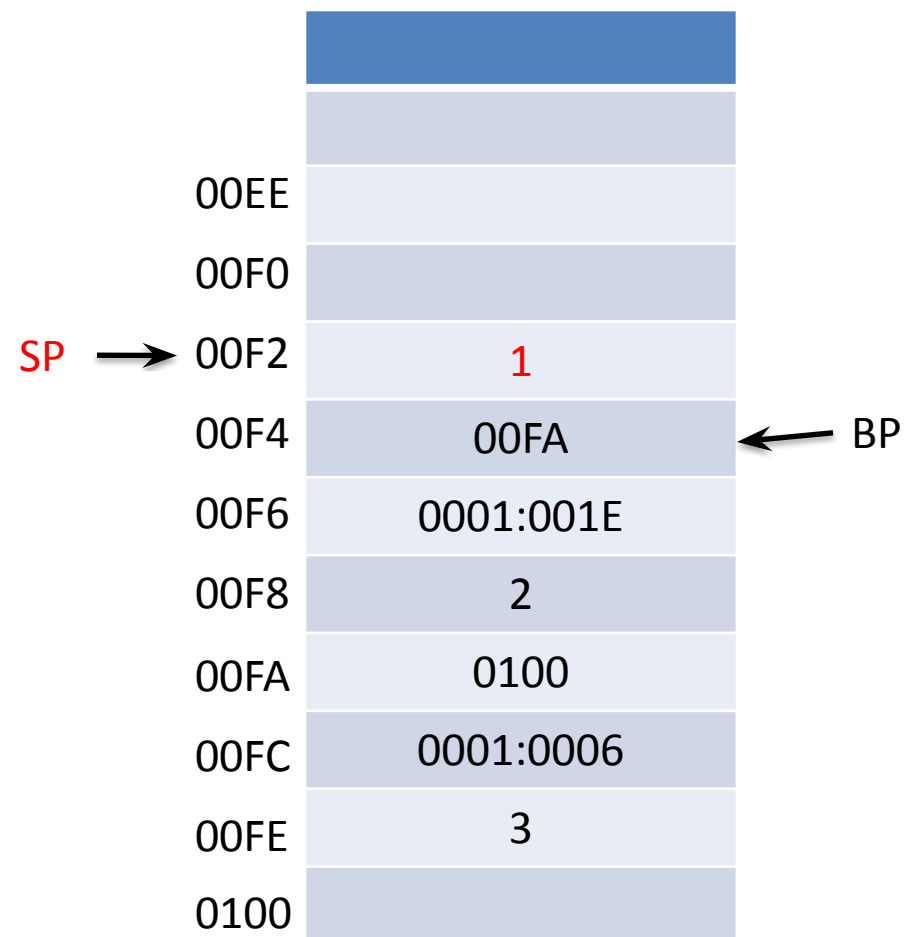
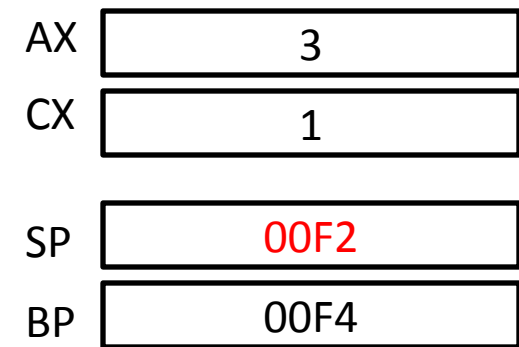


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

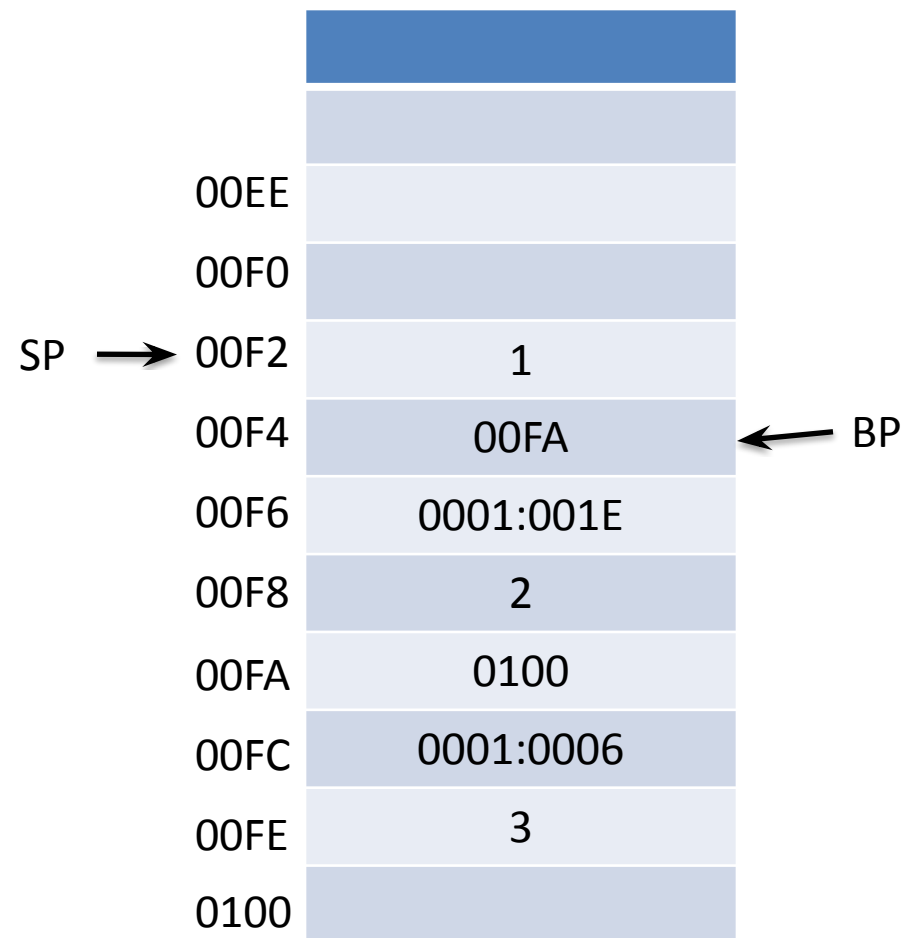
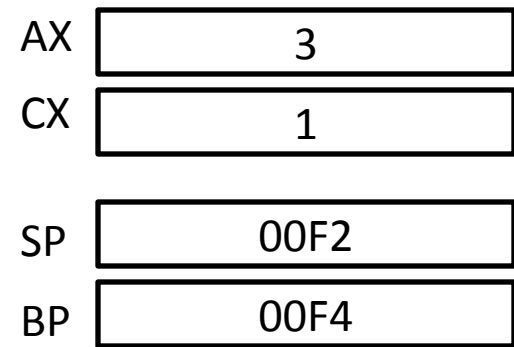
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP BP
0022      RET 2
END MAIN
```

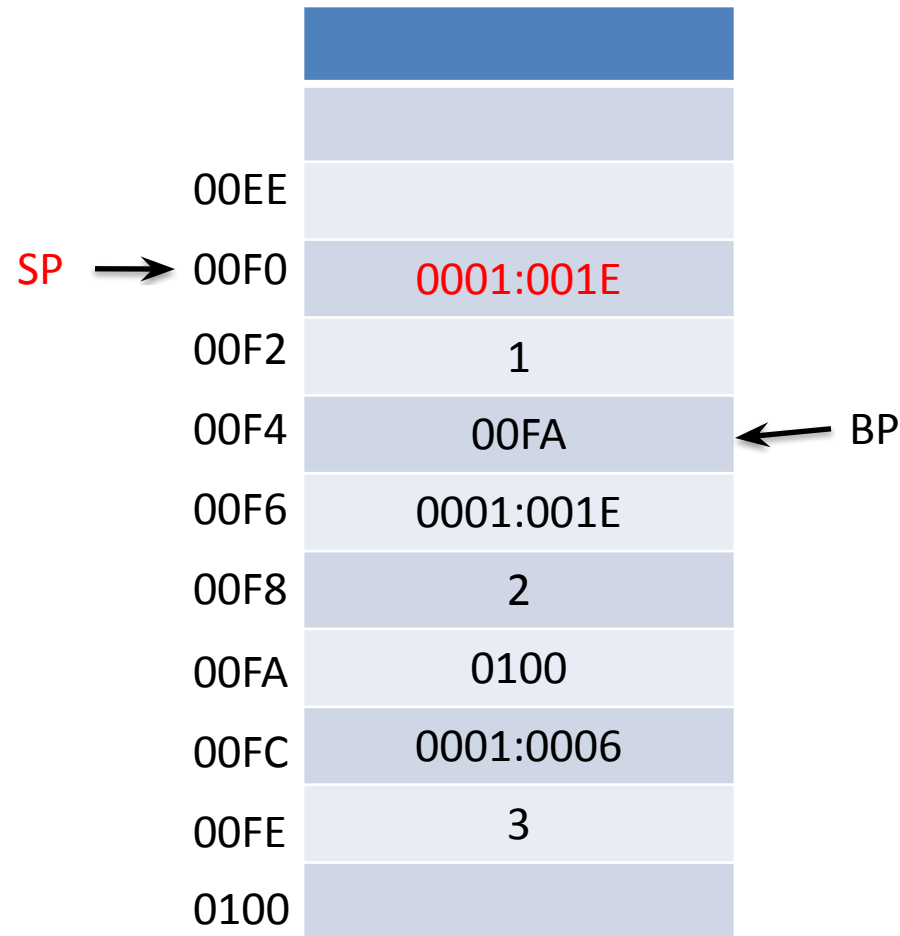
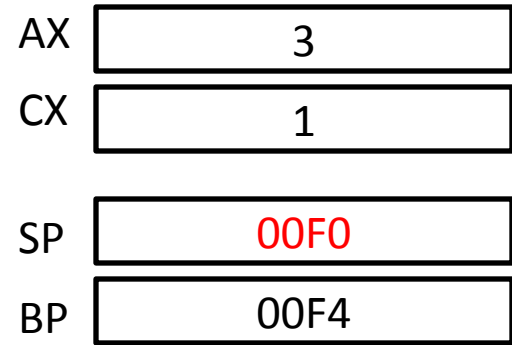


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```



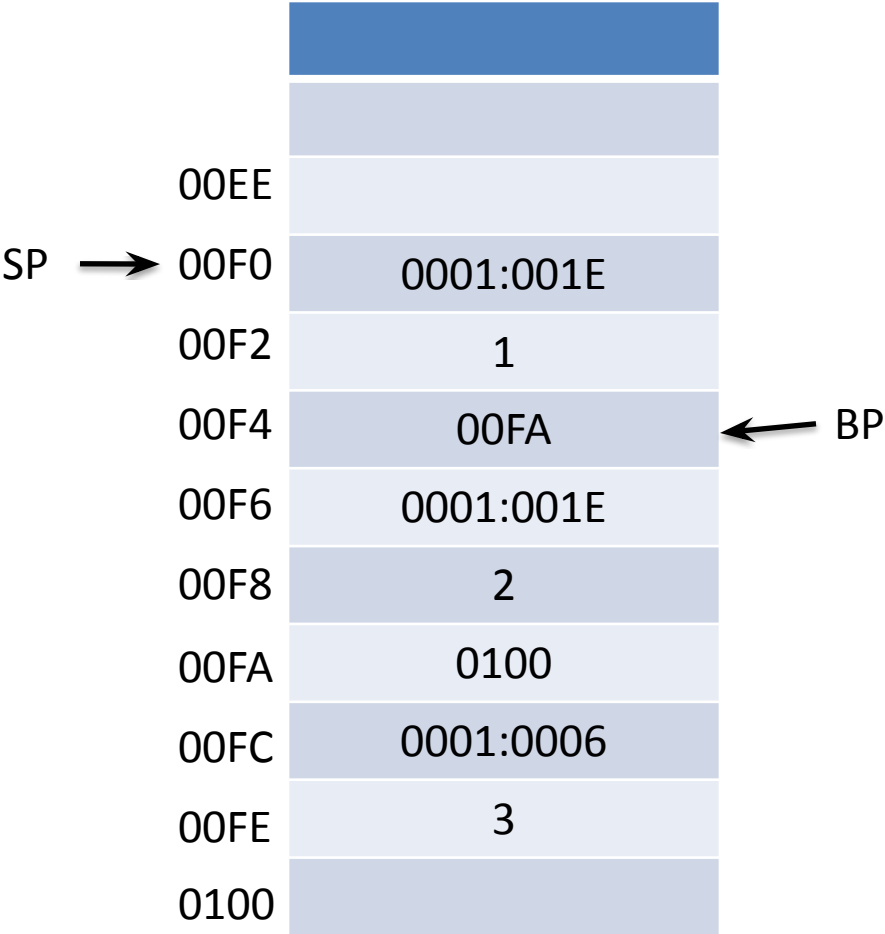
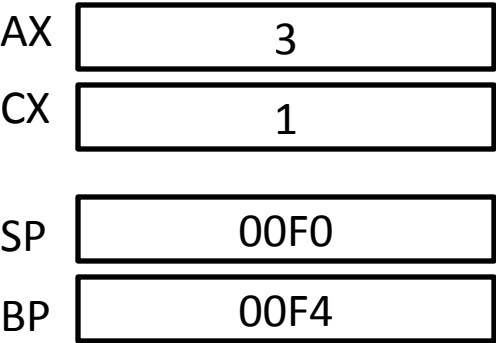

```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
```

MAIN ENDP

FACTORIAL PROC NEAR

```
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
```

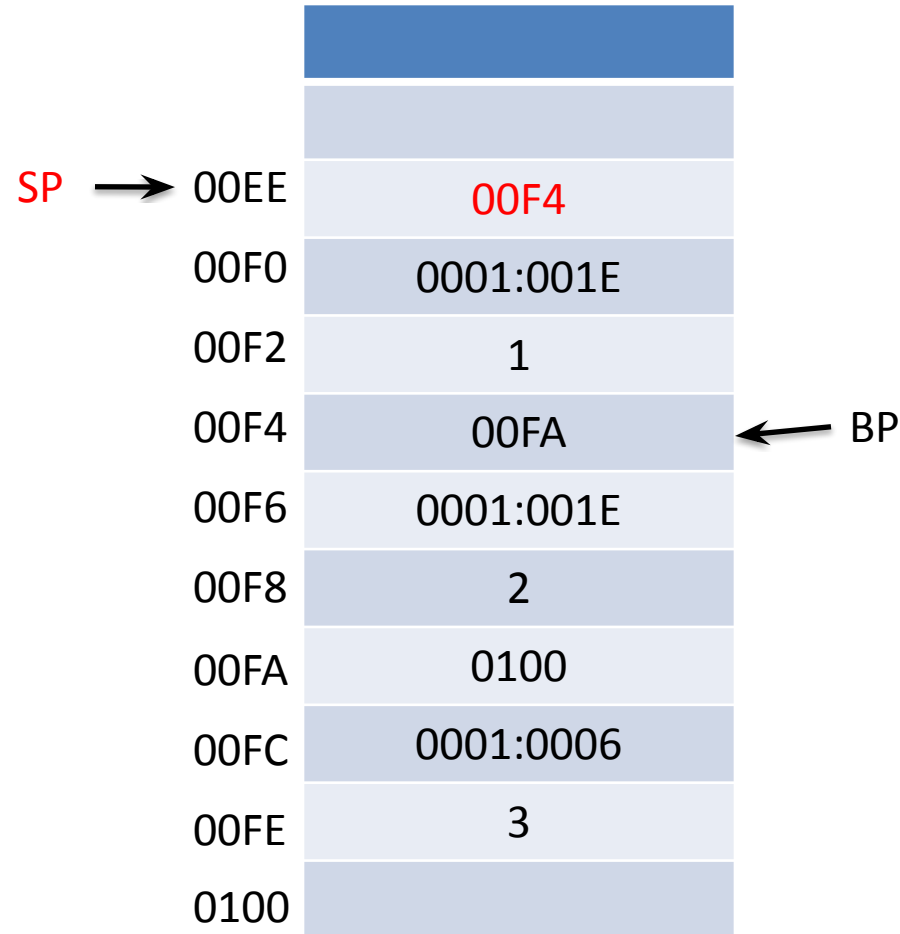
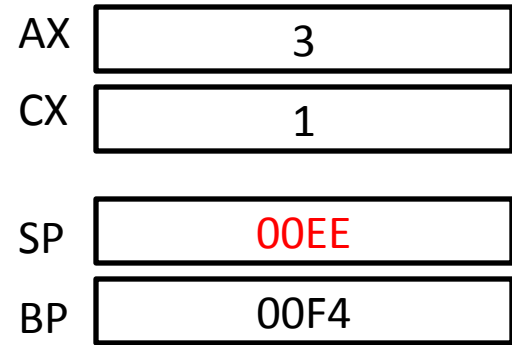
END MAIN



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

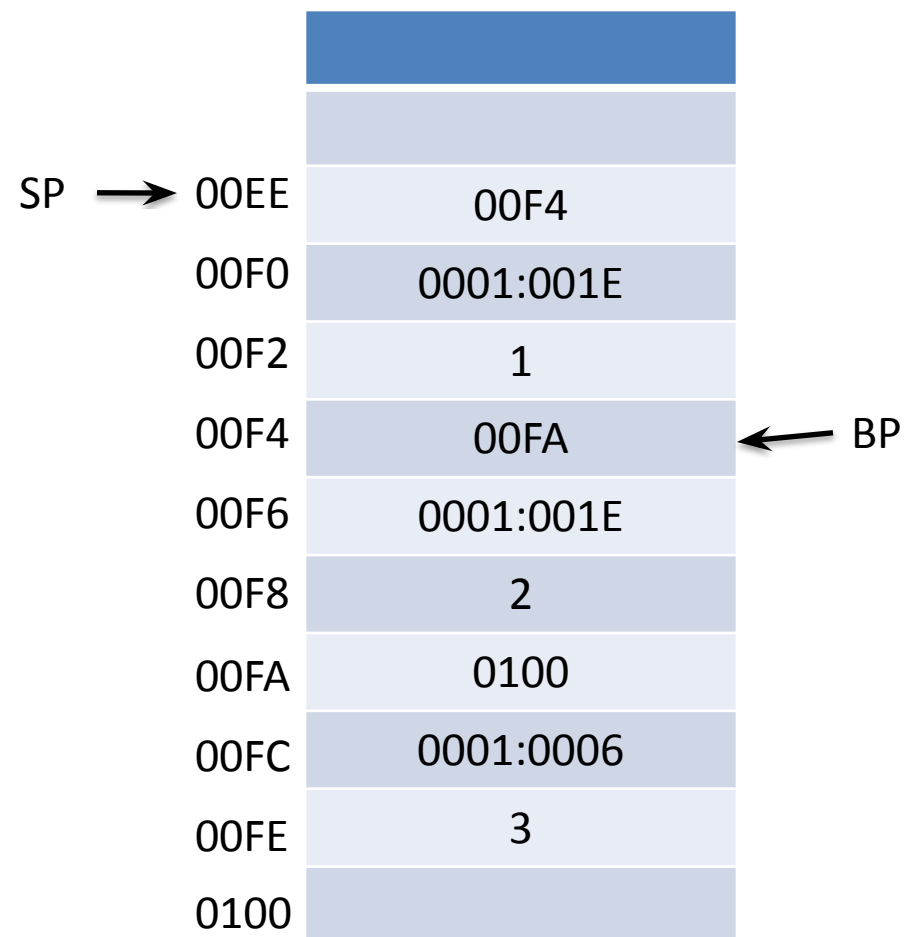
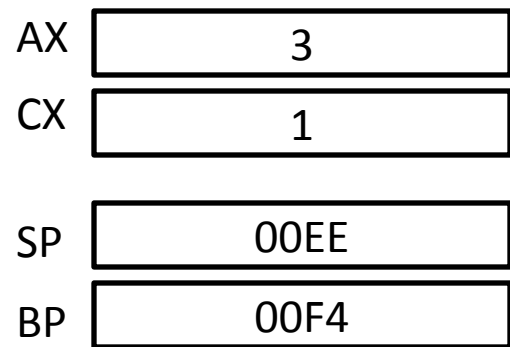
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```

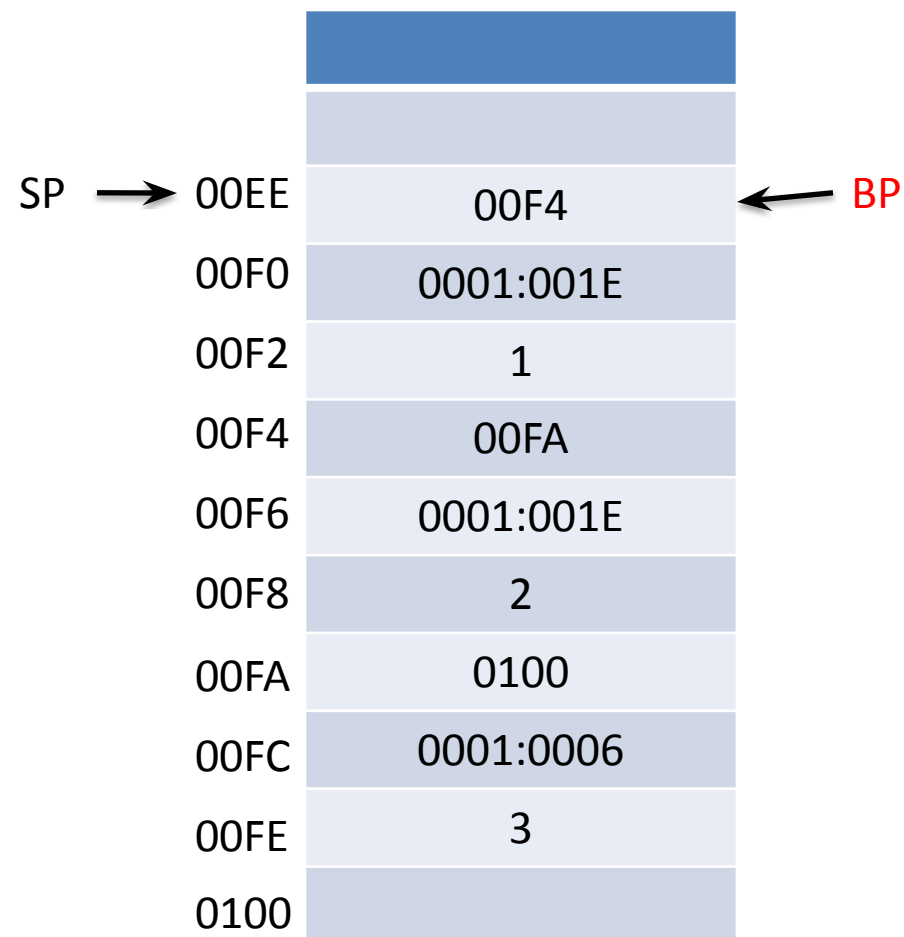
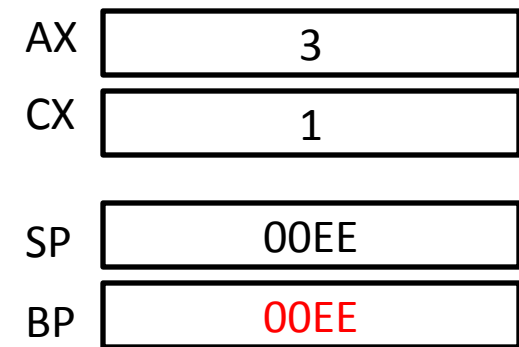


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```



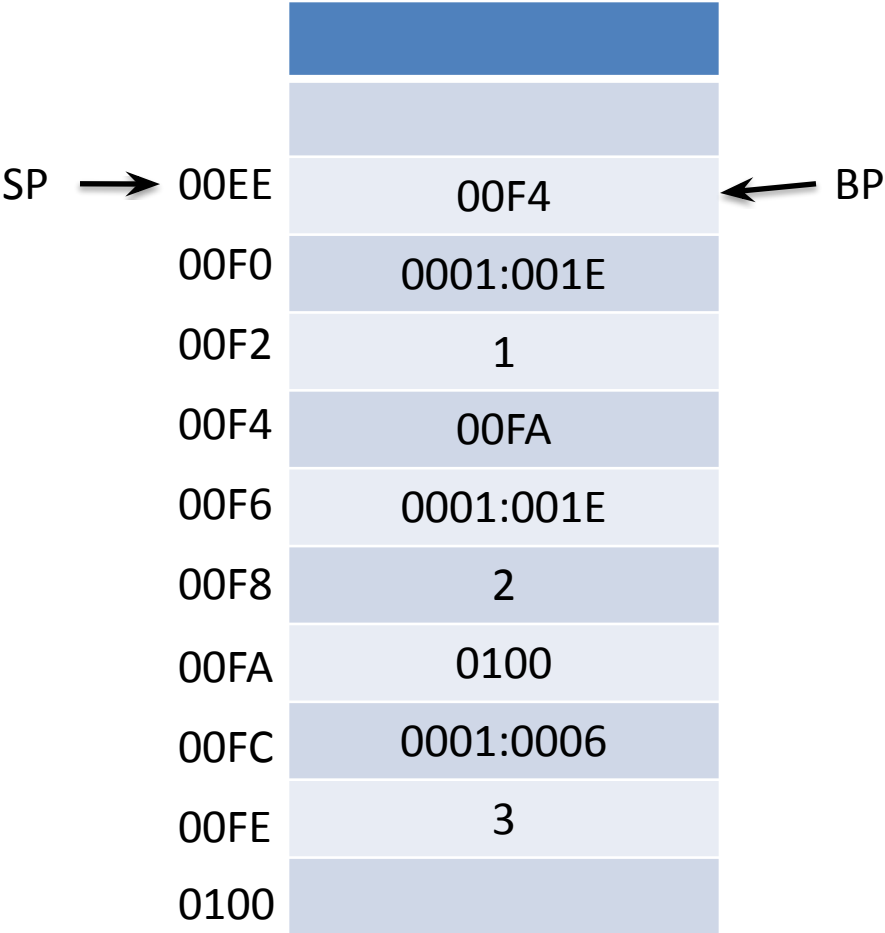
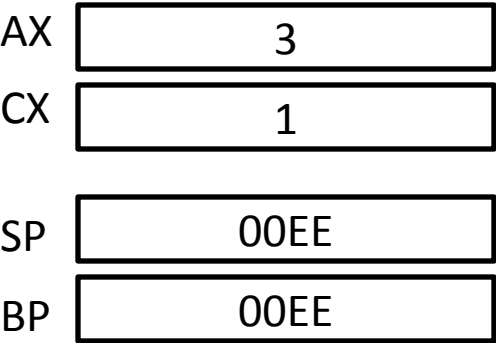
```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN

```

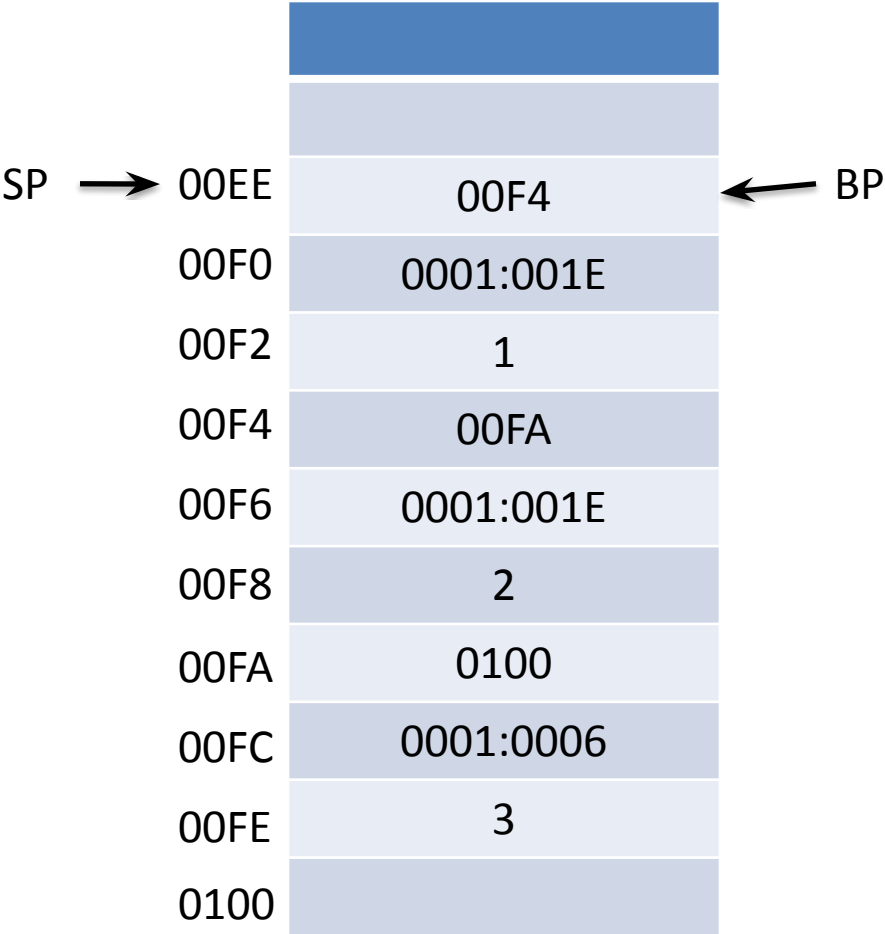
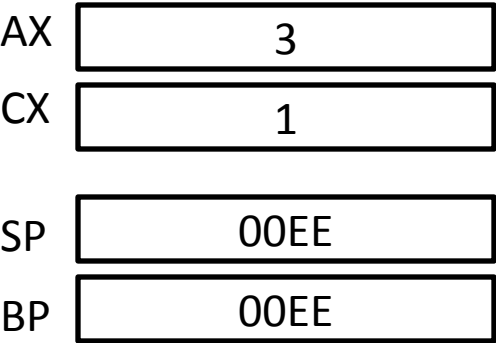


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

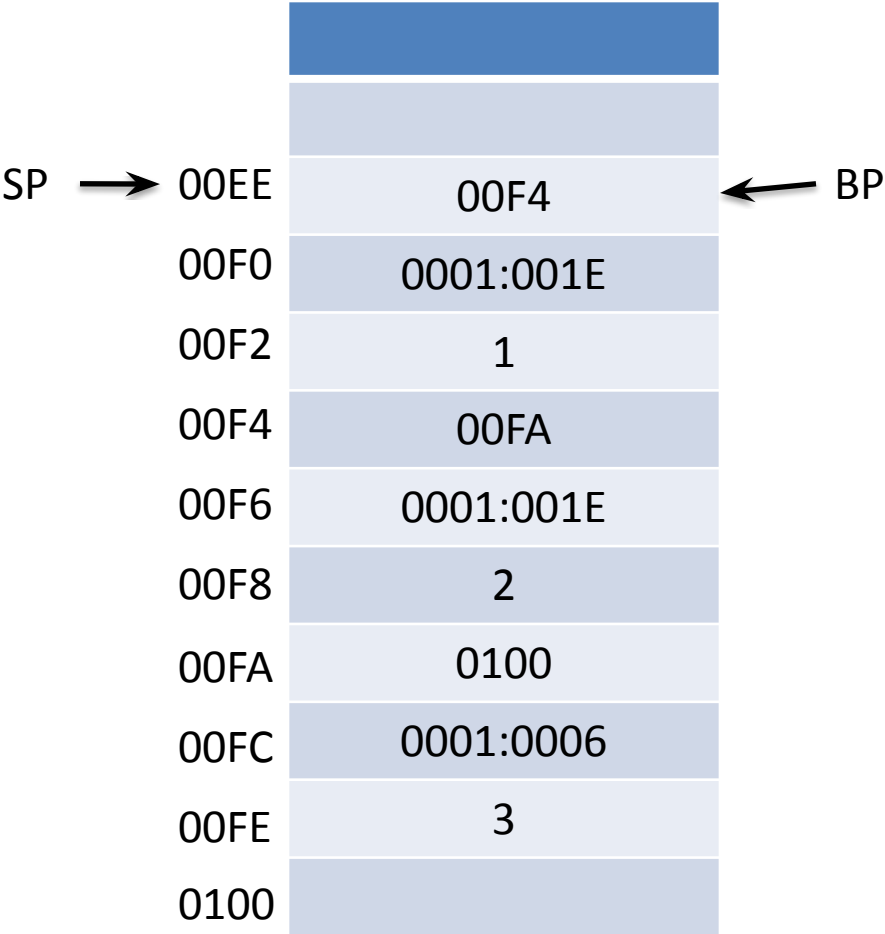
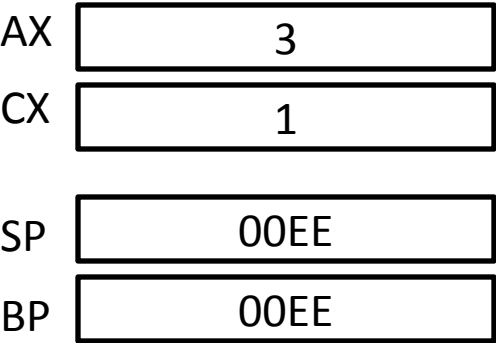
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

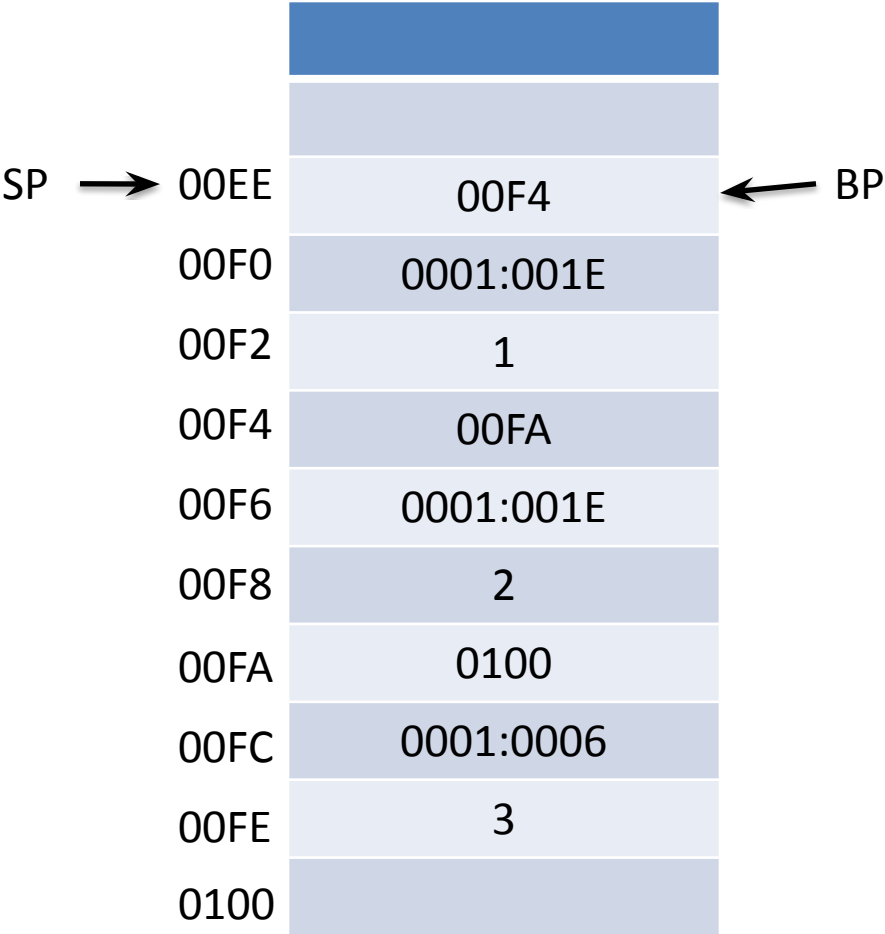
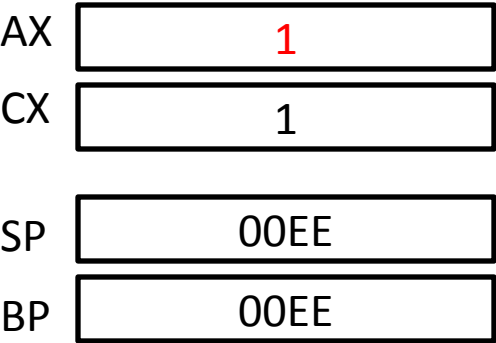
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

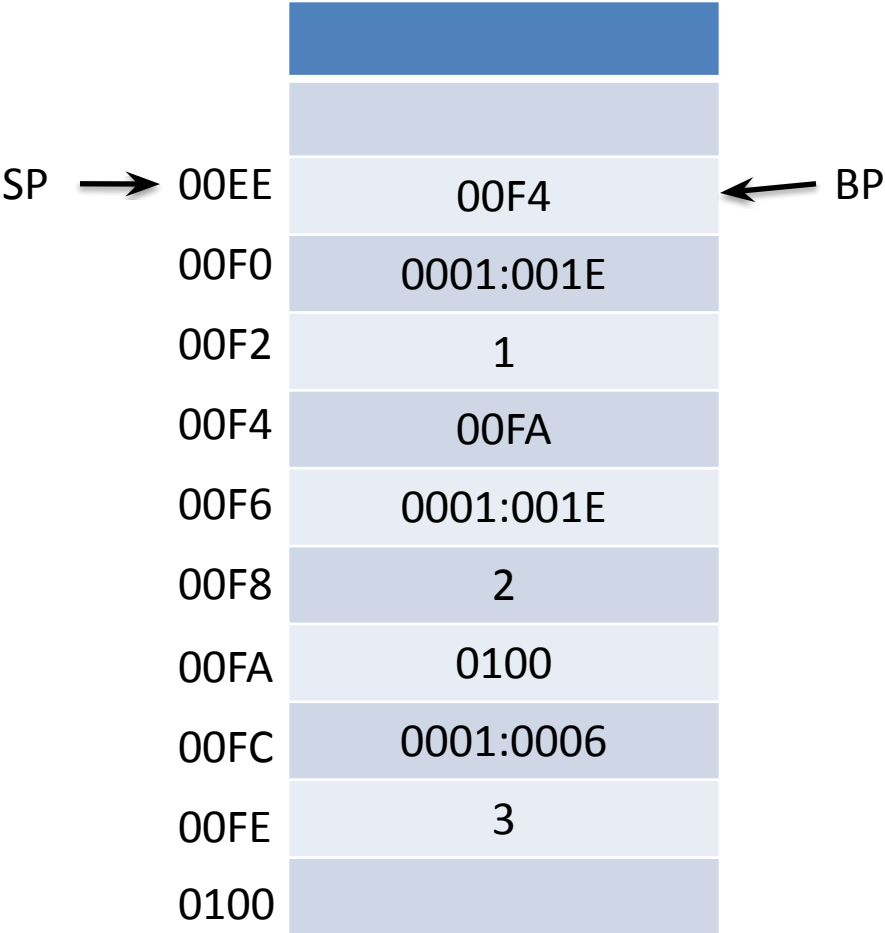
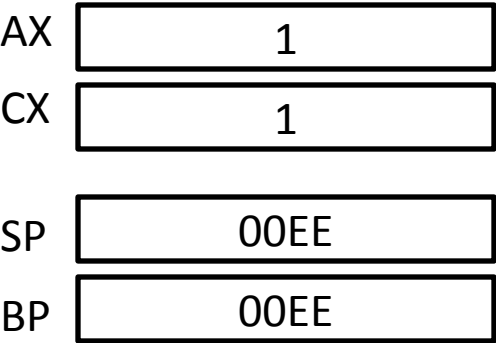
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```




```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

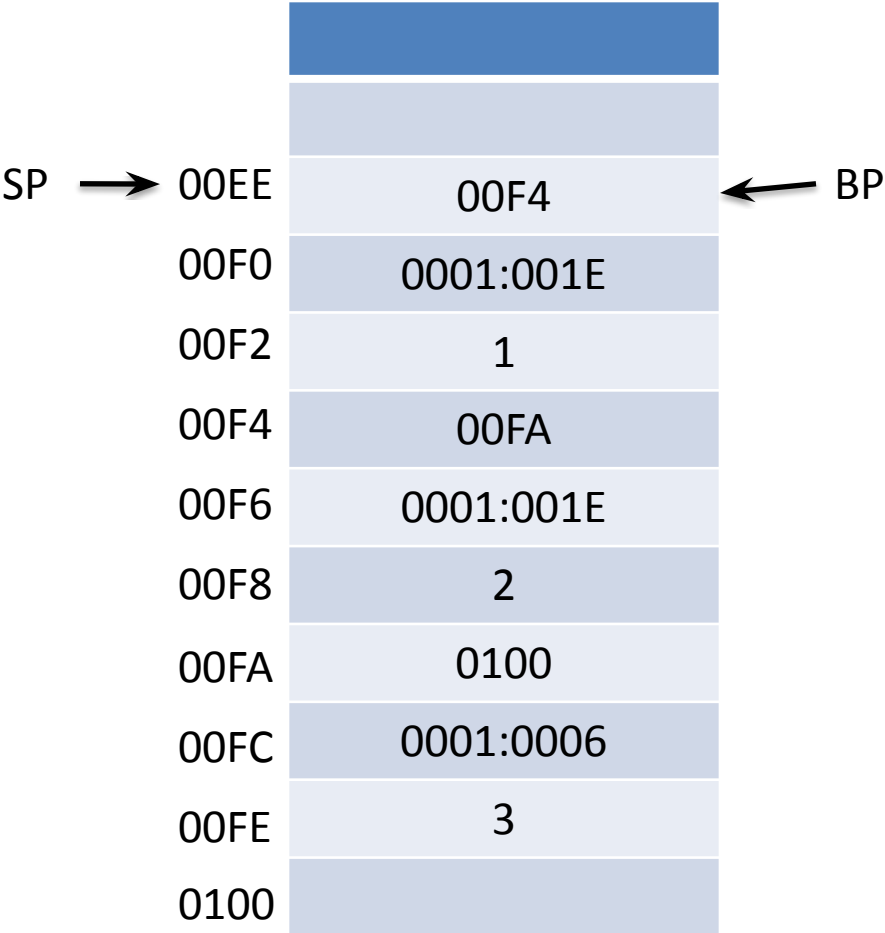
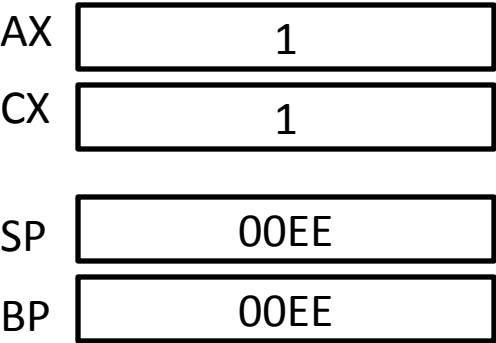
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

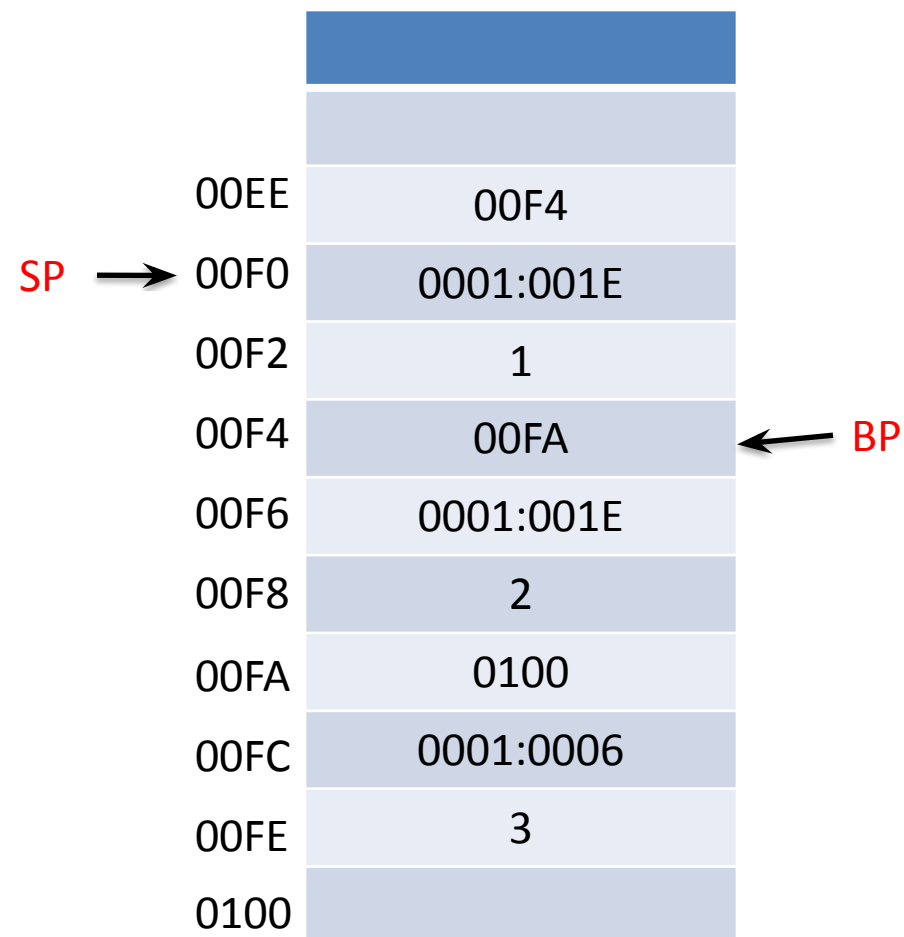
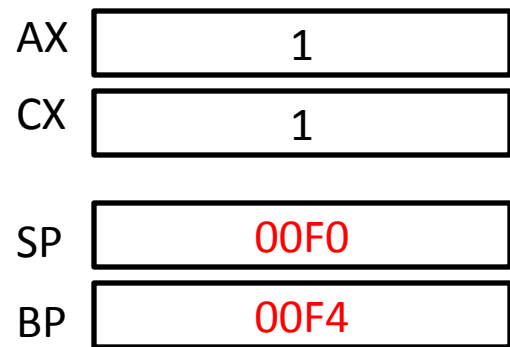
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

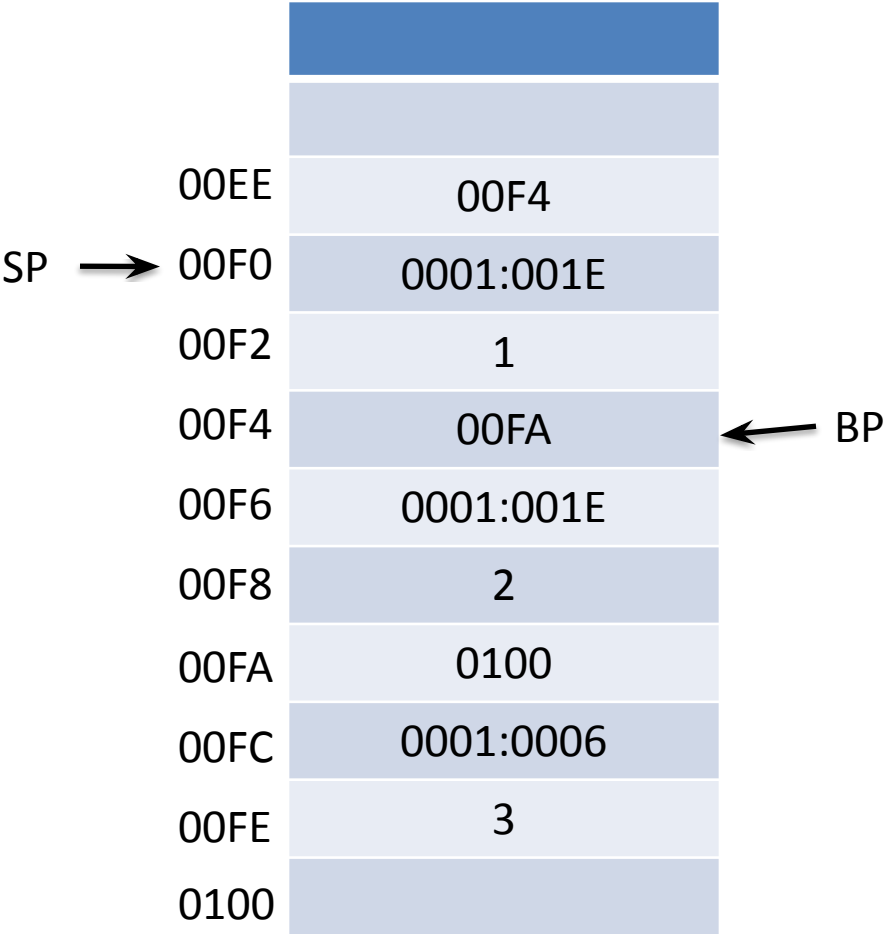
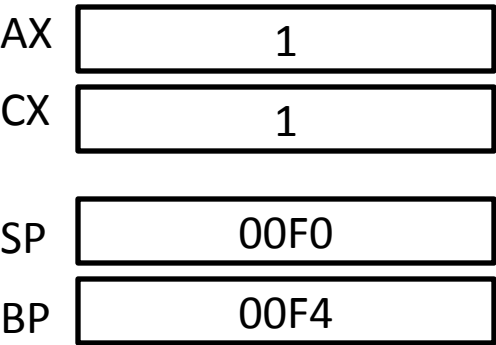
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```

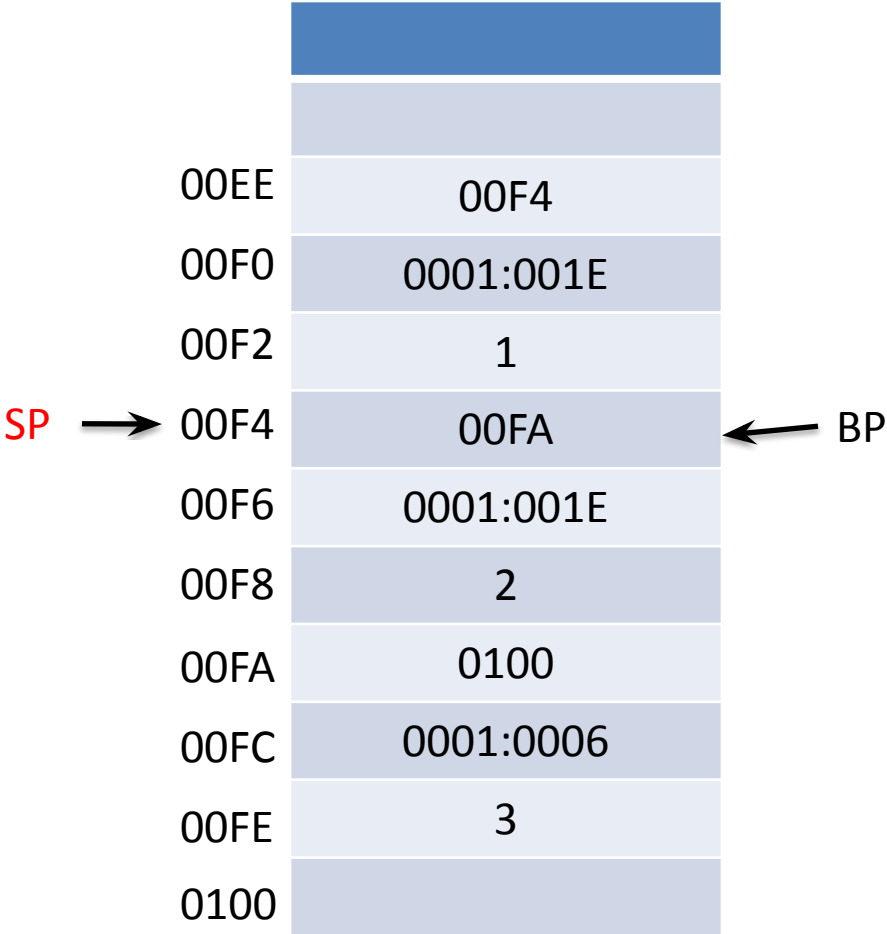
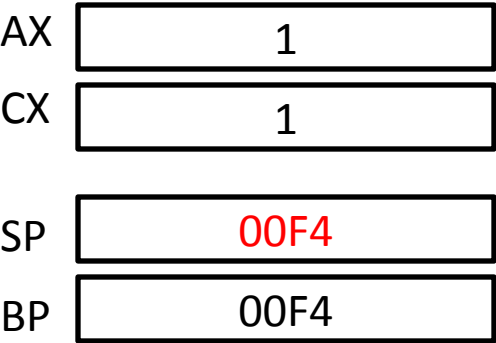


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL  WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

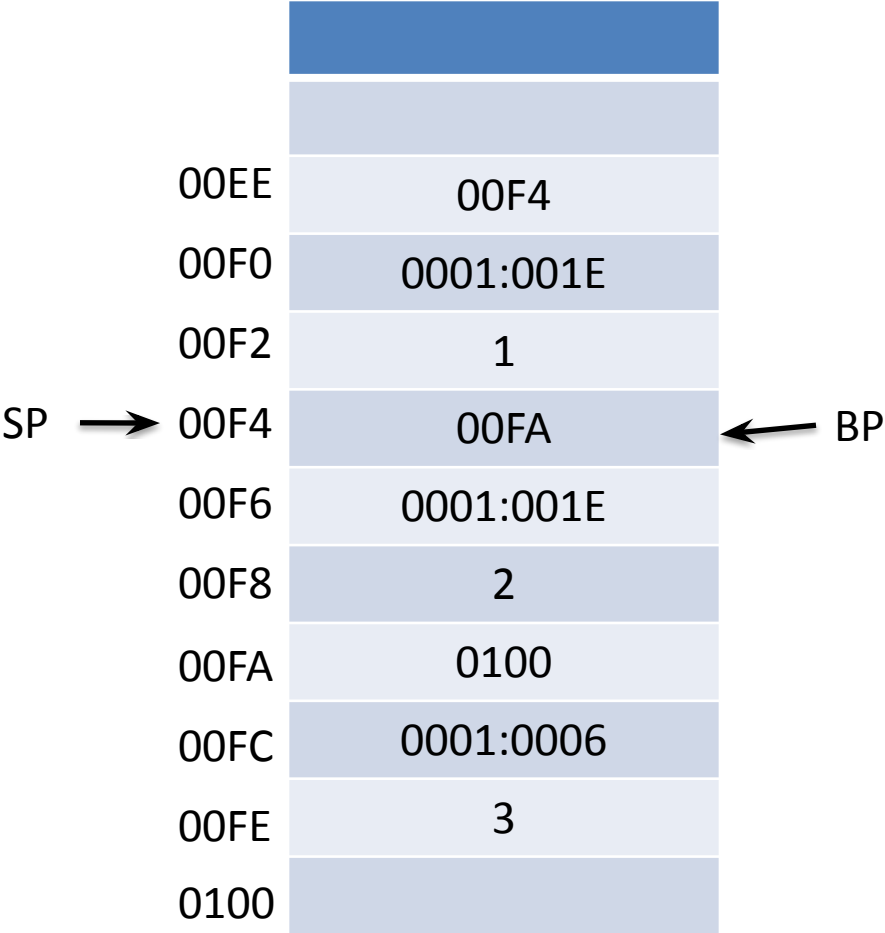
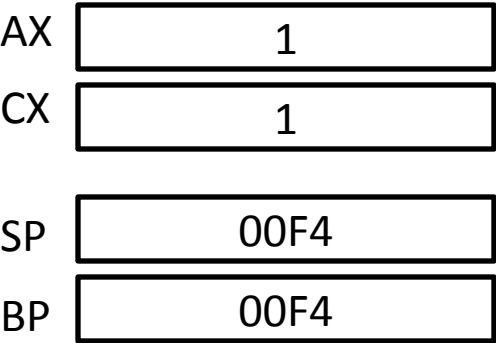


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

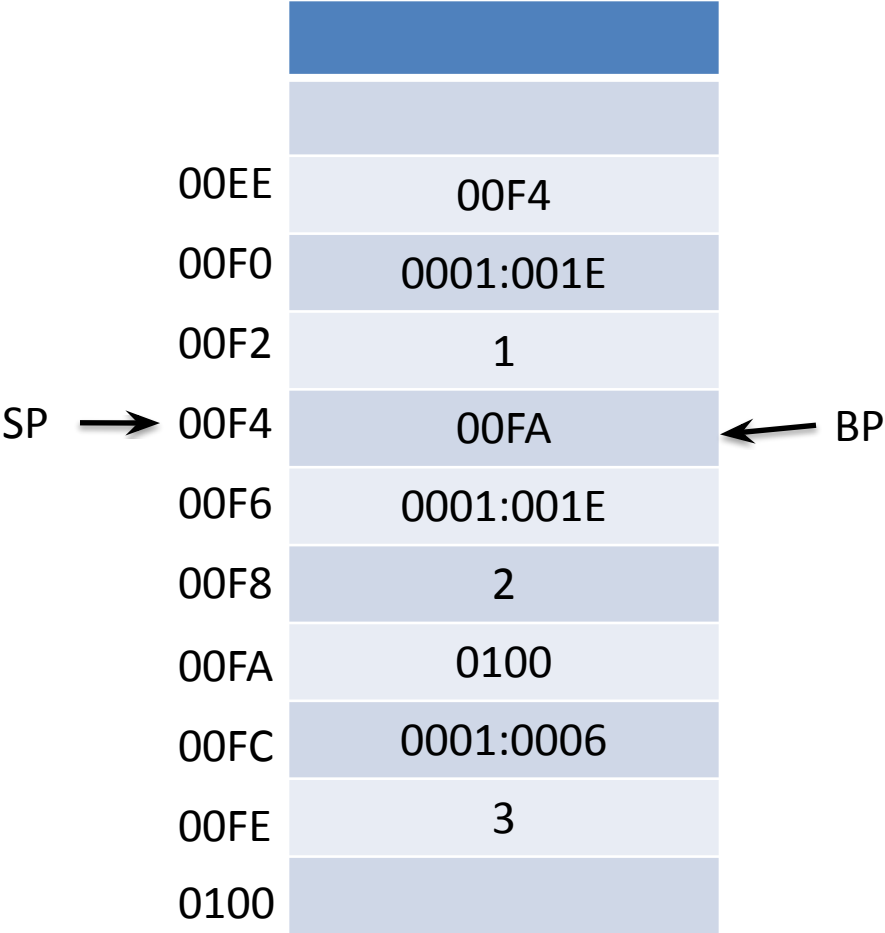
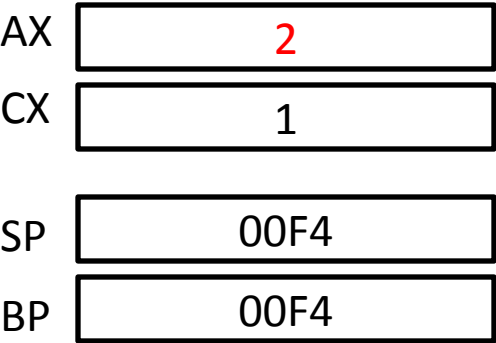


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP BP
0022      RET  2

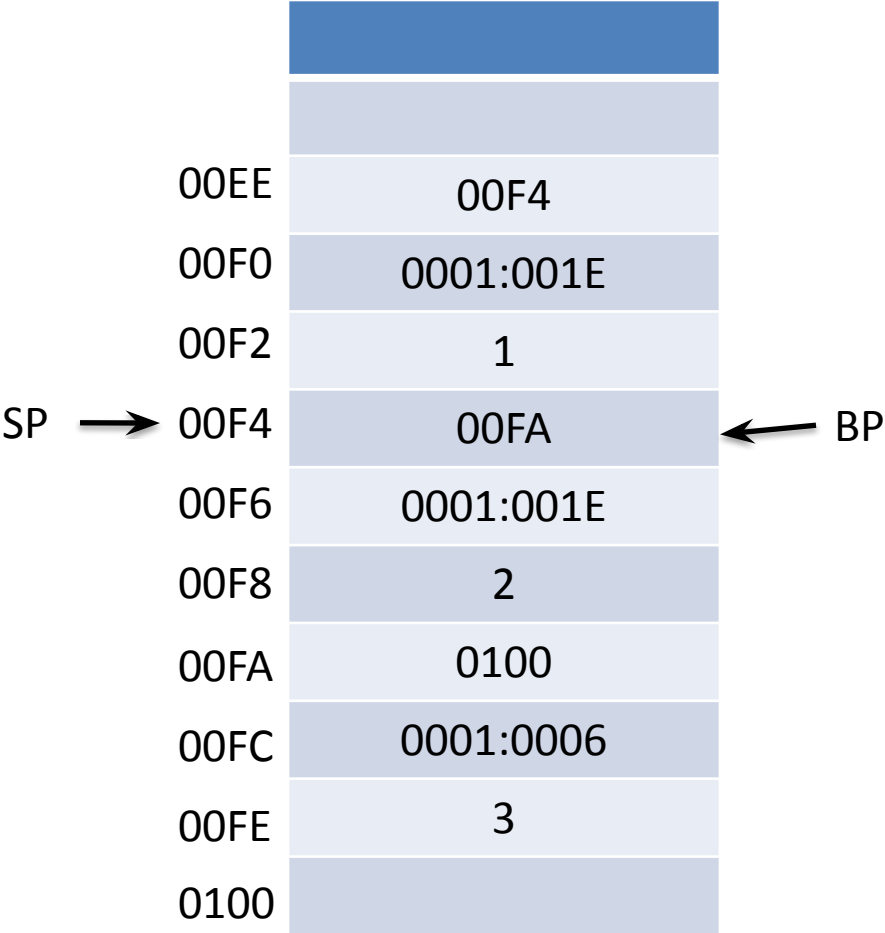
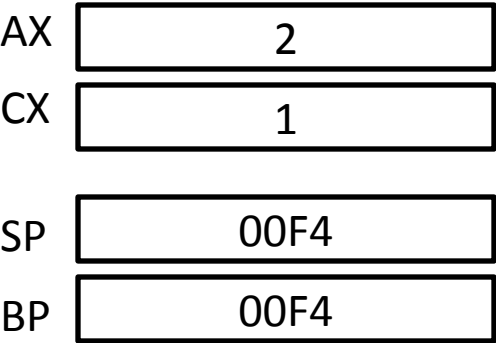
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```




```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
      END  MAIN

```

AX	2
CX	1
SP	00F6
BP	00FA

00EE	00F4	
00F0	0001:001E	
00F2	1	
00F4	00FA	
SP → 00F6	0001:001E	
00F8	2	
00FA	0100	← BP
00FC	0001:0006	
00FE	3	
0100		

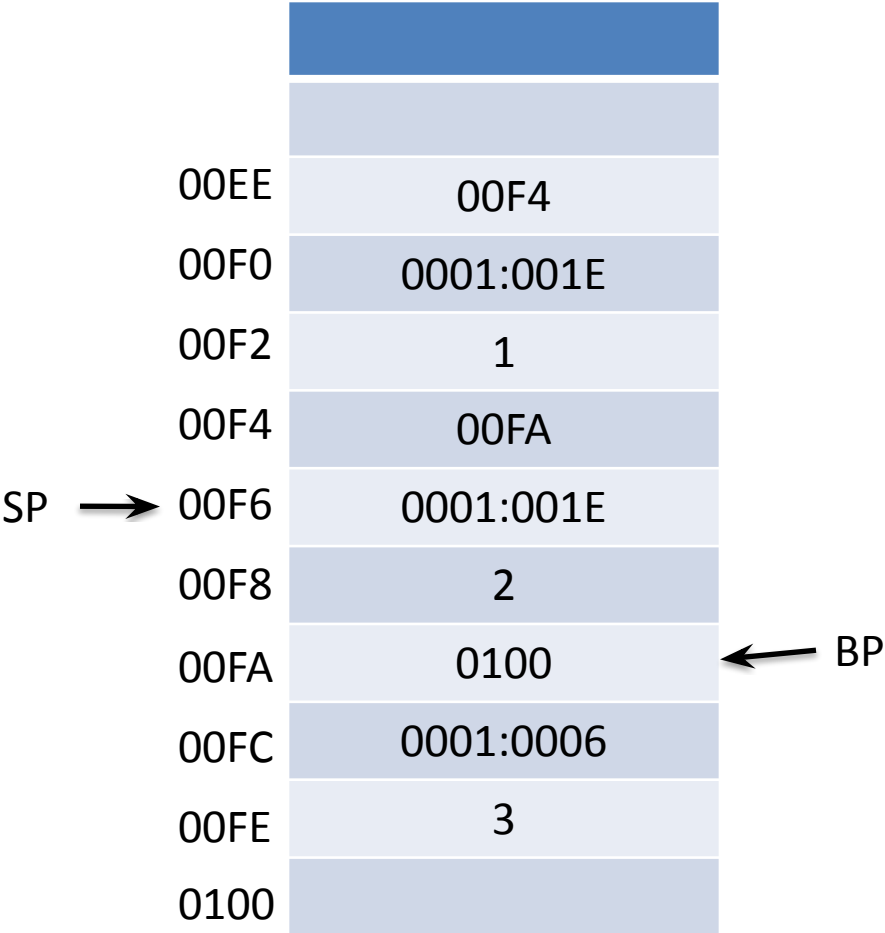
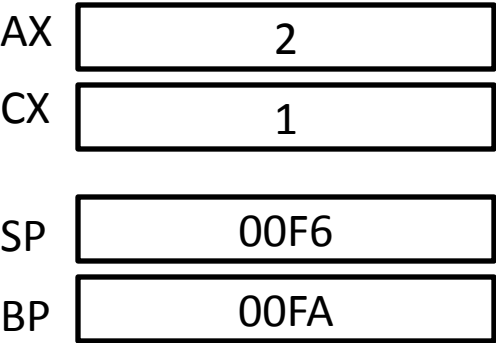
```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN

```

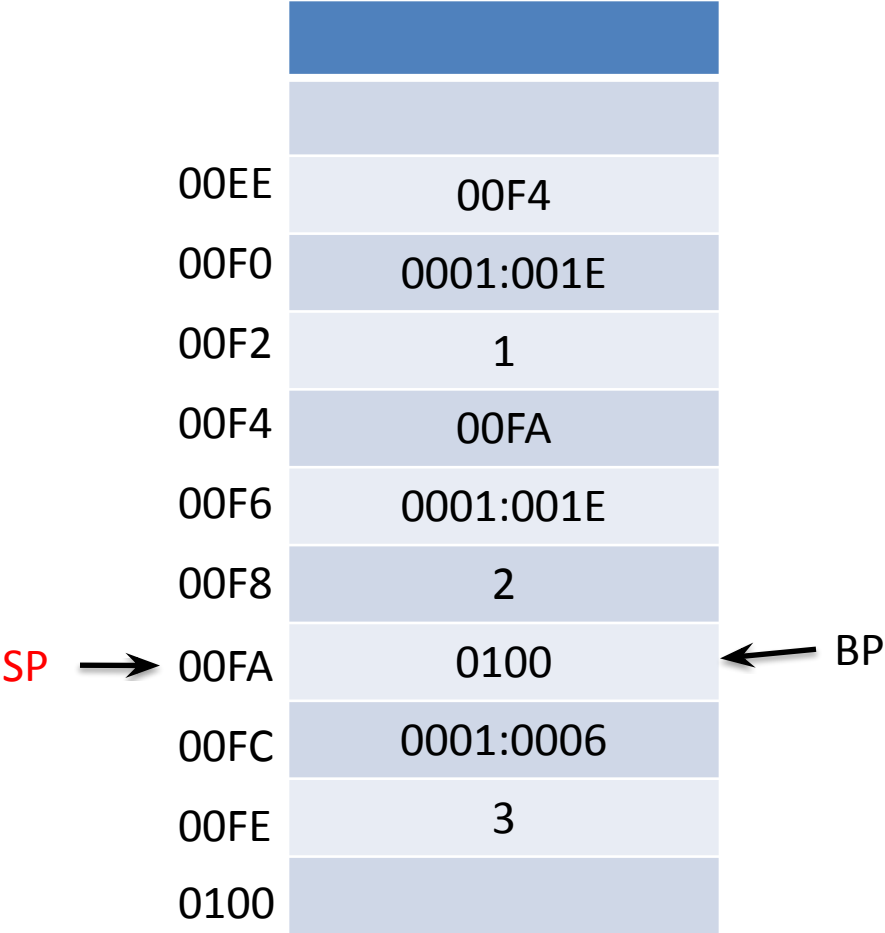
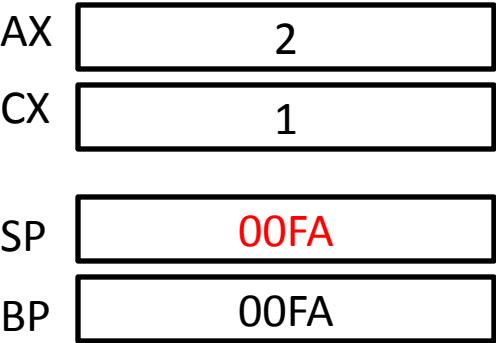


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```

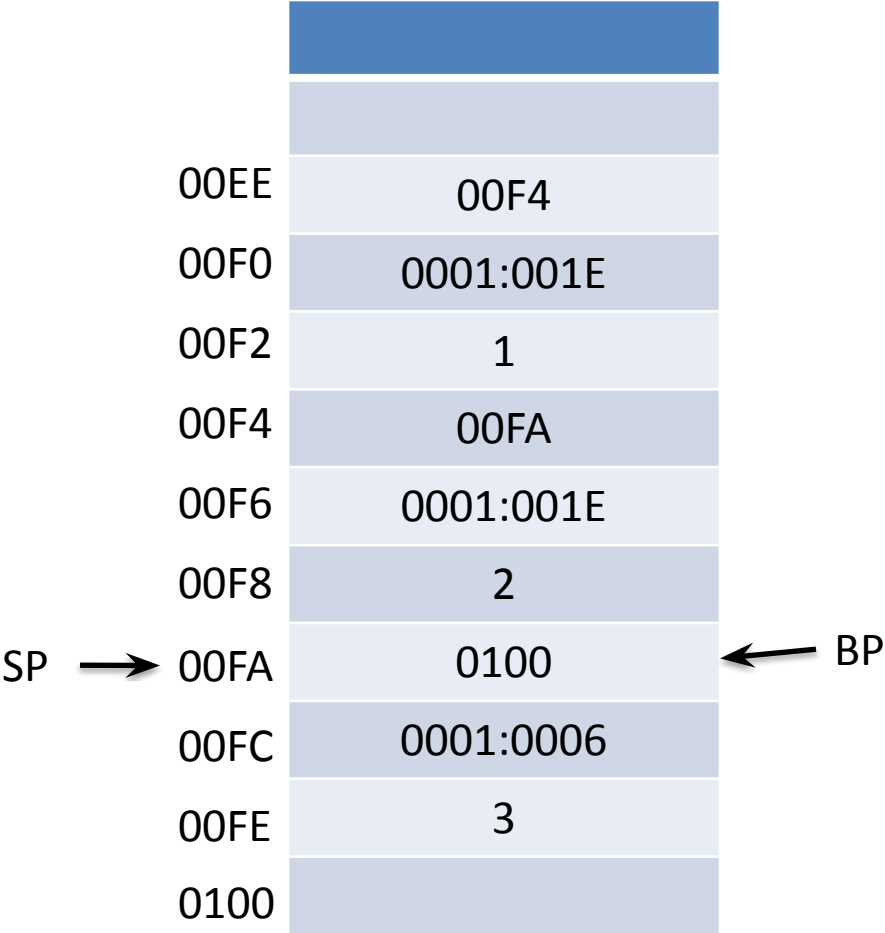
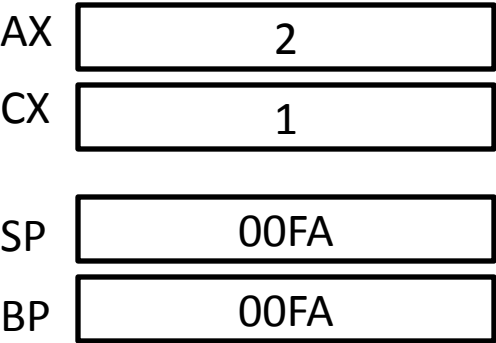


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2

END MAIN
```



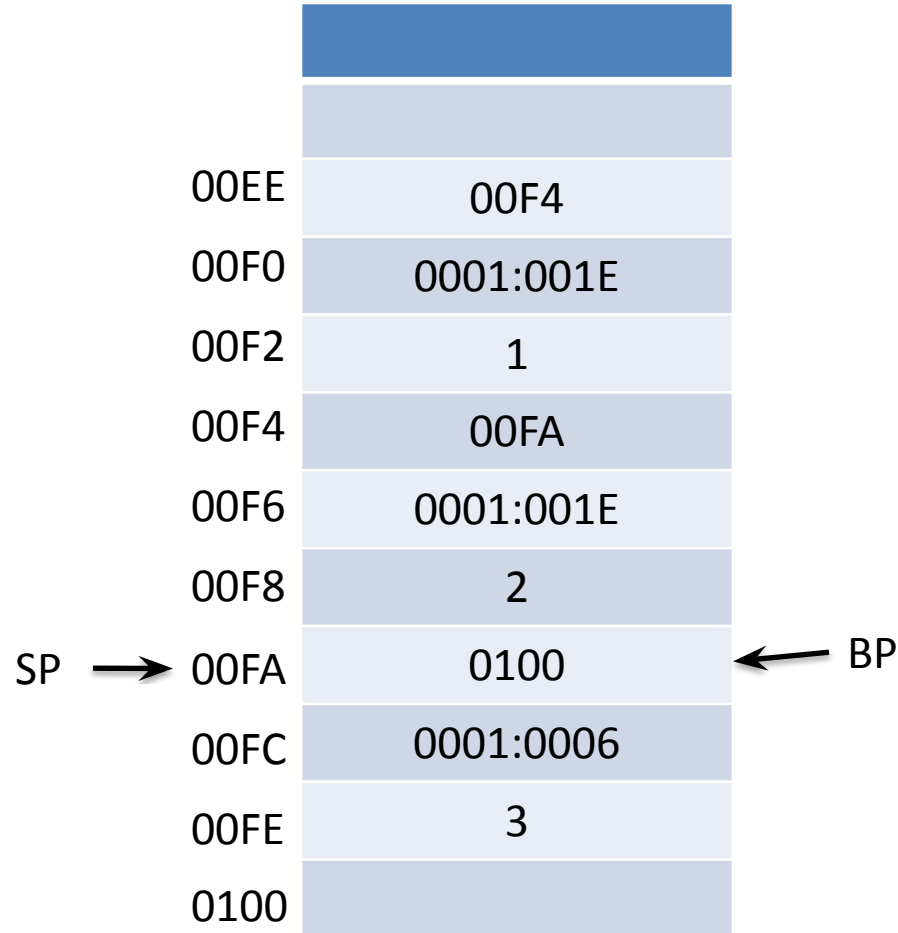
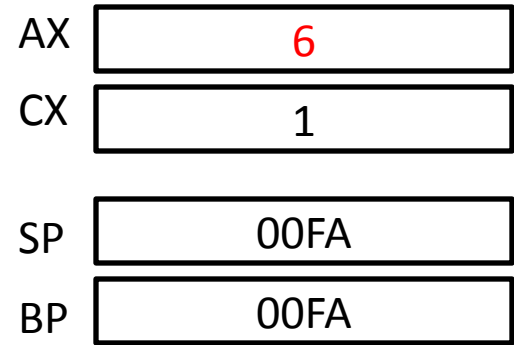
```

MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN

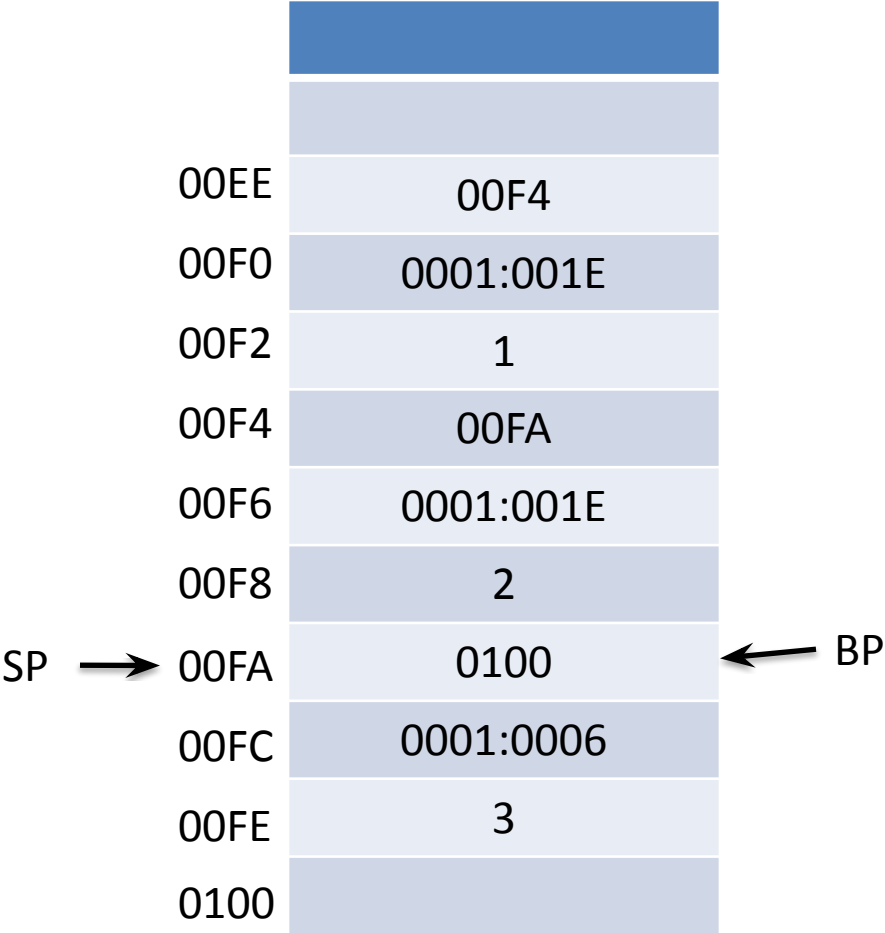
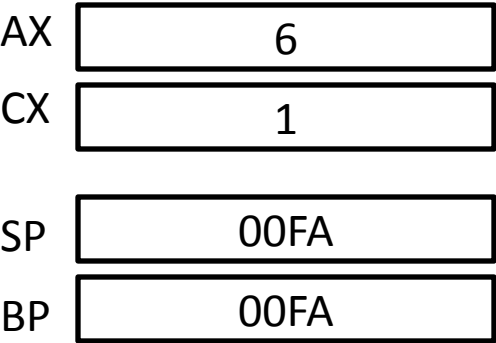
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

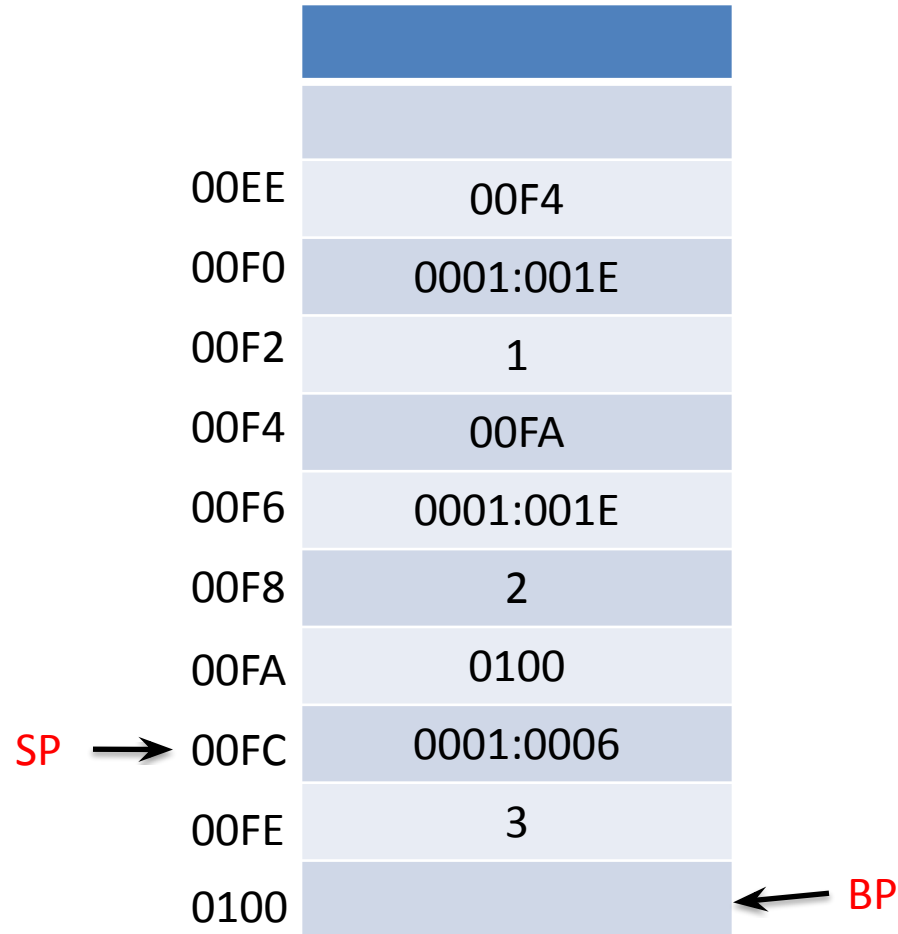
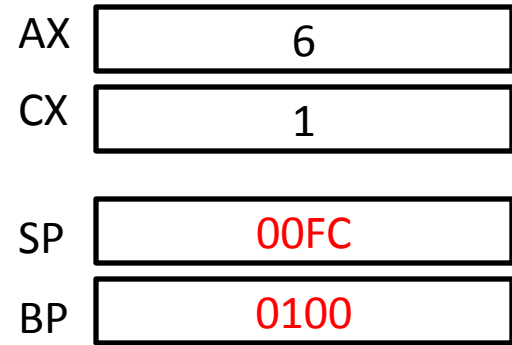
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

MAIN ENDP

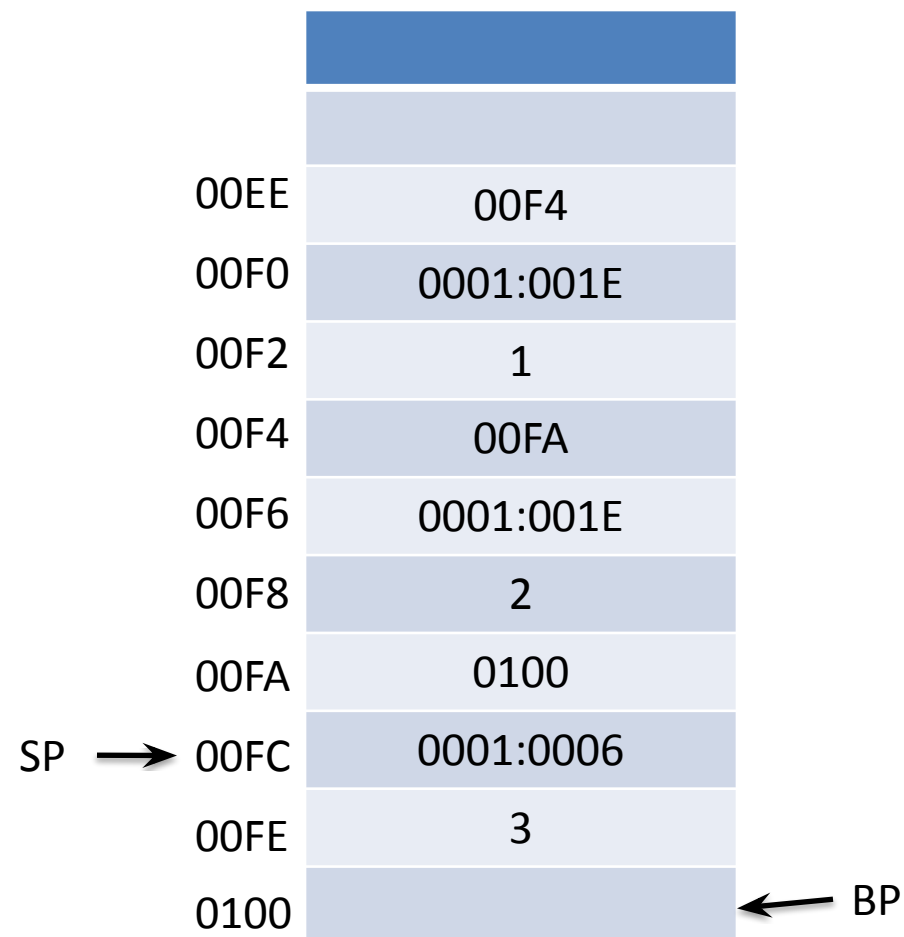
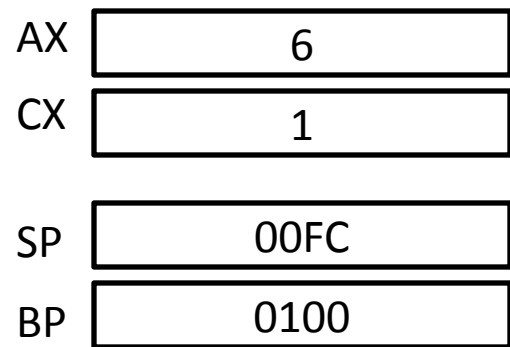
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN : POP  BP
0022      RET  2
      END  MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H

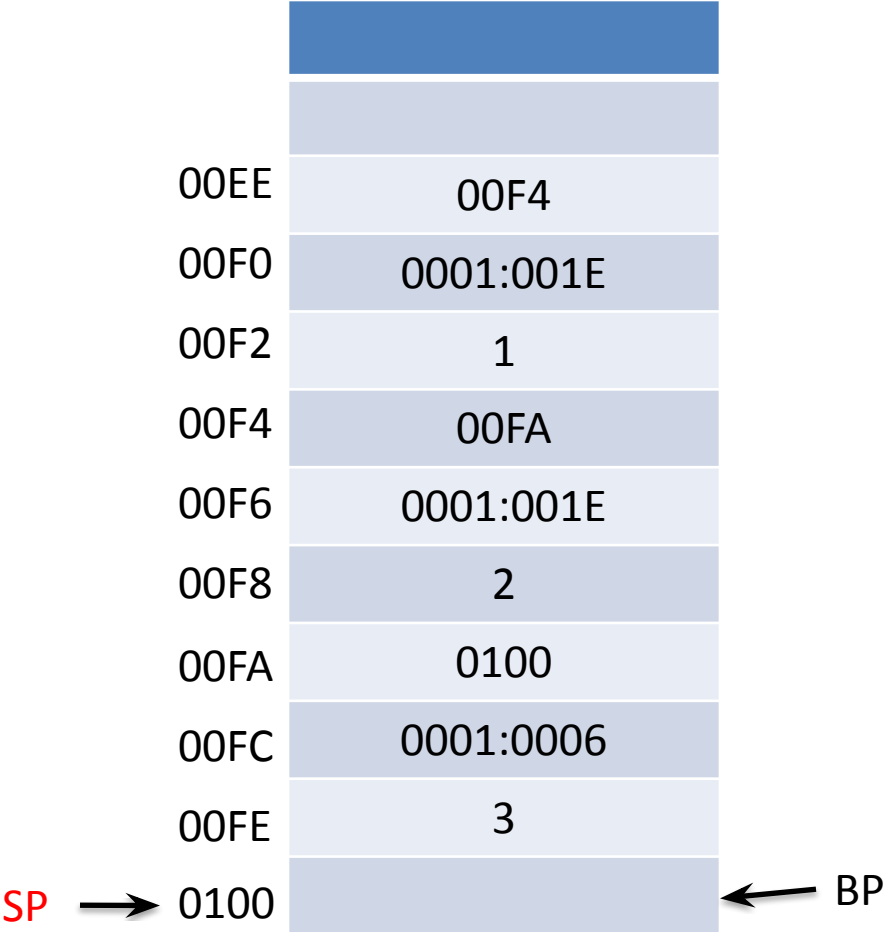
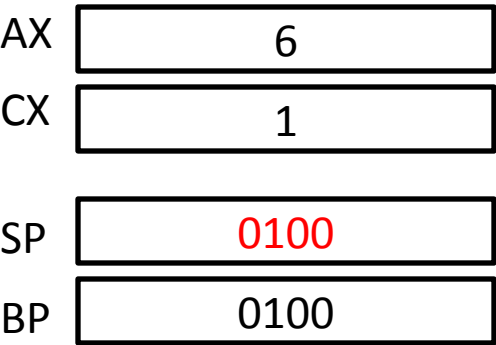
MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



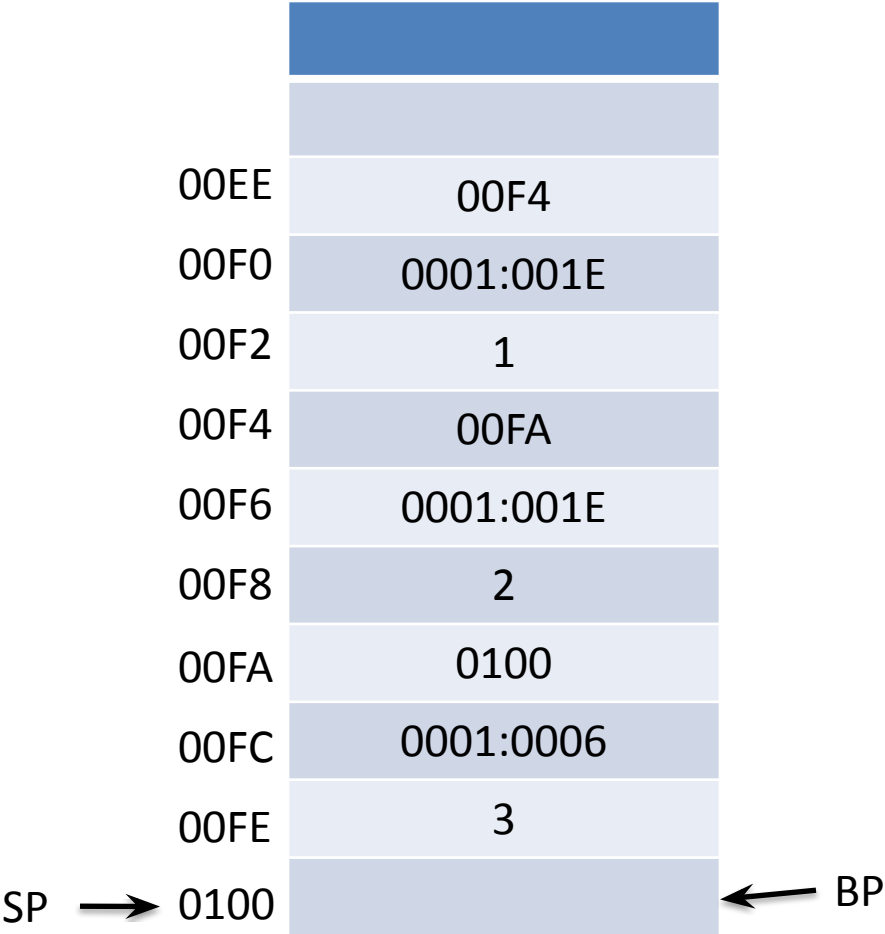
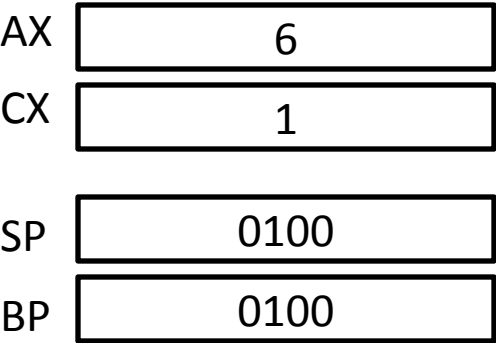

```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



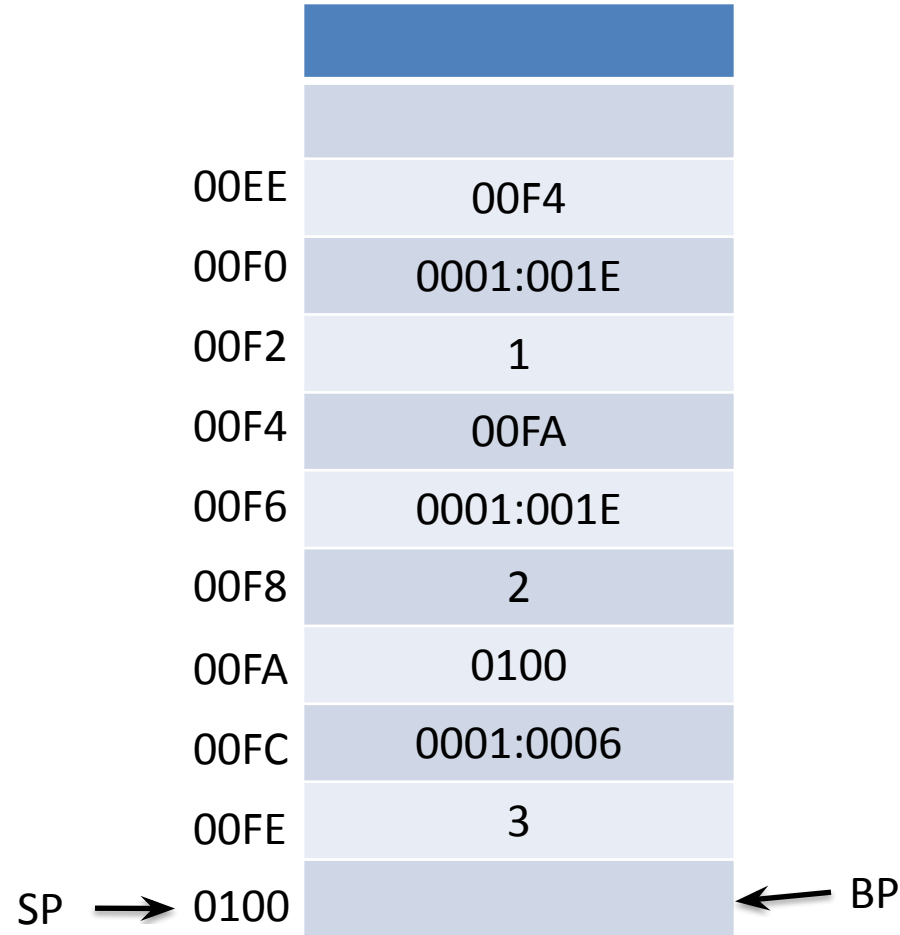
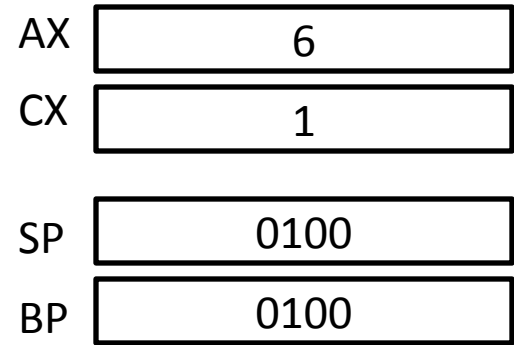
```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
MAIN ENDP

FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG   END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```

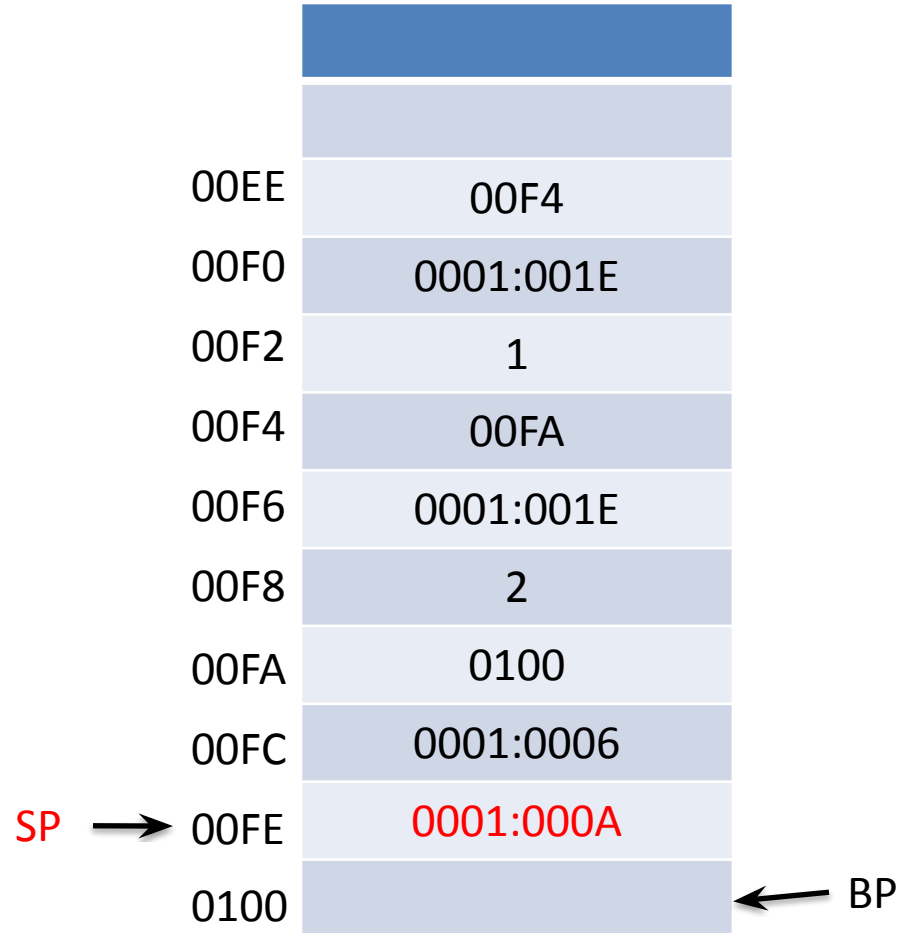
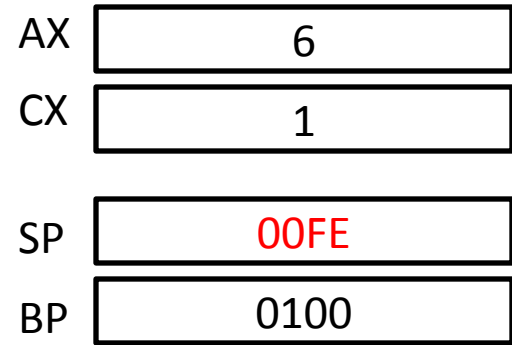


```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
```

```
MAIN ENDP
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP  RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC  CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP  BP
0022      RET  2
END MAIN
```



```
MAIN PROC
0000      MOV AX,3
0002      PUSH AX
0004      CALL FACTORIAL
0006      MOV AH,4CH
0008      INT  21H
MAIN ENDP
FACTORIAL PROC NEAR
000A      PUSH BP
000C      MOV BP,SP
000E      CMP WORD PTR[BP+4],1
0010      JG  END_IF
0012      MOV AX,1
0014      JMP RETURN
0016 END_IF:  MOV CX, [BP+4]
0018      DEC CX
001A      PUSH CX
001C      CALL FACTORIAL
001E      MUL WORD PTR[BP+4]
0020 RETURN :  POP BP
0022      RET 2
END MAIN
```



References

- Ch 8, 17 Assembly Language Programming –
by Yu and Marut

Courtesy

- All the slides on 8086 assembly language have been prepared by:

Dr. Atif Hasan Rahman

Associate Professor,

Department of CSE, BUET